

PUSH DS

MOV SI, OFFSET TEXT

CLD

CALL WRIT

RET

ENDP

XOR PUSH DS

PUSH AX

MOV

XOR AX, AX

PROC

HLODSB

PUSH AX

MOV SI, OFFSET NEXTAL, 0

CLD

JE VEGE

MOV AX, DATA

CALL WRIT

MOV AH, 14

MOV DS, AX

INT 10 H

MOV SI, OFFSET TEXT

CLD

CALL WRIT

RET

ENDP

PROC

HLODSB

CMP AL, 0

JE VEGE

MOV AH, 14

INT 10 H

PROC

HLODSB

CMP AL, 0

JE VEGE

MOV AH, 14

INT 10 H

XOR AX, AX

PUSH AX

Máté Eörs

Assembly programozás

Máté Eörs: Assembly programozás

Máté Eörs:

Assembly programozás



Írta: dr. Máté Eörs - Szegedi Egyetem

Lektorálta: dr. Makay Árpád - Szegedi Egyetem

© dr. Máté Eörs

ISBN 963 9056 27 8

Kiadja:
Novadat Bt.

Felelős kiadó:
a bt. ügyvezető igazgatója

Készült:
Text-Print Kft. - Győr Mester u. 9.

Felelős vezető:
Bozsoki Rudolf ügyvezető igazgató

TARTALOM

BEVEZETÉS.....	5
1. Assembly.....	7
1.1 Példa program	8
1.1.1 Feladatok.....	10
2. Az Intel 8086/8088 processor	11
2.1 Az Intel 8086/8088 társ szervezése	11
2.2 Az Intel 8086/8088 regiszter készlete	12
2.2.1 Szegmens regiszterek.....	12
2.2.2 Vezérlő regiszterek.....	12
2.2.3 Általános regiszterek.....	13
2.3 Memória címzés.....	14
2.3.1 Adat terület címzés.....	14
2.3.2 Stack terület címzés.....	16
2.3.3 Program terület címzés.....	16
2.4 Az utasítások szerkezete	17
2.4.1 Címzési mód byte.....	18
2.4.2 Feladatok.....	19
3. Utasítás rendszer 1. rész.....	20
3.1 Adat mozgató utasítások	20
3.1.1 Feladatok.....	21
3.2 Aritmetikai utasítások	21
3.2.1 Feladatok.....	25
3.3 Eljárás deklaráció, hívás, paraméter átadás.....	26
3.3.1 A paraméterek szabványos helyen történő átadása	29
3.3.2 A paraméterek regiszterekben történő átadása.....	31
3.3.3 A paraméterek veremben történő átadása	32
3.3.4 Lokális adat terület, rekurzív és reentrent eljárások.....	34
3.3.5 Feladatok.....	35
3.4 Vezérlés átadó utasítások	36
3.4.1 Eljárásokkal kapcsolatos utasítások	36
3.4.2 Feltétlen vezérlés átadás (ugrás)	37
3.4.3 Feltételes vezérlés átadás (ugrás)	37
3.4.4 Ciklus szervező utasítások	39
3.4.5 Feladatok.....	40
3.5 String kezelő utasítások.....	40
3.5.1 Feladatok.....	43
4. Egyszerűsített lexikális elemző	44

4.1.1	Feladatok	47
5.	Utasítás rendszer 2. rész	48
5.1	Logikai utasítások	48
5.1.1	Feladatok	48
5.2	Bit forgató (Rotate) és léptető (Shift) utasítások	49
5.2.1	Feladatok	50
5.3	Processzor vezérlő utasítások	50
5.4	Input, output (I/O) utasítások	51
5.5	Megszakítás rendszer, interrupt utasítások	51
5.5.1	Szemafor	54
5.5.2	Feladatok	56
6.	Pszedo utasítások	57
6.1	Adat definíciós utasítások	57
6.1.1	Egyszerű adat definíciós utasítások	58
6.1.2	Összetett adat definíciós utasítások	59
6.1.3	Feladatok	63
6.2	Kifejezés	63
6.2.1	Konstans	63
6.2.2	Szimbólum	64
6.2.3	Kifejezés	65
6.2.4	Feladatok	69
6.3	Feltételes fordítás	69
6.3.1	Feladatok	70
6.4	Makró és blokk ismétlés	70
6.4.1	Makró	70
6.4.2	Feladatok	80
6.4.3	Blokk ismétlés	81
6.4.4	Feladatok	83
6.5	Szegmens, szegmens csoport	83
6.5.1	Feladatok	86
6.6	Globális szimbólumok	86
6.7	INCLUDE utasítás	87
6.7.1	Feladatok	88
6.8	Lista vezérlési utasítások	88
6.8.1	Feladatok	91
7.	Assembler	92
7.1	Két menetes assembler, első menet	93
7.1.1	Feladatok	97
7.2	Két menetes assembler, második menet	97

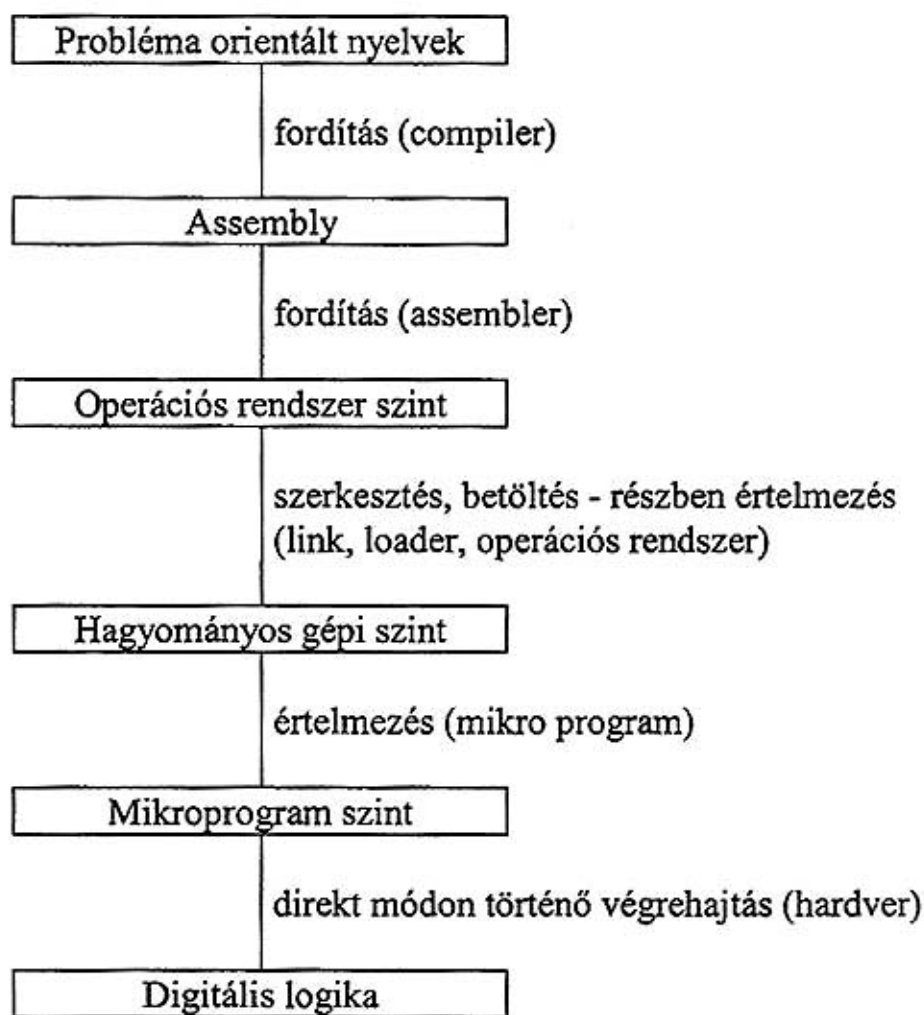
7.2.1 Feladatok	100
8. Makró generátor	101
8.1.1 Feladatok	102
9. Szerkesztő	103
9.1.1 Feladatok	105
10. Cím hozzárendelés (binding)	106
10.1 Dinamikus szerkesztés	108
10.1.1 Feladatok	110
11. Programok hangolása	111
FÜGGELÉK.....	113
Utasítás rendszer (összefoglaló táblázat)	117
Felhasznált irodalom	120
TÁRGYMUTATÓ	121

BEVEZETÉS

Egy L programozási nyelven írt program akkor futtatható, ha van olyan M valós vagy virtuális gép, amely „érti” az L nyelvet.

Legyen L_n programozási nyelv az M_n valós vagy virtuális gépen.

Virtuális gép esetén kell legyen egy valós vagy virtuális M_{n-1} gép, azon egy értelmező (interpreter) vagy egy fordító, amely értelmezni vagy fordítani tudja az L_n nyelven írt programot az M_{n-1} gépen az L_{n-1} nyelvre. Ezek alapján a programozási nyelveket az alábbiak szerint osztályozhatjuk:



Jelen tárgyalásunk során az „Assembly”-től a „Hagyományos gépi szint”-ig terjedő résszel fogunk foglalkozni.

1. ASSEMBLY

Az assembly olyan programozási nyelv, amely lehetővé teszi a hardver nyújtotta összes lehetőség kihasználását. Ennek megfelelően eltérő processzorokhoz eltérő assembly nyelvek tartoznak. Ugyanazon assembly nyelv általában csak egy gépcsaldon érvényes. Ennek ellenére a különböző assembly nyelvek több közös vonást mutatnak. Ezek közül az egyik legfontosabb a kötött írásmód: egy sorba csak egy utasítás írható, egy sor szerkezete általában:

címresz *operációs_kód* *operandusok* *kommentár*

címresz: egyes adatok illetve utasítások szimbolikus megjelölésére szolgál.

operációs_kód: mnemonic, a utasítás, művelet megnevezésére szolgál.

operandusok: itt adhatjuk meg az utasítás paramétereit.

kommentár: A program jobb olvashatóságát teszi lehetővé, de a program működésére nincs hatása.

Az assembler-ek általában alapértelmezésben nem, csak megfelelően beállított opció hatására különböztetik meg a kis és nagy betűket, a példa programjainkban a nagy betűket kiemelésre fogjuk használni.

Sajnos a különböző assembly nyelvek ott is eltérést mutatnak, ahol ez nem lenne feltétlenül indokolt. Példaként álljon itt egy – két elterjedt processzorra készült – apró programrészlet:

Intel 80386:

```
FORMUL:      MOV      EAX, I ; a 4 byte-os I kerül EAX-be
...
I            DD      2      ; I 4 byte-on, inicializálva 2-vel
```

Motorola 68030:

```
FORMUL:      MOVE.L I, D0 ; a 4 byte-os I kerül D0-ba
...
I:           DC.L    2      ; I 4 byte-on, inicializálva 2-vel
```


Mindkét esetben **FORMUL:** címke, amely lehetővé teszi, hogy erre az utasításra hivatkozassunk. A **MOV** és a **MOVE** operációs kód adat átvitelt ír elő. A **MOV** a második operandus tartalmát tölti az elsőbe, míg a **MOVE** az első a másodikba. Ez a különbség a két processzor eltérő felépítésével magyarázható. Mindkét utasítás 4 byte-ot mozgat, de míg a **MOV** az operandusokból állapítja meg az adat hosszát, addig a **MOVE** a **.L** kiterjesztésből. **EAX** illetve **DO** a két processzor egy-egy regiszterének megnevezése. A **;** a kommentár bevezető karaktere. A **DD** és a **DC.L** egyaránt 4 byte-os egész megadását teszi lehetővé.

Látható, hogy az eltérések csak részben vezethetők vissza a processzorok eltérésére, a további eltérések csak a stílusok különbözőségével magyarázhatók.

A továbbiakban az Intel 8086/8088-ra írt Microsoft assembly-vel fogunk foglalkozni.

1.1 Példa program

A részletes tárgyalás előtt bemutatunk egy szöveget kiíró programot, amely segítségével szolgálhat az olvasó első programjainak megírásához.

```
; Assembly főprogram, amely adott szöveget ír a képernyőre
; =====
KOD          SEGMENT      PARA PUBLIC 'CODE' ; Szegmens kezdet
; KOD: a szegmens neve
; align-type (igazítás típusa), lehet: BYTE, WORD, PARA, PAGE
; combine-type: PUBLIC, COMMON, AT <Kifejezés>, STACK
; class: 'CODE', 'DATA', ('CONSTANT',) 'STACK', 'MEMORY' ajánlott
; értelemszerűen

                ASSUME CS:KOD, DS:ADAT, SS:VEREM, ES:NOTHING
; feltételezett szegmens regiszter értékek. A beállításról ez
; az utasítás nem gondoskodik

KIIR          PROC FAR    ; A fő eljárás mindig FAR
                                ; FAR: távoli, NEAR: közeli eljárás
; Az operációs rendszer úgy hívja meg a főprogramokat, hogy a
; CS és IP a program végén lévő END utasításban megadott
; címke szegmens és OFFSET címét tartalmazza, SS és SP a
; a STACK kombinációs típusú szegmens végét mutatja,
; a visszatérés szegmens címe DS-ben van, OFFSET-je pedig 0
```

```

        PUSH DS      ; DS-ben van a visszatérési cím
                        ; SEGMENT része
        XOR  AX, AX  ; AX←0, az OFFSET rész = 0
        PUSH AX      ; A veremben a visszatérési cím (FAR)
        MOV  AX, ADAT ; AX← az ADAT SEGMENT címe
        MOV  DS, AX
; Most teljesült, amit az ASSUME utasításban írtunk
; Eddig tartott a főprogram előkészületi része

        MOV  SI, OFFSET SZOVEG ; SI←SZÖVEG OFFSET címe
        CLD                      ; ld. később
        CALL KIIRO ; Eljárás hívás
        RET                      ; Visszatérés az op. rendszerhez
                        ; a veremből visszaolvasott
                        ; szegmens és OFFSET címre
KIIIR    ENDP                ; A KIIIR eljárás vége

KIIRO    PROC                ; NEAR eljárás,
                        ; a NEAR megadása nem kötelező

CIKLUS:  LODSB                ; AL←a következő karakter
        CMP  AL, 0 ; AL=? 0
        JE   VEGE ; ugrás a VÉGE címkéhez, ha AL=0
        MOV  AH, 14 ; BIOS rutin paraméterezése
        INT  10H ; a 10-es interrupt hívása: az AL-ben
                        ; lévő karaktert kiírja a képernyőre
        JMP  CIKLUS ; ugrás a CIKLUS címkéhez,
                        ; a kiírás folytatása
VEGE:    RET                  ; Visszatérés a hívó programhoz
KIIRO    ENDP                ; A KIÍRÓ eljárás vége
KOD      ENDS                ; A KÓD szegmens vége
; =====

ADAT      SEGMENT      PARA PUBLIC 'DATA'
SZOVEG    DB      'Ezt a szöveget kiírja a képernyőre'
          DB      13, 10, 0 ; 13: a kocsi vissza,
                        ; 10: a soremelés kódja,
                        ; 0: a szöveg vége jel
ADAT      ENDS            ; Az ADAT szegmens vége
;
VEREM     SEGMENT      PARA STACK
          DW      100 DUP (?) ; Helyfoglalás 100
                        ; inicializálatlan szó számára
VEREM     ENDS            ; A VEREM szegmens vége
;
          END  KIIIR ; Modul vége,
                        ; a program kezdőcíme: KIIIR

```

1.1.1 Feladatok

1. Módosítsuk az előző programban SZOVEG -et, hogy az alábbi verset írja ki:

Ravasz kelepcek s török közt futunk,
tán mennybe, tán pokolba száll utunk,
Te vén Varázsló, Te rendelted így,
és a Te bűnöd lesz, ha elbukunk!

Omár Khájjám (Szabó Lőrinc fordítása)

2. Írjuk ki a fenti verset a KIÍRO többszöri felhívásával.

2. AZ INTEL 8086/8088 PROCESSOR

2.1 Az Intel 8086/8088 társzervezése

A memória byte szervezésű. Egy byte 8 bitből áll. Bármely két szomszédos byte egy szót (word), bármely négy egy dupla szót (double word) alkot. Szó, dupla szó a legalacsonyabb című byte-jával címezhető. Szóban és dupla szóban az alacsonyabb címen az alacsonyabb helyértékű byte helyezkedik el: LSBfirst (Least Significant Byte first) tárolási mód, szemben a MSBfirst (Most Significant Byte first) tárolási móddal. A negatív számok 2-es komplementum kódban vannak ábrázolva (egy negatív szám ábrázolását úgy kaphatjuk meg, hogy a szám abszolút értékének minden bit-jét ellenkezőjére váltjuk, majd a legalacsonyabb helyértéken 1-et hozzáadunk. Könnyű belátni, hogy – ha eltekintünk a lehetséges túlcsoordulástól, akkor – összeadáskor nem kell tekintettel lennünk az összeadandók előjelére.

A memória terület címzése két szinten történik. A szegmens (összefüggő memória terület) kezdő címét nevezzük szegmens címnek, a szegmensen belüli „relatív” címet logikai címnek, virtuális címnek (más elnevezései: OFFSET, displacement, eltolás, Effective Address, rövidítve EA). A kettőből alakul ki a fizikai cím (Address). Megjegyezzük, hogy az OFFSET és az EA nem mindig ugyanazt jelenti (*Lásd* GROUP utasítás), továbbá azt, hogy pl. Motorola processzorokon alapuló rendszerekben az Effective Address nem a logikai, hanem a fizikai címet jelenti.

Az Intel 8086/8088 regiszterei 16 bitesek. A fizikai cím kiszámítását úgy valósították meg, hogy a szegmens cím mindig osztható 16-tal (paragrafus határ). A szegmens regiszter a szegmens kezdő paragrafusának a sorszámát tartalmazza (valós üzemmód). A fizikai cím úgy számítható ki, hogy a megfelelő szegmens regiszter tartalmát 16-tal megszorozzuk (hexadecimálisan egy 0-t írunk utána) és hozzáadjuk a logikai címet. Az eredmény alsó 20 bitjén kapott érték a fizikai cím függetlenül attól, hogy az így kapott érték elfér-e biten ($\geq 2^{20}$) vagy sem.

Megjegyzés: A fejlettebb processzorok valós és védett (protected) üzemmódban is képesek dolgozni. Védett üzemmódban a szegmens

regiszter a page táblázat egy elemére mutat, és ez az elem határozza meg a szegmens kezdőcímét. A védett üzemmódnak elsősorban nagy méretű programok, vagy nagy adat területet használó programok írásakor van előnye. Céljainkhoz elegendő a valós üzemmóddal megismerkednünk.

2.2 Az Intel 8086/8088 regiszter készlete

2.2.1 Szegmens regiszterek

16 bites regiszterek.

CS	(Code Segment) utasítások címezéséhez
SS	(Stack Segment) verem címezéséhez
DS	(Data Segment) (automatikus) adat terület címezéséhez
ES	(Extra Segment) másodlagos adat terület címezéséhez

2.2.2 Vezérlő regiszterek

16 bites regiszterek

IP	(Instruction Pointer) az éppen végrehajtandó utasítás logikai címét tartalmazza a cs által mutatott szegmensben
SP	(Stack Pointer) a stack-be (verembe) utolsónak beírt elem logika címét tartalmazza az ss által mutatott szegmensben
STATUS	(STATUS, SR vagy FLAGS) a processzor állapotát jelző regiszter
BP	(Base Pointer) a stack indexelt címezéséhez használatos
SI	(Source Index) a kiindulási adat terület indexelt címezéséhez használatos
DI	(Destination Index) a cél adat terület indexelt címezéséhez használatos

STATUS bitjei (flag-jei, ez alapján FLAGS-nek is nevezik):

-	-	-	-	O	D	I	T	S	Z	-	A	-	P	-	C
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Néha a flag betűje után egy F-et is írnak a jobb olvashatóság érdekében.

O	Overflow	előjeles túlsordulás
D	Direction	a string műveletek iránya, 0: növekvő, 1: csökkenő
I	Interrupt	1: megszakítás engedélyezése (enable), 0: tiltása (disable)
T	Trap	1: „single step”, 0 automatikus üzemmód
S	Sign	az eredmény legmagasabb helyértékű bit-je (előjel bit)
Z	Zero	1, ha az eredmény 0, különben 0
A	Auxiliary Carry	átvitel a 3. és 4. bit között (decimális aritmetika)
P	Parity	az eredmény alsó 8 bitjének paritása
C	Carry	átvitel előjel nélküli műveleteknél

A flag-ek közül az O, S, Z, A, P, C az elvégzett művelet eredménye szerint áll be (A kivételével ezeket a flag-eket aritmetikai flag-eknek szokás nevezni). Ilyen értelemben az adat mozgó utasítások nem szolgáltatnak „eredményt”, ezek nem módosítják a fenti flag-eket.

A D és I flag csak külön erre a célra szolgáló utasítással módosítható, a T flag pedig csak a POPF utasítással, ami normálisan a STATUS korábban PUSHF utasítással elmentett állapotának visszaállítására szolgál.

Az A flag állapota a legtöbb utasítás végrehajtása után definiálatlan, a többi flag értéke általában „logikusan” alakul, az ettől eltérő esetekre külön felhívjuk a figyelmet. Az Utasítás rendszer (összefoglaló táblázat) című részben minden utasítás mellett feltüntettük a flag-ek viselkedését is.

2.2.3 Általános regiszterek

16 és 8 bites regiszterek (szavak esetén a higher byte a magasabb, lower byte az alacsonyabb helyértékű byte-ot jelenti).

word	higher byte	lower byte	
AX	AH	AL	Accumulator (szorzás, osztás is)
BX	BH	BL	Base Register (címző regiszter)
CX	CH	CL	Counter Register (számláló regiszter)
DX	DH	DL	Data Register (szorzás, osztás, I/O)

2.3 Memória címzés

2.3.1 Adat terület címzés

Az adat címzése esetén – ha nem adunk meg explicit módon szegmens regisztert – akkor a DS (automatikus) szegmens regiszter vesz részt a fizikai cím kiszámításában. Ez alól egyedüli kivétel a kódba épített adat.

Kódba épített adat (immediate – közvetlen operandus megadás): az adat a kód területen, az utasítás részeként kerül elhelyezésre, pl.:

```
MOV    AL, 6           ; AL új tartalma 6
MOV    AX, 0FFH        ; AX új tartalma hexadecimális FF
```

Direkt memória címzés: a címrészen az operandus logikai címe (eltolás, displacement) szerepel. Pl.:

```
MOV    AX, SZO          ; AX új tartalma SZO tartalma
MOV    AL, KAR           ; AL új tartalma KAR tartalma
```

Valahol a DS által mutatott szegmensben:

```
SZO    DW    1375H
KAR     DB    3FH
```

A művelethez a DS regiszter által mutatott memória szegmensben belül a SZO ill. KAR azonosítóval megnevezett word-öt ill. byte-ot fogja használni az utasítás – ezt a címet röviden (DS:SZO) ill. (DS:KAR) formában szokás megadni. Ügyelnünk kell arra, hogy az ASSUME utasítás és az alkalmazott (automatikus) szegmens regiszter (DS) tartalma összhangban legyen, különben hibásan fog működni a programunk.

Ugyancsak hibás a következő két utasítás:

```
MOV    AX, KAR          ; hibás
MOV    AL, SZO          ; hibás
```

mert az utasítás operandusainak típusa meg kell egyezzen, de helyes:

```
MOV    AX, WORD PTR KAR
MOV    AL, BYTE PTR SZO
```

mert WORD PTR ill. BYTE PTR hatására a fordító word-nek ill. byte-nak fogja tekinteni a KAR ill. a SZO -nál elhelyezett memória tartalmat. A típus egyezés követelménye természetesen a többi címzési módnál is érvényes.

Indexelt címzés: Az operandusban lévő 8 vagy 16 bites számot (displacement – eltolás) hozzáadja SI vagy DI tartalmához, ez lesz az operandus logikai címe. Pl.:

```
MOV    AX, [SI]
MOV    AX, 10H[SI]
MOV    AX, -10H[SI]
```

A 8 biten megadott eltolás, displacement érték előjel helyesen 16 bitesre bővül a cím kiszámításakor. Fontos tudni, hogy a logikai cím kiszámolása során a 16. biten esetlegesen keletkező túlcsoordulás elvész.

Megjegyezzük, hogy BX is használható hasonló módon, de akkor inkább bázis relatív címzésről beszélünk.

Regiszter indirekt címzés: Az indexelt címzés azon speciális esete, amikor nem adunk meg displacement-et. Ennél a címzési módnál az SI, DI és a BX regiszter használható. Pl.:

```
MOV    AX, [BX]
MOV    AX, [SI]
```

Bázis relatív címzés: A legösszetettebb adat címzési mód. A logikai cím meghatározásához a BX és az SI vagy DI regiszter valamelyikét is felhasználjuk. Pl.:

```
MOV    AX, 10H[BX][SI]
```

vagy másképp írva:

```
MOV    AX, [BX+SI+10H]
```

A logikai cím ilyenkor a displacement és a két regiszter tartalmának összege.

2.3.2 Stack terület címzés

A stack (verem) terület címzése stack relatív címzéssel valósul meg, ami megegyezik a bázis relatív címzéssel, csak a BX regiszter helyett a BP szerepel. Ilyenkor a fizikai cím meghatározásához a DS helyett természetesen SS az automatikus szegmens regiszter.

2.3.3 Program terület címzés

Egy-egy utasítás végrehajtása közben IP tartalma az utasítás hosszával (byte-jainak számával) növekszik, ezáltal a memóriában következő utasításra fog mutatni. Ha nem ezzel, hanem másik utasítással szeretnénk folytatni a programot, akkor vezérlés átadó utasítást kell alkalmaznunk. Vezérlés átadó utasítással a program területet címezzük. Program terület címzésénél CS az automatikus szegmens regiszter.

IP relatív címzés: Az IP (már módosult) pillanatnyi értékéhez hozzáadódik a 8 bites előjeles közvetlen operandus. A programban operandusként címkét adunk meg, az assembler az utasítás és a címke előjeles távolságának meghatározásával gondoskodik a 8 bites érték kiszámításáról. Fontos tudni, hogy a feltételes vezérlés átadás és a ciklus utasítás mindig ilyen címzési móddal valósul meg, tehát ezekkel az utasításokkal nem tudunk messzire ugrani.

Direkt utasítás címzés: Az operandus annak az utasításnak a címe, ahova a vezérlést átadni kívánjuk. A vezérlés átadás lehet közeli (NEAR) és távoli (FAR). Közeli a vezérlés átadás, ha nem történik közben szegmens váltás. Ilyenkor a 16 bites operandus lesz az IP új tartalma. Távoli vezérlés átadáskor IP és CS is megváltozik (megváltozhat), ekkor az operandus 32 bites. Pl.:

CALL VALAMI

közeli vagy távoli eljárás hívás attól függően, hogy a VALAMI eljárást NEAR-nek vagy FAR-nak minősítettük (tehát függetlenül attól, hogy a hívott eljárás másik szegmensben van-e). Ha másik szegmensben van, akkor természetesen csak abban az esetben érhető el, ha FAR-ként definiáltuk.

Szerencsére az assembler jelzi, ha egy eljárás nem érhető el, így szükség esetén módunkban áll NEAR eljárást FAR eljárássá átminősíteni.

Indirekt utasítás címzés: Bármilyen adat címzési móddal megadott szóban vagy dupla szóban tárolt címre történő vezérlés átadás. Pl.:

```
JMP    AX      ; ugrás az AX-ben tárolt címre
JMP    [BX]    ; ugrás a (DS:BX) által címzett szóban
                ; tárolt címre.
```

2.4 Az utasítások szerkezete

<i>prefixum</i>	<i>operációs kód</i>	<i>címzési mód</i>	<i>operandus</i>
0 - 2 byte	1 byte	0 - 1 byte	0 - 4 byte

Prefixum: utasítás ismétlés (*Lásd* string kezelő utasítások), explicit szegmens megadás vagy LOCK (*Lásd* szemafor). Az explicit szegmens megadás a használni kívánt szegmens regiszter után írt kettősponttal történik, pl.:

```
MOV    AX, CS:S    ; S nem a DS,
                    ; hanem a CS regiszterrel címezendő
```

Operációs kód: ez mondja meg, hogy mit kell csinálni. Az operációs kód szimbolikus alakját mnemonic-nak nevezzük. Általában operációs, műveleti vagy utasítás kódon is a mnemonic-ot fogjuk érteni, de megjegyezzük, hogy ugyanazon mnemonic-hoz különböző gépi utasítás kódok is tartozhatnak.

Címzési mód byte: hogyan kell az operandust értelmezni.

Operandus: mivel kell a műveletet elvégezni. Az operandus szót egyes számban használtuk, mert a két operandus közül az egyik mindig regiszter és ezt vagy a gépi műveleti kód vagy a címzési mód byte határozza meg.

2.4.1 Címzési mód byte

A legtöbb utasítás kód után szerepel. Szerkezete:

7	6	5	4	3	2	1	0
Mód		Regiszter			Reg/Mem		

A következő táblázat segít a címzési mód byte értelmezésében.

Regiszter			Reg/Mem jelentése, ha Mód =			
	byte	word	00	01	10	11
000	AL	AX	BX + SI	"00"	"00"	Regiszter
001	CL	CX	+ DI	+	+	
010	DL	DX	BP + SI	8	16	
011	BL	BX	+ DI	bit	bit	
100	AH	SP	SI	disp.	disp.	
101	CH	BP	DI			
110	DH	SI	közv. op.	BP +8 bit d.	BP +16 bit d.	
111	BH	DI	BX	"00"+8 bit d.	"00"+16 bit d.	Memória

A regiszter aszerint byte vagy szó, hogy a műveleti kód legalacsonyabb helyértékű bit-je 0 vagy 1.

Szimbolikus alakban az operandusok sorrendje, gépi utasítás formájában a gépi utasítás kód mondja meg a regiszter és a memória közti adatátvitel irányát. Pl. az alábbi két utasítás esetén a címzési mód byte megegyezik.

```
MOV AX, 122H[SI+BX] ; hexadecimális 8B 80 0122
MOV 122H[BX][SI], AX ; hexadecimális 89 80 0122
```

A címzési mód byte-ról leolvasható, hogy az általános regiszterek, a két index (SI, DI) továbbá a stack pointer és base pointer (SP, BP) regiszter korlátlanul használható, míg a többi (a szegmens regiszterek, IP és STATUS) csak speciális utasításokkal érhető el. Pl.:

```
MOV DS, ADAT ; hibás!
```

Ezt a feladatot az alábbi két utasítással oldhatjuk meg:

```
MOV    AX, ADAT    ; helyes!  
MOV    DS, AX      ; helyes!
```

A „többi” regiszter nem lehet aritmetikai utasítás operandusa, sőt, IP és CS csak vezérlés átadó utasításokkal módosítható, közvetlenül nem is olvasható.

2.4.2 Feladatok

1. A memória melyik byte-ján található az éppen végrehajtandó utasítás, ha CS=37FEH és IP=9DA7H ?
2. A memória melyik byte-ját tölti AL-be az alábbi utasítás:

```
MOV    AL, 37H[BX][SI]
```

ha BX=7F3BH, SI=0DA02H, és DS=34E0H ?

3. Mi BX tartalma, ha DS=0AEBCH és a

```
MOV    AL, -1FH[BX]
```

utasítás a memória 0BA029H című byte-ját tölti AL-be?

4. Mit mondhatunk egy utasítás operandusairól, ha a címzési mód byte 0E7H?

3. UTASÍTÁS RENDSZER 1. RÉSZ

Az utasítás rendszer ismertetése során a következő jelöléseket fogjuk alkalmazni:

$\Leftarrow$: értékadás

$\Leftrightarrow$: felcserélés

op, op1, op2: tetszőlegesen választható operandus (közvetlen, memória vagy regiszter). op1 és op2 közül az egyik regiszter kell legyen!

reg: általános, bázis vagy index regiszter

mem: memória operandus

ipr: (8 bites) IP relatív cím

port: port cím (8 bites eltolás vagy DX)

[op]: az op által mutatott cím tartalma

Egy utasítás operandusait, -vel választjuk el egymástól. Az operandusok elé és után tetszés szerint írhatunk szóközt vagy tabulátort. Az eredmény általában az első címen képződik, ennek megfelelően op1 (op) általában nem lehet közvetlen operandus.

3.1 Adat mozgató utasítások

Az adat mozgató utasítások nem módosítják a flag-eket (természetesen ez nem vonatkozik a POPF és a SAHF utasításra).

MOV	op1, op2	; op1 $\Leftarrow$ op2 (MOVE)
XCHG	op1, op2	; op1 $\Leftrightarrow$ op2 (eXCHanGe), op2 sem lehet ; közvetlen operandus
XLAT		; AL $\Leftarrow$ [BX+AL] (trans(X)LATe), a BX ; által címzett maximum 256 byte-os ; tartomány AL-edik byte-jának ; tartalma lesz AL új tartalma
LDS	reg, mem	; reg $\Leftarrow$ mem, mem+1 ; DS $\Leftarrow$ mem+2, mem+3 (Load DS)
LES	reg, mem	; reg $\Leftarrow$ mem, mem+1 ; ES $\Leftarrow$ mem+2, mem+3 (Load ES)
LEA	reg, mem	; reg $\Leftarrow$ mem effektív (logikai) címe ; (Load Effective Address)

A veremmel (stack-kel) kapcsolatos adat mozgató utasítások:

```

PUSH  op          ; SP  $\leftarrow$  SP-2; (SS:SP)  $\leftarrow$  op
PUSHF                ; (PUSH Flags)
                    ; SP  $\leftarrow$  SP-2; (SS:SP)  $\leftarrow$  STATUS
POP    op          ; op  $\leftarrow$  (SS:SP); SP  $\leftarrow$  SP+2
POPF                ; (POP Flags)
                    ; STATUS  $\leftarrow$  (SS:SP); SP  $\leftarrow$  SP+2

```

Az Intel 8080-nal való kompatibilitást célozza az alábbi két utasítás:

```

LAHF          ; AH  $\leftarrow$  STATUS alsó 8 bitje
SAHF          ; STATUS alsó 8 bitje  $\leftarrow$  AH

```

3.1.1 Feladatok

1. Legyen A word típusú változó. Írjunk program részletet a PUSH, és a POP utasítás segítségével, amelynek hatása megegyezik az

```
XCHG AX, A
```

utasítás hatásával!

3.2 Aritmetikai utasítások

```
ADD op1, op2 ; op1  $\leftarrow$  op1 + op2 (ADD)
```

Az összeadás és kivonás után a C és O flag értékét az határozza meg, hogy az operandusokat előjel nélkülinek (C) illetve előjelesnek tekintve (O) az eredmény helyes (0) vagy túlcsondult (1). Ha pl. $AX = -1$ ($= 0FFFFH$), akkor könnyű látni, hogy

```
ADD AX, 2
```

után $AX=1$, $C=1$ és $O=0$.

```

ADC op1, op2 ; op1  $\leftarrow$  op1 + op2 + C
              ; (ADD with Carry)

```

Az ADC utasítás a több szavas összeadást támogatja. Ha pl. a két operandus a (DX:AX) és a (CX:BX) regiszter párban van, akkor az összegük – függetlenül attól, hogy előjeles vagy előjel nélküli mennyiségek – az

```
ADD    AX, BX
ADC    DX, CX
```

utasításokkal számítható ki.

```
INC    op    ; op  $\leftarrow$  op + 1, C változatlan! (INCrement)
```

A pakolatlan (ASCII kódú, egy számjegy 8 biten) és pakolt (egy számjegy 4 biten) decimális aritmetika megvalósításához nyújt segítséget az A flag (Auxiliary Carry): ha a művelet során a 3. bit-ről túlcordulás történik, akkor A=1, különben A=0. A decimális aritmetika utasításai ennek a flag-nek az értékét is használják.

A pakolatlan decimális számok összeadását támogatja az

```
AAA          ; (Ascii Adjust after Addition) két
              ; ASCII szám összeadása után korrigál:
              ; 1. Ha AL alsó 4 bitje  $\leq 9$ ,
              ;    és az A flag = 0, akkor  $\rightarrow 3$ .
              ; 2. AL  $\leftarrow$  AL + 6, AH  $\leftarrow$  AH + 1,
              ;    A flag  $\leftarrow$  1
              ; 3. AL felső 4 bitje  $\leftarrow$  0
```

utasítás. Pl.: Legyen AX=735H és BX=39H (AH = 7 és AL=35H = 5 ASCII kódja), ekkor

```
ADD    AL, BL ; AL = 6EH, A flag = 0
AAA          ; AL alsó 4 bitje (E) > 9, ezért
              ; AL  $\leftarrow$  AL + 6, AH  $\leftarrow$  AH + 1
              ; AL felső 4 bitje  $\leftarrow$  0, tehát
              ; AL  $\leftarrow$  4, AH  $\leftarrow$  8, azaz AX  $\leftarrow$  0804H
```

A pakolt decimális számok összeadását támogatja a

```
DAA          ; (Decimal Adjust after Addition)
              ; két pakolt decimális szám összeadása
              ; után korrigál:
              ; 1. Ha A=1 vagy AL alsó 4 bitje > 9,
              ;    akkor AL  $\leftarrow$  AL + 6, A flag  $\leftarrow$  1
              ; 2. Ha C=1 vagy AL felső 4 bitje > 9,
              ;    akkor AL  $\leftarrow$  AL + 60H, C  $\leftarrow$  1
```


utasítás. Pl.: Legyen AL=48H és BL=68H, ekkor

```
ADD  AL, BL ; AL = 0B0H, A flag = 1, C = 0
DAA      ; 1. A=1 miatt
          ; AL  $\leftarrow$  0B0H+6 = 0B6H és A  $\leftarrow$  1
          ; 2. AL felső négy bitje=B > 9 miatt
          ; AL  $\leftarrow$  0B6H + 60H = 16H és C  $\leftarrow$  1
```

```
SUB  op1, op2 ; op1  $\leftarrow$  op1 - op2 (SUBtraction)
CMP  op1, op2 ; op1 - op2 szerint állítja be a
              ; flag-eket (CoMPare)
SBB  op1, op2 ; op1  $\leftarrow$  op1 - op2 - C
```

SBB a több szavas kivonás elvégzését segíti.

```
DEC  op ; op  $\leftarrow$  op - 1, C változatlan (DECrement)
AAS      ; (Ascii Adjust after Subtraction)
          ; két ASCII szám kivonása után
          ; korrigál:
          ; 1. Ha AL alsó 4 bitje  $\leq$  9,
          ;    és A flag = 0  $\rightarrow$  3.
          ; 2. AL  $\leftarrow$  AL - 6, AH  $\leftarrow$  AH - 1, A  $\leftarrow$  1
          ; 3. AL felső 4 bitje  $\leftarrow$  0
          ; 4. C  $\leftarrow$  A
DAS      ; (Decimal Adjust after Subtraction)
          ; két pakolt decimális szám kivonása után
          ; korrigál:
          ; 1. Ha A=1 vagy AL alsó 4 bitje > 9,
          ;    akkor AL  $\leftarrow$  AL - 6, A  $\leftarrow$  1
          ; 2. Ha C=1 vagy AL felső 4 bitje > 9,
          ;    akkor AL  $\leftarrow$  AL - 60H, C  $\leftarrow$  1
NEG  op ; op  $\leftarrow$  -op (NEGate)
```

Az összeadástól és kivonástól eltérően a szorzás és osztás esetében különbséget kell tennünk, hogy előjeles vagy előjel nélküli számábrázolást alkalmazunk-e. További lényeges eltérés, hogy két 8 bites vagy 16 bites mennyiség szorzata ritkán fér el 8 illetve 16 biten, ezért a szorzás műveletét úgy alakították ki, hogy 8 bites tényezők szorzata 16, 16 biteseké pedig 32 biten keletkezzék:

```
MUL  op ; (MULtiplicate)
          ; előjel nélküli szorzás
          ; op nem lehet közvetlen operandus
          ; AX  $\leftarrow$  AL * op, ha op 8 bites,
          ; (DX:AX)  $\leftarrow$  AX * op, ha op 16 bites
```



```

IMUL  op          ; (Integer MULTiplicate)
                  ; előjeles szorzás
                  ; op nem lehet közvetlen operandus
                  ;  $AX \leftarrow AL * op$ , ha op 8 bites,
AAM    ;  $(DX:AX) \leftarrow AX * op$ , ha op 16 bites
                  ; (Ascii Adjust after Multiplication)
                  ;  $AH \leftarrow AL/10$  hányadosa,  $AL \leftarrow$  maradéka

```

Tegyük fel, hogy $AL=7$ és $BL=8$ és a kettő szorzatát akarjuk binárisan kódolt decimális (BCD, pakolatlan binárisan kódolt decimális) formában kiszámolni. Ezt a következőképpen tehetjük meg:

```

MUL    BL          ;  $AX \leftarrow 38H = 56$ 
                  ; (tudjuk, hogy AH-ban is elfér)
AAM    ;  $AH \leftarrow 5$ ,  $AL \leftarrow 6$ ,
                  ; tehát  $AX \leftarrow 56$  BCD kódban

```

Megjegyezzük, hogy az AAM utasítás nem használja ki, hogy AH tartalma egy szorzás eredménye, ezért felhasználható az egy byte-on elférő számok decimálisra konvertálásához.

Az osztás, mint a szorzás inverz művelete szintén lehet előjeles és előjel nélküli, az osztandó 16 illetve 32 bites aszerint, hogy az osztó 8 vagy 16 bites. Ha a szám, amit osztani akarunk nem eleve 16 illetve 32 bitből áll, akkor természetesen gondoskodnunk kell az osztandó megfelelő kiterjesztéséről.

Előjel nélküli osztás előkészítése AH illetve DX nullázásával történik. Előjeles osztás előkészítésére szolgál az alábbi két előjel kiterjesztő utasítás:

```

CBW    ; (Convert Byte to Word)
        ;  $AX \leftarrow AL$  előjel helyesen
CWD    ; (Convert Word to Double word)
        ;  $(DX:AX) \leftarrow AX$  előjel helyesen

```

Pozitív számok esetén (az előjel 0) az előjel kiterjesztés az AH illetve a DX regiszter nullázását, negatív számok esetén (az előjel 1) csupa 1-es bittel való feltöltését jelenti.

```

DIV    op          ; (DIVide) előjel nélküli osztás
                  ; op nem lehet közvetlen operandus,
                  ; ha op 8 bites:
                  ;  $AL \leftarrow AX/op$  hányadosa,
                  ;  $AH \leftarrow AX/op$  maradéka,

```

```

; ha op 16 bites:
;   AX  $\leftarrow$  (DX:AX)/op hányadosa,
;   DX  $\leftarrow$  (DX:AX)/op maradéka
IDIV op      ; (Integer DIVide) előjeles osztás
; op nem lehet közvetlen operandus,
; a nem 0 maradék előjele megegyezik
; az osztóéval
; ha op 8 bites:
;   AL  $\leftarrow$  AX/op hányadosa,
;   AH  $\leftarrow$  AX/op maradéka,
; ha op 16 bites:
;   AX  $\leftarrow$  (DX:AX)/op hányadosa,
;   DX  $\leftarrow$  (DX:AX)/op maradéka

```

Megjegyzés: Osztásnál előfordulhat, hogy a hányados nem fér el AH-ban illetve AX-ben. Egy ilyen osztásnak az a következménye, hogy azonnal elhal (abortál) a programunk. Ezt elkerülendő, célszerű, ha osztás előtt ellenőrzéseket végzünk: Nem nulla-e az osztó? Nem túl nagy-e az osztandó? Pl. előjel nélküli 8 bites osztás esetén túl nagy az osztandó, ha $AH \geq op$.

```

AAD          ; (Ascii Adjust before Division)
; AL  $\leftarrow$  AH * 10 + AL, AH  $\leftarrow$  0

```

AAD kis decimális számok binárisra konvertálásához használható. Az osztást már binárisan hajtjuk végre. Ha pl. a pakolatlan BCD kódú 72-t 9-cel akarjuk elosztani, akkor azt a következő program részlettel tehetjük meg:

```

MOV  AX, 0702H    ; AX  $\leftarrow$  72 pakolatlan BCD kódban
AAD          ; AX  $\leftarrow$  048H = 7*10+2 (osztandó)
MOV  BL, 9        ; (osztó)
DIV  BL          ; AL $\leftarrow$ 8 (hányados), AH $\leftarrow$ 0 (maradék)

```

3.2.1 Feladatok

1. Írjunk program részletet $E=A+B$ és $E=A-B$ meghatározására, ha A, B és E dupla szó!
2. Írjunk program részletet $E=A/B$ meghatározására, ha A, B és E előjeles (előjel nélküli) byte (word)!
3. Írjuk meg a fenti program részletet úgy, hogy hibajelzést adunk, ha az eredmény nem fér el E-ben.

4. Legyen A előjeles, B előjel nélküli byte! Írjunk program részletet A/B és B/A meghatározására!
5. Írjunk program részletet $E=A*B+C$ meghatározására, ha A, B, C előjeles (előjel nélküli) byte (word), és E word (double word)!
6. Írjunk program részletet $E=(A*B+C)/D$ meghatározására, ha A, B, C, D előjeles (előjel nélküli) byte (word), és E word (double word)!
7. Írjunk program részletet az előjel nélküli AL decimális formában történő kiírására az AAM utasítás felhasználásával!
8. Írjunk program részletet az előjel nélküli (DX:AX) decimális formában történő kiírására, feltéve, hogy $DX < 10$!

3.3 Eljárás deklaráció, hívás, paraméter átadás

Gyakorlásként készítsük el két vektor skalár szorzatának programját több változatban. Mivel az assembler nem különbözteti meg a nagy- és kisbetűt, a nagybetűt az újdonságok kiemeléséhez fogjuk használni. Vastag betűket helyezünk el kommentárban azokon a pontokon, amelyekhez a későbbiek során megjegyzést fűzünk.

```
; Két vektor skalár szorzata. 1. változat
code          segment      para public 'code'
               assume      cs:code,ds:data,ss:stack,es:nothing

skalar        proc far
               push ds      ; visszatérési cím a verembe
               xor ax,ax    ; ax ← 0
               push ax      ; visszatérés offset címe
               mov ax,data  ; ds a data szegmensre mutasson
               mov ds,ax    ; sajnos „mov ds,data”
                           ; nem megengedett

; A
               mov cl,n     ; cl ← n, 0 ≤ n ≤ 255
               xor ch,ch    ; cx = n szavasan
               xor dx,dx    ; az eredmény ideiglenes helye
               JCXZ kész    ; ugrás a kész címkére,
                           ; ha CX (=n) = 0

               xor bx,bx    ; bx ← 0,
                           ; bx-et használjuk indexezéshez

ism:          mov al,a[bx]  ; al ← a[0], később a[1], ...
               imul b[bx]   ; ax ← a[0]*b[0], a[1]*b[1], ...
               add dx,ax    ; dx ← részösszeg
               inc bx       ; bx ← bx+1, az index növelése
```

```

; B
    dec    cx          ; cx ← cx-1, (vissza) számlálás
    JCXZ   kész        ; ugrás a kész címkére, ha cx=0
    jmp    ism         ; ugrás az ism címkére

kész:
; C
    mov    ax,dx       ; a skalár szorzat értéke ax-ben
    call   hexa        ; az eredmény kiírása
                    ; hexadecimálisan
    mov    si,offset kvse ; kocsi vissza sosemelés
    call   kiirro      ; kiírása
    ret     ; vissza az Op. rendszerhez
skalar
endp          ; a skalár eljárás vége
; D
; -----
hexa          proc      ; ax kiírása hexadecimálisan
    xchg   ah,al       ; ah és al felcserélése
    call   hexa_b      ; al (az eredeti ah) kiírása
    xchg   ah,al       ; ah és al visszacserélése
    call   hexa_b      ; al kiírása
    ret     ; visszatérés
hexa
endp          ; a hexa eljárás vége
; -----
hexa_b        proc      ; al kiírása hexadecimálisan
    push   cx          ; mentés a verembe
    mov    cl,4        ; 4 bit-es rotálás előkészítése
    ROR    al,CL       ; az első jegy az alsó 4 biten
    call   h_jegy      ; az első jegy kiírása
    ROR    al,CL       ; a második jegy az alsó 4 biten
    call   h_jegy      ; a második jegy kiírása
    pop    cx          ; visszamentés a veremből
    ret     ; visszatérés
hexa_b
endp          ; a hexa_b eljárás vége
; -----
h_jegy        proc      ; hexadecimális jegy kiírása
    push   ax          ; mentés a verembe
    AND    al,0FH      ; a felső 4 bit 0 lesz,
                    ; a többi változatlan
    add    al,'0'      ; + 0 kódja
    cmp    al,'9'      ; ≤ 9 ?
    JLE    h_jegy1     ; ugrás h_jegy1 -hez, ha igen
    add    al,'A'-'0'-0AH ; A-F hexadecimális jegyek
                    ; kialakítása
h_jegy1:      mov    ah,14 ; BIOS szolgáltatás előkészítése
    int    10H         ; BIOS hívás: karakter kiírás
    pop    ax          ; visszamentés a veremből
    ret     ; visszatérés
h_jegy
endp          ; a hexa_b eljárás vége
; -----
kiirro        proc      ; szöveg kiírás (DS:SI)-től
    push   ax
    cld

```

```

ki1:      lodsb          ; al ← a következő karakter
          cmp    al, 0    ; al =? 0
          je     ki2      ; ugrás a ki2 címkehez, ha al=0
          mov    ah,14     ; BIOS rutin paraméterezése
          int    10H       ; az AL-ben lévő karaktert
                          ; kiírja a képernyőre
          jmp    ki1       ; a kiírás folytatása

ki2:      pop    ax
          ret            ; visszatérés a hívó programhoz
kiiro     endp          ; a kiíró eljárás vége
; -----
code      ends          ; a code szegmens vége
; =====
data      segment       para public 'data'
n         db    3
a         db    1,2,3
b         db    3,2,1
kvse      db    13,10,0 ; kocsi vissza soremelés
data      ends          ; a data szegmens vége
; =====
stack     segment       para stack 'stack'
          dw    100 dup (?) ; 100 word legyen a verem
stack     ends          ; a stack szegmens vége
; =====
          end    skalar  ; modul vége,
                          ; a program kezdő címe: skalar

```

Megjegyezzük, hogy a **B** pontnál lévő 3 utasítás a

```

LOOP ism    ; ugrás az ism címke-re, ha kell ismételni

```

utasításra cserélhető (*Lásd* Ciklus szervező utasítások).

Ritkán fordul elő, hogy egy programnak csak egy skalár szorzatot kell kiszámítania. Ha úgy fogjuk fel, hogy az **A** és **C** pont közötti rész – a tulajdonképpeni skalár szorzat számítás – csak része egy nagyobb programnak, akkor illő, hogy gondoskodjunk azon regiszter tartalmak megőrzéséről, amelyeket – programozás technikai okokból – megváltoztattunk. Ez úgy érhető el, ha az **A** pontnál verembe mentjük ezen regiszterek tartalmát:

```

PUSH  BX
PUSH  CX
PUSH  DX

```

és a **C** pontnál visszamentjük:

POP	DX
POP	CX
POP	BX

Természetesen AX-et nem kell mentenünk, hiszen a program itt várja az eredményt.

Az így módosított programnak az a hátránya, hogy valahányszor skalár szorzatot akarunk számítani, mindannyiszor be kell másolnunk az A és C közti program részt. Ezen úgy tudunk segíteni, ha ezt a programrészt eljárássá tesszük.

3.3.1 A paraméterek szabványos helyen történő átadása

Eljárásnak a program egy

Elj_név PROC *típus*

pszeudo utasítással kezdődő (ez az eljárás belépési pontja), és

Elj_név ENDP

utasítással végződő szakaszát értjük (eljárás deklaráció), ahol *Elj_név* az eljárás neve, *típus* – az eljárás típusa – lehet NEAR (közeli) vagy FAR (távoli). Közeli eljárás esetén NEAR megadása nem kötelező. Eljárás hívásra a CALL utasítás szolgál (verembe helyezi a IP – távoli eljárásnál IP és CS – tartalmát, és átadja a vezérlést az operandusként megadott címkénél lévő utasításra). Az eljárásból a RET utasítással térhetünk vissza a CALL utasítás mögé. A RET utasítás kiolvassa a veremből IP – távoli eljárásnál CS és IP – tartalmát, ezáltal visszatér a hívás utáni utasításhoz.

A vektorok skalár szorzatát kiszámító program újabb változatához az eredeti változat A és D pontja közötti részt a következőképpen írjuk át:

```
; Két vektor skalár szorzata. 2. változat
...
; A
; ELJÁRÁS HÍVÁS
; A PARAMÉTEREK SZABVÁNYOS HELYEN TÖRTÉNŐ ÁTADÁSÁVAL
CALL SKAL ; ELJÁRÁS HÍVÁS
; AZ EREDMÉNY AZ AX REGISZTERBEN
call hexa ; az eredmény kiírása
```



```

        mov     si,offset kvse ; kocsi vissza solemelés
        call    kiirro         ; kiírása
...
; C
        ret     ; vissza az Op. rendszerhez
skalar  endp     ; a skalár eljárás vége
SKAL    PROC    ; KÖZELI (NEAR) ELJÁRÁS KEZDETE
        PUSH    BX      ; MENTÉSEK
        PUSH    CX
        PUSH    DX
        mov     cl,n     ;  $cl \leftarrow n, 0 \leq n \leq 255$ 
        xor     ch,ch    ;  $cx = n$  szavasan
        xor     dx,dx    ; az eredmény ideiglenes helye
        jcxz    kész     ; ugrás a kész címkére, ha  $n=0$ 
        xor     bx,bx    ;  $bx \leftarrow 0$ ,
                        ;  $bx$ -et használjuk indexezéshez
ism:    mov     al,a[bx] ;  $al \leftarrow a[0]$ , később  $a[1]$ , ...
        imul    b[bx]    ;  $ax \leftarrow a[0]*b[0]$ ,  $a[1]*b[1]$ , ...
        add     dx,ax    ;  $dx \leftarrow$  részösszeg
        inc     bx      ;  $bx \leftarrow bx+1$ , az index növelése
; B
        LOOP    ism     ; ugrás az ism címkére,
                        ; ha kell ismételni
kész:   mov     ax,dx    ; a skalár szorzat értéke ax-ben
        POP     DX      ; VISSZAMENTÉSEK
        POP     CX
        POP     BX
        RET          ; VISSZATÉRÉS A HÍVÓ PROGRAMHOZ
SKAL    ENDP         ; A SKAL ELJÁRÁS VÉGE
; D

```

Ebben a változatban az a és b vektor skalár szorzatát bármikor ki tudjuk számolni a

```
call skal ; eljárás hívás
```

utasítás segítségével (szükség lehet rá, hiszen az a és b vektor megváltozhat), de nem szép a programunkban, hogy csak e két vektor skalár szorzatát tudja kiszámolni, holott egy nagyobb feladatban különböző vektorok skalár szorzatának meghatározására lehet szükség (a skalár szorzat számítás persze egyszerű, néhány utasításban elintézhető, de az elvre gondoljunk!). A fenti változat csak úgy alkalmas különböző vektorok skalár szorzatának kiszámítására, ha az eljárás hívás előtt a vektorokat és a vektorok dimenzióját bemásoljuk a -ba, b -be és n -be. Jó lenne, ha bemásolás helyett paraméterként tudnánk megadni, hogy mely vektorok skalár szorzatát kell meghatározni.

3.3.2 A paraméterek regiszterekben történő átadása

Tekintve, hogy az 1. változatnak csak az A és D pontja közötti rész változik, csak azt fogjuk leírni:

```
; Két vektor skalár szorzata. 3. változat
...
; A
; ELJÁRÁS HÍVÁS
; A PARAMÉTEREK REGISZTEREKBE TÖRTÉNŐ ÁTADÁSÁVAL
      MOV     CL,N           ; PARAMÉTER BEÁLLÍTÁSOK
      XOR     CH,CH         ; CX = N
      MOV     SI,OFFSET a    ; SI ← a OFFSET CÍME
      MOV     DI,OFFSET b    ; DI ← b OFFSET CÍME
      call    skal           ; eljárás hívás
                                ; az eredmény az ax regiszterben
      call    hexa           ; az eredmény kiírása
      mov     si,offset kvse ; kocsi vissza soremelés
      call    kiirro         ; kiírása
...
skalar    ret                ; visszatérés az Op. rendszerhez
                                ; a skalár eljárás vége

skal      proc                ; Közele (NEAR) eljárás kezdete
      push    bx              ; mentések
      push    cx
      push    dx

      xor     bx,bx           ; bx ← 0,
                                ; bx-et használjuk indexezéshez
      xor     dx,dx           ; az eredmény ideiglenes helye
      jcxz    kész            ; ugrás a kész címkére, ha n=0
ism:      mov     al,[SI+BX]   ; FÜGGETLEN a-TÓL
      imul    BYTE PTR [DI+BX] ; FÜGGETLEN b-TÓL
; csak „BYTE PTR”-ből derül ki, hogy 8 bites a szorzás
      add     dx,ax           ; dx ← részösszeg
      inc     bx              ; bx ← bx+1, az index növelése
      loop    ism             ; ugrás az ism címkére,
                                ; ha kell ismételni
kész:     mov     ax,dx        ; a skalár szorzat értéke ax-ben

      pop     dx              ; visszamentések
      pop     cx
      pop     bx
      ret                ; visszatérés a hívó programhoz
skal      endp                ; a skal eljárás vége
; D
...
```


Vegyük észre, hogy míg a-nak és b-nek a címét adtuk át paraméterként, addig n-nek az értékét. Általában is szokásos a skaláris paraméterek érték szerinti, az összetett adatok cím szerinti átadása. Ezt indokolja, hogy pl. egy hosszabb vektor érték szerinti átadása (másolása) idő és hely igényes munka. Az érték szerinti paraméter átadásnál a hívott program nem tudja módosítani – a hívó program számára észrevehető módon – a paraméter értékét. Ez részben előny: nem tudja elrontani, részben hátrány, mert néha jó lenne, ha módosítani tudná – ilyenkor cím szerint kell átadni a paramétert.

3.3.3 A paraméterek veremben történő átadása

A 3. változatban erősen kihasználtuk, hogy elegendő számú regiszter áll rendelkezésünkre a paraméterek átadásához. Ez általában nem teljesül. A paraméter átadásnak a következő változatban bemutatott módja mindig lehetséges. Magasabb szintű nyelvek fordító programjai is hasonlóan oldják meg a paraméter átadás-átvételt. Ismét az eredeti változat A és D pontja közötti részét fogjuk átírni.

```
; Két vektor skalár szorzata. 4. változat
...
; A
; ELJÁRÁS HÍVÁS
; A PARAMÉTEREK VEREMBEN TÖRTÉNŐ ÁTADÁSÁVAL
    MOV    AL,N          ; AL-T NEM KELL MENTENI, MERT
    XOR    AH,AH         ; AX-BEN KAPJUK AZ EREDMÉNYT.
    PUSH   AX            ; AX=N
    MOV    AX,OFFSET a    ; AX ← a OFFSET CÍME
    PUSH   AX
    MOV    AX,OFFSET b    ; AX ← b OFFSET CÍME
    PUSH   AX
; a verem tartalma:
; (SS:SP)  b címe
;      +2  a címe
;      +4  n értéke
;
    call   skal           ; eljárás hívás
                        ; eredmény az ax regiszterben
    ADD    SP,6           ; A VEREMBŐL KIÜRÍTÍ
                        ; A PARAMÉTEREKET
...
ret                ; visszatérés az Op. rendszerhez
skalar            endp    ; a skalár eljárás vége
```

```

skal      proc                ; Közeli (near) eljárás kezdete
; a verem tartalma:
; (SS:SP)  visszatérési cím
;      +2   b címe
;      +4   a címe
;      +6   n értéke

      PUSH  BP                ; BP ÉRTÉKÉT MENTENÜNK KELL!
      MOV   BP,SP             ; BP ← SP,
                                ; A STACK RELATÍV CÍMZÉSHEZ
                                ; MENTÉSEK
      PUSH  SI
      PUSH  DI
      push  bx
      push  cx
      push  dx
; a verem tartalma:
; (SS:SP)  dx
;      +2   cx
;      +4   bx
;      +6   di
;      +8   si
;      +10  bp - - - - - (SS:BP)
;      +12  visszatérési cím      +2
;      +14  b címe                +4
;      +16  a címe                +6
;      +18  n értéke              +8
;
      MOV   SI,6[BP]           ; SI ← AZ EGYIK VEKTOR CÍME
      MOV   DI,4[BP]           ; DI ← A MÁSIK VEKTOR CÍME
      MOV   CX,8[BP]           ; CX ← A DIMENZIÓ ÉRTÉKE
      xor   bx,bx              ; bx ← 0, indexezéshez
      xor   dx,dx              ; az eredmény ideiglenes helye
      jcxz  kész               ; ugrás a kész címkére, ha n=0
ism:      mov  al,[si+bx]       ; független a-tól
      imul  byte ptr [di+bx]   ; független b-től[1]*b[1]
; csak „byte ptr”-ből derül ki, hogy 8 bites a szorzás
      add   dx,ax              ; dx ← részösszeg
      inc   bx                 ; bx ← bx+1, az index növelése
      loop  ism                ; ugrás az ism címkére,
                                ; ha kell ismételni
kész:     mov  ax,dx            ; a skalár szorzat értéke ax-ben

      pop   dx                 ; visszamentések
      pop   cx
      pop   bx
      pop   di
      pop   si
      pop   bp
      ret                     ; visszatérés a hívó programhoz
skal      endp                ; a skal eljárás vége
; D

```

A skal eljárásból a

```
RET    6           ; visszatérés a hívó programhoz
        ; verem ürítéssel
```

utasítással is visszatérhettünk volna. Ez az utasítás 6-tal megnöveli az SP értékét – kiveszi a veremből a három paramétert –, ekkor természetesen nincs szükség az

```
ADD    SP,6      ; A VEREMBÓL KIÜRÍTI A PARAMÉTEREKET
```

utasításra. Ez utóbbi megoldás előnye, hogy a hívó program így nem feledkezhet meg a paraméterek kiürítéséről, hátránya lehet viszont, hogy a hívott eljárás legfeljebb azt tudja megállapítani, hogy hány paramétert kellett volna átadni, de azt nem, hogy ténylegesen hányat adott meg a hívó eljárás.

Hogy egy eljárás különböző számú paraméterrel legyen hívható, azt úgy lehet elérni, hogy a paramétereket fordított sorrendben tesszük a verembe, mert ilyenkor a visszatérési cím alatt lesz az első, alatta a második, stb. paraméter, és általában a korábbi paraméterek döntik el, hogy hogyan folytatódik a paraméter sor. Történelmi oka van annak, hogy pl. a C fordítók is így működnek. Az újabb C fordítók esetén ennek már nincs jelentősége, mert a kötelező prototípus megadás következtében a hívó és hívott eljárás közti pontos kapcsolódás nélkül is megoldható.

A fentiekből látható, hogy a paraméter átadás-átvételnek sokféle módja van, ezért egy eljárás dokumentációjához feltétlenül hozzátartozik az is, hogy hogyan kell a paramétereket átadni, és hogy a paraméterek veremből történő kiürítése a hívó vagy a hívott eljárás feladata.

3.3.4 Lokális adat terület, rekurzív és reentrent eljárások

Ha egy eljárás működéséhez lokális adat területre, munkaterületre van szükség, és a működés befejeztével a munkaterület tartalma felesleges, akkor a munkaterületet célszerűen a veremben alakíthatjuk ki. A munkaterület lefoglalásának ajánlott módja:

```

...      proc      ...
          PUSH     BP           ; BP értékének mentése
          MOV      BP,SP       ; BP ← SP,
                                ; a stack relatív címzéshez
          SUB      SP,n        ; n a munkaterület byte-jainak
                                ; száma
          ...                ; további regiszter mentések

```

A munkaterület negatív displacement érték mellett stack relatív címzéssel érhető el. (A verembe elhelyezett paraméterek ugyancsak stack relatív címzéssel, de pozitív displacement érték mellett érhetőek el.) A munkaterület felszabadítása visszatéréskor a

```

...      ; visszamentések
MOV      SP,BP      ; BP értékének visszamentése
ret      ...        ; visszatérés

```

utasításokkal történhet. Ha egy eljárásunk készítésekor betartjuk, hogy az eljárás a paramétereit a vermen keresztül kapja, kilépéskor visszaállítja – az esetleg eredményt tartalmazó regiszterek kivételével – a belépéskori regiszter tartalmakat, továbbá a fenti módon kialakított munkaterületet használ, akkor az eljárásunk rekurzív is lehet, és a többszöri belépést is lehetővé teszi (reentrant). A többszöri belépés lehetősége azt jelenti, hogy az eljárás még nem fejeződött be, amikor újra felhívható. A rekurzív eljárással szemben a különbség az, hogy a rekurzív eljárásban „programozott”, hogy mikor történik az eljárás újra hívása, reentrant eljárás esetén az esetleges újra hívás ideje a véletlentől függ. Ez utóbbi esetben azt, hogy a munkaterületek ne keveredjenek össze, az biztosítja, hogy újabb belépés csak másik processzusból képzelhető el, és minden processzus saját vermet használ.

3.3.5 Feladatok

1. Írjunk program részletet m n -edik hatványának meghatározására (tégezzük fel, hogy az $n-1$ -ik hatvány még elfér egy szóban)!
2. Alakítsuk át a fenti program részletet eljárássá a megismert paraméter átadási módszerekkel.
3. Írjunk rekurzív eljárást $n!$ kiszámítására.

3.4 Vezérlés átvadó utasítások

Ez az utasítás csoport alkalmas az IP és a CS regiszter tartalmának átvadására.

Megjegyezzük, hogy minden utasítás végrehajtása azzal kezdődik, hogy IP tartalma az aktuális utasítás hosszával növekszik. Az alábbiakban IP értékén a már megnövelt értéket értjük.

3.4.1 Eljárásokkal kapcsolatos utasítások

Eljárás hívás:

CALL op ; eljárás hívás

Eltérő az utasítás működése attól függően, hogy op közeli (NEAR) vagy távoli (FAR) cím:

- közeli: *push* IP, $IP \leftarrow op$
- távoli: *push* CS, *push* IP, $(CS:IP) \leftarrow op$.

Visszatérés az eljárásból:

RET ; visszatérés a hívó programhoz (RETurn)

Az eljárás típusától (NEAR, FAR) függően:

- közeli: *pop* IP,
- távoli: *pop* IP, *pop* CS.

A RET utasításnak van

RET op

alakja is, ahol op csak közvetlen adat lehet. Ebben az esetben a fentiek után még $SP \leftarrow SP + op$ is végrehajtásra kerül.

A kommentárban használtuk a *push* és *pop* jelölést. A kurzív írással azt kívántuk jelezni, hogy nem PUSH és POP utasítást hajtunk ugyan végre, de ezeknek megfelelő hatás jön létre.

3.4.2 Feltétlen vezérlés átadás (ugrás)

```
JMP op          ; ha op közeli: IP   $\Leftarrow$  op,  
                ; ha távoli: (CS:IP)  $\Leftarrow$  op.
```

Lehetőleg kerüljük a JMP utasítást! Csak címkézett utasításra tudunk ugrani, és egy címke után a STATUS attól függhet, hogy honnan ugrottunk a címkére, és sajnos bárhol lehetett JMP. Sok JMP teljesen áttekinthetetlenné teheti a programot.

3.4.3 Feltételes vezérlés átadás (ugrás)

A feltételes ugró utasítások között több mnemonic ugyanazt a gépi utasítást takarja. Ennek az a célja, hogy az ugrás feltételét a lehető legtermészetesebben állapíthassuk meg, ne kelljen pl. flag-ekre konvertálnunk. Két nagy csoportra oszthatók: aritmetikai és a logikai feltételes ugrások. Az aritmetikai csoport utasításait általában aritmetikai (többnyire CMP vagy SUB) utasítások után, a logikai csoport utasításait a flag-ek állapota alapján szokás alkalmazni. Óvakodjunk, hogy pl. egy $\leq$ reláció teljesülése esetén szükséges ugrást megpróbáljunk a flag-ek állapota szerint végrehajtani.

Az aritmetikai és logikai ugrások csoportja átfedéseket tartalmaz, de kár lenne az átfedések megszüntetésére törekednünk, hiszen pl. egy CMP eredményének zéró volta és a zéró flag igaz állapota egyaránt szolgálhat az elágazás feltételül.

A mnemonic-ok kialakításában a következő szavak rövidítése játszott szerepet (a rövidítést nagybetűvel kiemeltük): Zero, Equal, No (Not), Greater, Less, Above, Below, Carry, Sign, Overflow, Parity Even, Parity Odd.

Az alábbi táblázat két részre osztja az aritmetikai ugrások csoportját aszerint, hogy az ugrás feltétele előjeles vagy előjel nélküli mennyiségek összehasonlításától függ.

Feltételes ugrások, aritmetikai csoport		
Előjeles	Reláció	Előjel nélküli
JZ $\equiv$ JE	=	JZ $\equiv$ JE
JNZ $\equiv$ JNE	$\neq$	JNZ $\equiv$ JNE
JG $\equiv$ JNLE	>	JA $\equiv$ JNBE
JGE $\equiv$ JNL	$\geq$	JA $\equiv$ JNB $\equiv$ JNC
JL $\equiv$ JNGE	<	JB $\equiv$ JNAE $\equiv$ JC
JLE $\equiv$ JNG	$\leq$	JBE $\equiv$ JNA

A logikai csoportba soroltuk a JCXZ utasítást is, bár ennél az utasításnál nem flag, hanem a CX regiszter tartalma szerint történik az elágazás.

Feltételes ugrások, logikai csoport		
a flag igaz (1)	flag	a flag hamis (0)
JZ $\equiv$ JE	Zero	JNZ $\equiv$ JNE
JC	Carry	JNC
JS	Sign	JNS
JO	Overflow	JNO
JP $\equiv$ JPE	Parity	JNP $\equiv$ JPO
JCXZ	CX = 0	

Valamennyi feltételes ugrás 8 bites IP relatív címzéssel valósul meg, ez azt jelenti, hogy csak -128 és +127 byte távolságra (SHORT) tudunk velük ugrani. Ha mégis nagyobb távolságra szeretnénk ugrani, akkor azt a negált feltételű ugrás segítségével oldhatjuk meg, pl. ha a

JZ MESSZE

utasításra a fordító hibát jelez, akkor ezt a hibát a következő formális átalakítással javíthatjuk:

JNZ IDE
JMP MESSZE

IDE: ...

3.4.4 Ciklus szervező utasítások

A ciklus szervező utasítások – a feltételes ugró utasításokhoz hasonlóan – relatív címezéssel valósulnak meg, tehát ezek is csak `SHORT` ugrásra képesek:

```

LOOP      ipr    ; CX ← CX-1, ugrás ipr-re, ha CX ≠ 0
LOOPZ     ipr    ; CX ← CX-1, ugrás ipr-re,
                  ; ha (CX ≠ 0 és Z=1)
LOOPE     ipr    ; ugyanaz mint LOOPZ
LOOPNZ    ipr    ; CX ← CX-1, ugrás ipr-re,
                  ; ha (CX ≠ 0 és Z=0)
LOOPNE    ipr    ; ugyanaz mint LOOPNZ

```

Példaként írjuk meg az A n -dik hatványát kiszámító

```
for(i=0, B=1; i<=n; i++) B *= A;
```

C utasítás megfelelőjét a `LOOP` utasítás segítségével, feltéve, hogy A és n előjel nélküli byte, B word, továbbá azt, hogy A $(n-1)$ -dik hatványa még elfér `AL`-ben:

```

                mov     CL, n           ; a ciklus előkészítése
                xor     CH, CH
                mov     AL, 1
                xor     AH, AH
                JCXZ     Kesz           ; ha n=0, akkor 0-szor
                                ; fut a ciklus mag
C_Mag:          mul     A               ; ciklus mag
                LOOP    C_Mag          ; ismétlés, ha kell
Kesz:           mov     B, AX

```

Megjegyezzük, hogy a

```
JCXZ Kesz
```

utasításra nincs szükség, ha tudjuk, hogy $n > 0$.

`LOOPZ` és `LOOPNZ` esetén az ugrásnak két feltétele is van: nem történik ugrás, ha már $CX=0$, de akkor sem, ha Z tartama nem megfelelő. Szükség lehet arra, hogy eldöntsük, hogy miért szakadt meg a ciklus futása: azért mert már elegendően sokszor futott, vagy azért, mert a Z flag állapota nem engedélyezte az ugrást. CX csak akkor dönti el a kérdést, ha $CX \neq 0$, különben

előfordulhat, hogy pontosan a ciklus utolsó futásakor állt be a z flag először ilyen állapotba. Természetesen a z flag állapota sem dönti el, hogy $CX=0$ vagy sem. Pl. a következő program részlettel dönthetjük el, hogy a 100 elemű array nevű tömbnek van-e 3-tól különböző eleme:

```

...
mov    cx,100
mov    di,-1          ; előbb lehessen inc, mint cmp
mov    al,3
NEXT:  inc    di
      cmp    array[di],al      ; array di-edik eleme = 3?
      LOOPE  NEXT              ; ugrás NEXT-re,
                                ; ha CX≠0 és a di-dik elem=3
      JNE    NEM3
      ...                      ; ARRAY = 3
      ...
NEM3:  ...                  ; di az első 3-tól különböző
                                ; elem indexe
      ...

```

3.4.5 Feladatok

1. Írjunk eljárást az előjel nélküli (DX:AX) decimális formában történő kiírására, feltéve, hogy $DX < 10000$!
2. Írjunk eljárást az előjel nélküli (DX:AX) decimális formában történő kiírására!
3. Írjunk program részletet az előjeles (DX:AX) abszolút értékének meghatározására!
4. Írjunk eljárást az előjeles (DX:AX) decimális formában történő kiírására!
5. Írjunk rekurzív eljárást két szám legnagyobb közös osztójának meghatározására!

3.5 String kezelő utasítások

Ez az utasítás csoport byte-okból vagy word-okból álló összefüggő adat terület kezelésére szolgál. Az adat terület lehet forrás (source) vagy cél (destination) terület. A forrás területet mindig (DS:SI), a cél területet pedig (ES:DI) címzi. Ennek megfelelően nem kötelező a műveletek operandusait megadnunk. Ha nem adjuk meg az operandusokat, akkor a mnemonic végén szereplő B vagy W dönti el, hogy byte-os vagy szavas műveletet kell

végeznünk. Ha megadjuk az operandusokat, az nem fogja befolyásolni a forrás és cél terület helyét, csupán a művelet típusát (byte vagy word) fogja meghatározni.

Minden string kezelő utasítás módosítja az általa használt index regiszter(ek) tartalmát, méghozzá 1-gyel byte-os, 2-vel word-ös művelet esetén. A STATUS szó D flag-je (Direction) dönti el, hogy a módosítás pozitív (D=0, növekvő címek felé haladó) vagy negatív (D=1, csökkenő címek felé haladó) irányú.

Az alábbi utasítások – mint általában az adat mozgó utasítások – érintetlenül hagyják a flag-eket.

Byte illetve word átvitel az (ES:DI) által mutatott címre a (DS:SI) által mutatott címről:

```
MOVSB      ; MOVe String Byte
MOVSW      ; MOVe String Word
MOVS  d,s   ; MOVe String (byte vagy word)
```

Betöltés AL-be illetve AX-be a (DS:SI) által mutatott címről:

```
LODSB      ; LOaD String Byte
LODSW      ; LOaD String Word
LODS  s     ; LOaD String (byte vagy word)
```

Tárolás az (ES:DI) által mutatott címre AL-ből illetve AX-ből:

```
STOSB      ; STOrE String Byte
STOSW      ; STOrE String Word
STOS  d     ; STOrE String (byte vagy word)
```

Az alábbi utasítások beállítják a flag-eket.

Az (ES:DI) és a (DS:SI) által mutatott címen lévő byte illetve szó összehasonlítása:

```
CMPSB      ; CoMPare String Byte
CMPSW      ; CoMPare String Word
CMPS  d,s   ; CoMPare String (byte vagy word)
```

A flag-ek s - d értékének megfelelően állnak be.

Az (ES:DI) által mutatott címen lévő byte illetve szó összehasonlítása AL-lel illetve AX-szel, a flag-ek AX - d értékének megfelelően állnak be.

```
SCASB      ; SCAn String Byte
SCASW      ; SCAn String Word
SCAS d     ; SCAn String (byte vagy word)
```

A string kezelő utasítások elé ismétlő prefixumot írhatunk. Az ismétlő prefixumok a következők: REP = REPZ = REPE és REPNZ = REPNE. A Z, E, NZ és NE végződésnek hasonló szerepe van, mint a LOOP utasítás esetén. Ismétlő prefixum használata esetén a string kezelő utasítás többször kerül(het) végrehajtásra. Ha CX = 0, akkor a string kezelő utasítás egyszer sem kerül végrehajtásra, különben mindenegyes végrehajtást követően 1-gyel csökken a CX regiszter tartalma. Amennyiben CX csökkentett értéke 0, akkor nem történik további ismétlés. Ha a string kezelő utasítás beállítja a flag-eket, akkor az ismétlésnek további feltétele, hogy a flag állapota megegyezzen a prefixum végződésében előírttal. A program jobb olvashatósága érdekében flag-et nem állító utasítások előtt mindig REP-et használjunk, flag-et beállító utasítás elé pedig REP helyett a vele egyenértékű REPE-t vagy REPZ-t!

A LOOP utasítás ismertetésénél bemutatott feladatot string kezelő utasítás segítségével a következőképpen oldhatjuk meg:

```

...
mov     cx,100
mov     di,offset array
MOV     AL,3
REPE    SCAS array      ; array 0., 1., ... eleme = 3?
JNE     NEM3
...
...
; ARRAY = 3,
NEM3:   DEC     DI      ; DI az első ≠ 3 elemre mutat
...
```

Megjegyzések:

- A mostani megoldás az ES szegmens regisztert használja, és feltételeztük, hogy a tartalma jó.
- A két megoldás közötti eltérés nem minden pontban lényeges, pl. az első változat úgy is elkészíthető lett volna, hogy ott is

```
mov     di,offset array
```

szerepeljen, de a mostani megoldásnál ez az utasítás kötelező!

- A mostani megoldás lényegesen gyorsabb végrehajtást biztosít, mint a korábbi.

3.5.1 Feladatok

Minden feladat megoldásához használjunk ismétlési prefixumot!

1. Legyen s byte-okból álló n hosszúságú string. Töltsük fel minden byte-ját 'A' -val!
2. Töltsük fel az előző stringet az ismétlődő 'ABC' karakter mintával!
3. Töltsük át a byte-okból álló n hosszúságú s sting-et a vele egy szegmensben lévő t -be úgy, hogy a forrás és a cél terület egymásba nyúlhat!
4. Keressük meg a byte-okból álló n hosszúságú s sting-ben az 'A' karakter első (utolsó) előfordulását!
5. Keressük meg a byte-okból álló n hosszúságú s sting-ben az 'ABC' karakter lánc első (utolsó) előfordulását!
6. Döntsük el, hogy két n hosszúságú string közül abc sorrendben melyik van előbb.
7. Döntsük el, hogy egy m és egy n hosszúságú string közül abc sorrendben melyik van előbb.
8. Oldjuk meg a fenti feladatokat, ha nem ismerjük a string hosszát, hanem azt tudjuk, hogy 0 értékű byte jelzi a string végét.
9. Legyen v m word-öt tartalmazó tömb! Minden egyes word egy pointer, amely egy n hosszúságú (illetve 0-ra végződő) string kezdőcímét tartalmazza. Keressük ki az abc sorrendben legelső string-re mutató pointert.
10. Készítsünk lexikografikus rendező programot az előző feladat feltételei mellett.
11. Igaz-e, hogy az alábbi két program részlet hatása megegyezik?

```
rep    mov    cx, n ; 1. program részlet
       movsw

       mov    cx, n ; 2. program részlet
       shl    cx, 1
rep    movsb
```

Igaz-e, hogy a két program részlet hatása megegyezik, ha $n=10$?

4. EGYSZERŰSÍTETT LEXIKÁLIS ELEMZŐ

Fordítóprogramokban szokásos, hogy a lefordítandó programot olyan elemzésnek vetik alá, amely felismeri a programban előforduló nyelvi elemeket (kulcsszavak, azonosítók, ...), és ez alapján a fordítás további részéhez átalakítja a program szövegét. Ezt a feladatot a lexikális elemző végzi.

Most egy rendkívüli mértékben leegyszerűsített lexikális elemző programját ismertetjük, amelyben néhány érdekes programozási fogást tudunk bemutatni. Az elemzőnknek legyen az a feladata, hogy felismerje a szövegben előforduló azonosítókat, számokat, speciális jeleket és a program vége jelet, és ezek előfordulásakor rendre A, 0, , és . karaktert írjon a képernyőre. Az esetleges hibákat ? jelezze.

A program írását az adat szegmenssel kezdjük. A lehetséges szintaktikus helyzetekhez ugró táblákat definiálunk. Az ugró táblákhoz igazodva átkódoló táblát készítünk. Figyeljük meg, hogy a feladat megoldása a megfelelő adat szegmens kialakítása után triviális, az elemzés során az egyes karakterek feldolgozásához csak néhány utasítás végrehajtása szükséges.

```
DATA          segment para public 'DATA'
```

```
; ugró táblák a szintaktikus helyzetnek megfelelően:
```

```
; kezdetben, speciális és hibás karakter után
```

```
t_s          dw      hiba          ; hibás karakter:  
              ; áttérés spec. jel szintre  
              dw      lev_a        ; betű:  
              dw      lev_n        ; számjegy:  
              dw      lev_s        ; speciális jel:  
              dw      vege         ; szöveg vége: program vége
```

```
; azonosító szint
```

```
t_a          dw      hiba          ; hibás karakter:  
              ; áttérés spec. jel szintre  
              dw      OK          ; betű: nincs teendő  
              dw      OK          ; számjegy: nincs teendő  
              dw      lev_s        ; speciális jel: azonosító vége  
              dw      vege         ; szöveg vége: program vége
```



```

        mov     al,c_h      ; hibás karakter kódja
        mov     cx,256      ; a tábla hossza
REP     stos     table      ; table ← minden karakter hibás

        mov     al,c_b      ; betű kódja
        mov     di,'A'      ; az első nagybetű ASCII kódja
        add     di,bx        ; az A betű helyének offset címe
        mov     cx,'Z'-'A'+1 ; a nagybetűk száma
; kihasználjuk, hogy a betűk ASCII kódja folyamatos
REP     stosb

        mov     di,'a'      ; az első kisbetű ASCII kódja
        add     di,bx        ; az a betű helyének offset címe
        mov     cx,'z'-'a'+1 ; a kisbetűk száma
REP     stosb

        mov     al,c_n      ; számjegy kódja
        mov     di,'0'      ; az első számjegy ASCII kódja
        add     di,bx        ; a 0 helyének offset címe
        mov     cx,'9'-'0'+1 ; a számjegyek száma
; kihasználjuk, hogy a számjegyek ASCII kódja folyamatos
REP     stosb

        mov     si,offset specjel ; speciális jelek
                                ; feldolgozása
        xor     ah,ah        ; ki fogjuk használni,
                                ; hogy AX=AL
pr1:     lods     specjel      ; speciális jelek olvasása
        mov     di,ax         ; ah=0 miatt ax = a spec. jel
        cmp     al,vegjel     ; vegjel közvetlenül a
                                ; speciális jelek után!
        je      pr2           ; ez már a vegjel
        mov     al,c_s        ; speciális karakter kódja
        mov     [BX+DI],al    ; elhelyezés a táblában
        loop    pr1           ; ciklus vége

pr2:     mov     al,c_v        ; a végjel kódja
        mov     [BX+DI],al    ; elhelyezés a táblában
        ret                     ; vissza a hívó eljáráshoz
prepare  endp

; -----
parsing  proc                ; elemzés
; az eljárás rontja AX, BX, CX, DI, SI tartalmát
        cld                     ; a string műveletek iránya pozitív
        mov     bx,offset table
        mov     di,offset t_s ; kezdetben is
                                ; spec. jel szint
lv1:     mov     level,di      ; szint beállítás
        xor     ah,ah          ; kihasználjuk, hogy AX=AL
OK:      lods     text         ; a következő karakter
        XLAT                     ; AL ← 0, 2, 4, 6 vagy 8
        MOV     DI,LEVEL      ; DI ← az akt. ugró tábla címe

```



```

        ADD    DI,AX          ; DI ← a megfelelő elem címe
        JMP    [DI]          ; kapcsoló utasítás

hiba:   mov     di,offset t_s      ; hibás karakter,
                                   ; spec. jel szint

lv2:    mov     al,'?'
        mov     ah,14             ; BIOS hívás előkészítése
        int     10h              ; BIOS hívás:
                                   ; karakter írás a képernyőre
        jmp     lv1

lev_a:   mov     di,offset t_a      ; azonosító kezdődik
        mov     al,'A'
        jmp     lv2

lev_n:   mov     di,offset t_n      ; szám kezdődik
        mov     al,'0'
        jmp     lv2

lev_s:   mov     di,offset t_s      ; speciális jel
        mov     al,',',''
        jmp     lv2

vege:    mov     al,'.'           ; szöveg vége
        mov     ah,14             ; BIOS hívás előkészítése
        int     10h              ; BIOS hívás:
                                   ; karakter írás a képernyőre
        ret                     ; elemzés vége, vissza a hívóhoz

parsing endp
; -----
CODE     ends
; =====
STACK    segment      para stack 'STACK'
        dw      100 dup (?) ; 100 word legyen a verem
STACK    ends
        end      Lex      ; modul vége, start cím: Lex

```

4.1.1 Feladatok

1. Módosítsuk a fenti elemzőt úgy, hogy készítsen táblázatot (listát vagy bináris fát) a szövegben előforduló azonosítókából.

5. UTASÍTÁS RENDSZER 2. RÉSZ

5.1 Logikai utasítások

Bitenkénti logikai műveleteket végeznek. 1 reprezentálja az igaz, 0 a hamis logikai értéket.

AND	op1,op2	; op1 $\leftarrow$ op1 & op2, bitenkénti és
TEST	op1,op2	; flag-ek op1 & op2 szerint
OR	op1,op2	; op1 $\leftarrow$ op1 op2, bitenkénti vagy
XOR	op1,op2	; op1 $\leftarrow$ op1 ^ op2 (eXclusive OR), ; bitenkénti kizáró vagy
NOT	op	; op $\leftarrow$ -op, bitenkénti negáció

Felhívjuk a figyelmet arra, hogy NOT nem módosítja STATUS tartalmát. A többi logikai utasítás az aritmetikai flag-ek értékét az eredménytől függően állítja be (a c és o flag értéke 0 lesz, hiszen nem következhet be sem előjeles, sem előjel nélküli túlsordulás).

5.1.1 Feladatok

1. Írjunk program részletet a bitenkénti implikáció és ekvivalencia logikai műveletekre!
2. Írjunk program részletet xor felhasználásával, melynek hatása
Megegyezik
NOT AX
hatásával!
3. Írjunk program részletet xor, felhasználásával, melynek hatása
megegyezik
MOV AX, BX
hatásával!
4. Írjunk program részletet xor, felhasználásával, melynek hatása
megegyezik
XCHG AX, BX
hatásával! (1. cx értéke elromolhat, 2. csak AX és BX használható)

5.2 Bit forgató (Rotate) és léptető (Shift) utasítások

Az alábbi utasítások forgatják (Rotate) vagy léptetik (Shift) az első címen megadott op tartalmát. A forgatás/léptetés történhet 1 bittel, vagy byte illetve word esetén a CL regiszter alsó 3 illetve 4 bit-jén megadott bittel jobbra (Right) vagy balra (Left). Az utoljára kilépő bit lesz a Carry új tartalma.

A rotálás történhet a Carry-n keresztül, ilyenkor a belépő bit a Carry-ből kapja az értékét:

```
RCR    op,1/CL    ; Rotate through Carry Right
RCL    op,1/CL    ; Rotate through Carry Left
```

vagy úgy, hogy a Carry csak a kilépő bitet fogadja, ilyenkor a belépő bit értékét a kilépő bit szolgáltatja:

```
ROR    op,1/CL    ; Rotate Right
ROL    op,1/CL    ; Rotate Left
```

Léptetésnél a kilépő bit csak a Carry-be kerül. A belépő balra léptetéskor mindig 0, jobbra léptetéskor lehet 0 (logikai léptetés):

```
SHR    op,1/CL    ; SHift Right
SHL    op,1/CL    ; SHift Left
```

vagy lehet op előjele (aritmetikai léptetés):

```
SAR    op,1/CL    ; Shift Arithmetical Right
SAL    op,1/CL    ; Shift Arithmetical Left
```

Megjegyezzük, hogy a SHL és a SAL mnemonic ugyanazt a kódot jelöli.

A léptetések 2 hatványával való szorzásra (SHL, SAL) és osztásra (SHR, SAR) használhatók, előjel nélküli egészek esetén SHL és SHR, előjeles egészek esetén SAL és SAR.

5.2.1 Feladatok

1. Tanulmányozzuk át részletesen a 3.3 fejezet programjában a `hexa`, `hexa_b` és `h_jegy` eljárást!
2. Írjunk program részletet, amely kiszámítja, `AX` hány bitje 1-es!
3. Írjunk program részletet, amely kiszámítja, `AX` hány bitje 1-es, ha `AX` tartalma elromolhat!
4. Írjunk eljárást a gyors hatványozásra (a kitevő bináris alakját felhasználva, a megfelelő 2^k kitevőjű hatványok összeszorzásával)!

5.3 Processzor vezérlő utasítások

A processzor állapotát módosít(hat)ják.

A STATUS bitjeinek módosítása			
flag	Clear	SeT	CoMplement
C	CLC	STC	CMC
D	CLD	STD	
I	CLI	STI	

`NOP` ; NO oPeration, üres utasítás,
; nem végez műveletet.

A `NOP` utasítás végrehajtása is időt igényel, ezért alkalmas lehet arra, hogy a program futási idejét megnövelje, pl. várakozási ciklus belsejében. Az assembler pl. olyankor használja hely kitöltésre, amikor (a második menetben) kiderül, hogy a tervezettnél kevesebb byte-ot igénylő (és gyorsabb) gépi utasításra fordítható az assembly utasítás.

`WAIT` ; A processzor külső megszakításig
; várakozik.

Nem a legjobb megoldás várakozásra, mert pl. multiprogramozásos környezetben egy másik program esetleg ki tudná használni a processzor idejét.

`HLT` ; HaLT, leállítja a processzort.

Normális körülmények között a `HLT` utasítás használatára nincs szükség.

5.4 Input, output (I/O) utasítások

A külvilággal történő információ csere port-okon (kapukon) keresztül zajlik. A kapu egy memória cím, az információ csere erre a címre történő írással, vagy erről a címről való olvasással történik. Egy-egy cím vagy cím csoport egy-egy perifériához kötődik. A központi egység oldaláról a folyamat egységesen az IN (input) és az OUT (output) utasítással történik. A periféria oldaláról a helyzet nem ilyen egyszerű. Az input információ „csomagokban” érkezik, az output információt „csomagolva” kell küldeni. A csomagolás (vezérlő információ) mondja meg, hogy hogyan kell kezelni a csomagba rejtett információt (adatot). Éppen ezért az operációs rendszerek olyan egyszerűen használható eljárásokat tartalmaznak (BIOS rutinok, stb.), amelyek elvégzik a ki- és becsomagolás munkáját, és ezáltal lényegesen megkönnyítik a külvilággal való kommunikációt.

A perifériától függ, hogy a hozzá tartozó port 8 vagy 16 bites. A központi egységnek az AL illetve AX regisztere vesz részt a kommunikációban. A port címezése 8 bites közvetlen adattal vagy a DX regiszterrel történik.

```
IN      AL/AX, port ; AL/AX  $\Leftarrow$  egy byte/szó a port-ról  
OUT     port, AL/AX ; port  $\Leftarrow$  egy byte/szó AL/AX-ből
```

Az IN és OUT utasításokra nem adunk gyakorló feladatokat. Az információ csere perifériától függő címének és protokolljának tökéletes ismerete nélkül írt hibás program akár hardver rongálást is eredményezhet.

5.5 Megszakítás rendszer, interrupt utasítások

A számítógépek fejlődésének korai szakaszában az I/O utasítások elvégzéséig a központi egység várakozott. Ezen később úgy segítettek, hogy a perifériákat (egyszerűség kedvéért gondoljunk egy karakter nyomtatóra) regiszterrel (port) látták el, és miután a port és a központi egység között megtörtént az információ átadás – ez „elektronikus” sebességgel zajlott – a periféria autonóm módon elvégezte a feladatát, és eközben a központi egység tovább dolgozhatott. Előfordult, hogy a központi egység újra a

perifériához fordult, még mielőtt az befejezte volna a korábbi feladatát. Ilyenkor a központi egység várakozni kényszerült.

További fejlődést jelentett, amikor lehetővé vált, hogy a központi egység ellenőrizze, vajon elkészült-e a periféria a feladatával. Ezáltal lehetővé vált, hogy a futó program időről időre megkérdezze a periféria állapotát, és csak akkor adjon ki újabb I/O utasítást, amikor a periféria már fogadni tudta (pollozósos technika). Ennek a lehetőségnek a hatékony kihasználása mindig az éppen futó programtól függött. Ezen a nehézségen segített a megszakítási rendszer bevezetése.

A (program) megszakítás azt jelenti, hogy az éppen futó program végrehajtása átmenetileg megszakad – a processzor állapota megőrződik, hogy a program egy későbbi időpontban folytatódhassék – és a processzor egy másik program, az úgynevezett megszakító rutin végrehajtását kezdi meg. A megszakító rutin, miután elvégezte munkáját, gondoskodik a processzor megszakításkori állapotának visszaállításáról, és visszaadja a vezérlést a megszakított programnak.

A megszakítási rendszer lehetővé tette pl. a nyomtatás oly módon történő szervezését, hogy a program a tényleges nyomtatás helyett csak pufferbe helyezze a nyomtatandó anyagot, majd – amikor már minden összegyűlt – elindítsa a tényleges nyomtatást végző programot. Ez úgy működik, hogy előkészíti a nyomtatást, majd megnyitja a nyomtatót és kiad egy WAIT utasítást, ennek hatására a processzor várakozó állapotba kerül. A továbbiak során a nyomtató megszakítást okoz, ha kész újabb karakter nyomtatására. A megszakító rutint úgy kell megírni, hogy amíg van nyomtatandó anyag, addig a következő adatot nyomtassa, ha már nincs, akkor zárja le a nyomtatót, és módosítsa a visszatérési címet, hogy a WAIT utáni utasítás kapja meg a vezérlést.

Ez a megoldás azt eredményezi, hogy viszonylag hosszú ideig a processzor csak a WAIT utasítást és a megszakító rutint hajtja végre. Jobb megoldás, ha a WAIT utasítás helyett az operációs rendszer fölfüggeszti a nyomtató program működését, és elindítja egy másik program futását. Ez a módszer vezetett a multiprogramozás kialakulásához.

Anélkül, hogy nagyon mélyen belemennénk a megszakítási rendszer tanulmányozásába néhány kérdést érdemes megvizsgálnunk.

Probléma: Nem látszik célszerűnek, hogy a megszakító rutin is megszakítható legyen, mert az újabb megszakítás elrontaná a visszatérési címet, a már beállított értékeket, stb. A verem sem tökéletes megoldás, mert

valószínűleg a korábbi megszakításhoz tartozó tevékenységet kell előbb végrehajtani.

Megoldás: A központi egység két állapota: „alap” állapot – lehet megszakítás, „megszakítási” állapot – nem lehet újabb megszakítás. Ennél a radikális megoldásnál jobb, ha a megszakítási okokat hierarchiába soroljuk: megszakítási állapotban csak magasabb szintű ok eredményezhet megszakítást.

Általánosítás: A központi egységnek több állapota van, és bizonyos utasítások csak a központi egység bizonyos kitüntetett (privilegizált) állapotában hajthatók végre, alap állapotban nem.

Megoldás: Szoftver megszakítás. Az utasítások bizonyos csoportja, a memória meghatározott részei, a hardver egyes elemei csak a processzor kitüntetett állapotában érhetők el. A processzor (szoftver) megszakítást eredményező utasítással juttatható a kívánt állapotba. Ilyenkor az operációs rendszer egy meghatározott programja kapja meg a vezérlést. Ennek módjában áll a kívánság „jogosságának” ellenőrzése. Jogtalan kívánság, vagy nem kitüntetett állapotban történő kísérlet esetén – ami szintén megszakítást eredményezhet, tehát szintén az operációs rendszer kapja meg a vezérlést – az operációs rendszernek módjában áll a program további működését beszüntetni.

Következmény: Megoldható az operációs rendszer védelme, a tár védelem stb. Sajnos a gyakorlatban a megoldások általában nem tökéletesek, a biztonság nem teljes, ezt bizonyítják a számítógépes vírusok is.

Több ok (pl. többféle periféria) okozhat megszakítást. Az Intel 8088 processzor esetén a megszakítási okokat 0 és 255 közé eső i számmal jellemezhetjük. Az i . megszakítási okhoz tartozó megszakító rutin FAR címe a memória $4 \cdot i$ címén található. A megszakítási ok sorszámát hardver megszakítás esetén a hardver egység installáláskor adott száma, szoftver megszakítás esetén az operandus rész szolgáltatja.

Megszakítási ok jelentkezésekor – ha az itt nem részletezett megszakíthatósági feltételek is teljesülnek – a STATUS, CS és IP a verembe kerül, az τ és a τ flag 0 értéket kap (az úgynevezett maszkolható megszakítások tiltása és folyamatos üzemmód beállítása), majd (CS:IP) felveszi a megszakító rutin címét.

Interrupt utasítások: szoftver megszakítást eredményeznek

INT i ; $0 \leq i \leq 255$, megszakítás az i. ok szerint

Megjegyezzük, hogy az INT 3 utasítás kódja csak egy byte (a többi 2 byte), így különösen alkalmas nyomkövető (DEBUG) programokban történő alkalmazásra. A DEBUG program saját magához irányítja a 3-as megszakítást. Ezután már csak az a feladata, hogy az ellenőrzendő program megfelelő pontján (törés pont, break point) lévő utasítást átmenetileg az INT 3 utasításra cserélje, és átadhatja a vezérlést az ellenőrzendő programnak. Amikor a program az INT 3 utasításhoz ér, a DEBUG kapja meg a vezérlést. Kiírja a regiszterek tartalmát, visszaírja azt a tartalmat, amit INT 3 -ra cserélt, elhelyezi az újabb törés pontra az INT 3 utasítást és visszaadja a vezérlést az ellenőrzendő programnak. Ezek alapján érthetővé válik a DEBUG program néhány „furcsasága”, pl. miért „felejt el” a töréspontot (ha ugyanis nem felejtene el – azaz nem cserélné vissza a töréspontra elhelyezett utasítást az eredeti utasításra – akkor nem tudna továbblépni a program nyomkövetésében), miért nem lehet egy ciklus futásait egyetlen töréspont elhelyezésével figyeltetni, stb.

INTO ; megszakítás csak 0=1 (Overflow) esetén
 ; a 4. ok szerint

Visszatérés a megszakító rutinból:

IRET ; IP, CS, STATUS feltöltése a veremből

5.5.1 Szemafor

Multiprogramozásos környezetben gyakran előfordul, hogy ugyanazt az erőforrást (pl. nyomtató berendezést) egyidejűleg több program is szeretné használni. Erre természetesen nincs lehetőség, ezért biztosítani kell, hogy egyidejűleg legfeljebb egy program használhassa az illető erőforrást. Szemafornak nevezzük azt a jelzést, amely megmutatja, hogy az erőforrást használja-e pillanatnyilag valamely program.

Legyen az s szemafor egy olyan word típusú változó, amely mindegyik program számára elérhető. Jelentse $s=0$ azt, hogy az erőforrás szabad, és $s \neq 0$ azt, hogy az erőforrás foglalt. Próbáljuk meg a szemafor kezelését!

```

; 1. kísérlet
ujra:      mov    cx,S
           jcxz   szabad
           ...    ; foglalt az erőforrás, várakozás
           jmp    újra
szabad:    mov    cx,0ffffh
           mov    S,cx ; a program lefoglalta az erőforrást

```

Sajnos ez a kísérlet nem ad megfelelő megoldást. Tegyük fel hogy $s=0$! Ha az

```
ujra:      mov    cx,S
```

utasítás végrehajtása után, de még a

```
mov    S,cx
```

utasítás végrehajtása előtt megszakítás következik be, akkor elképzelhető, hogy az operációs rendszer egy másik program futtatását engedélyezi, hogy ez a másik program is használni akarja ezt az erőforrást. Tekintve, hogy még $s=0$, meg is kapja. Ha később újra futhat a korábban megszakított program, akkor az is úgy látja, hogy szabad az erőforrás, hiszen s -et már betöltötte cx -be, és a másik program csak ezután módosította s -et.

Jobb megoldás a következő:

```

ujra:      MOV     CX,0FFFFH
           XCHG    CX,S ; már foglaltat jelez a szemafor!
           JCXZ    szabad ; ellenőrzés S korábbi
                           ; tartalma szerint
           ...    ; foglalt volt az erőforrás, várakozás
           jmp     újra
szabad:    ...    ; szabad volt az erőforrás,
                           ; de a szemafor már foglalt

```

Ezzel a megoldással már csak több processzoros rendszer esetén kerülhetünk bajba. Az utasítás mikro program szinten pl. így működhet (valójában nem így működik, de az nem változtat a lényegen):

```

Segéd regiszter  $\leftarrow$  S
S  $\leftarrow$  CX
CX  $\leftarrow$  Segéd regiszter

```

Több processzoros rendszer esetén elképzelhető, hogy az egyik processzor éppen végrehajtotta az első mikroutasítást, amikor egy másik processzor is ugyanezt az utasítást kívánja végrehajtani. Ekkor ugyanúgy, mint az előző kísérletnél mindkét program szabadnak fogja találni a szemaforot. A korrekt megoldást a LOCK prefixum segítségével tudjuk megadni.

```

ujra:      mov     cx,0ffffh
          LOCK  xchg  cx,S   ; S már foglaltat jelez
          jcxz  szabad      ; ellenőrzés S korábbi
                                ; tartalma szerint
          ...              ; foglalt az erőforrás, várakozás
          jmp   újra
szabad:    ...              ; használható az erőforrás,
                                ; de a szemafor már foglalt

```

Amíg a LOCK prefixumot követő utasítás be nem fejeződött, addig másik processzor nem férhet hozzá az utasítás cél operandusához, jelen esetben S-hez, tehát a LOCK prefixum hatására a másik processzor nem tudja megkezdeni az XCHG utasítás végrehajtását – nem fér hozzá a szemaforhoz, amíg az első processzor be nem fejezte az utasítás végrehajtását – a szemafor foglalt állítását.

Ha a programnak nincs szüksége tovább a korábban lefoglalt erőforrásra, akkor ezt a szemafor szabadra állításával jelezheti. A szemafor szabadra állítása a közönséges

```
MOV    S,0
```

utasítással történhet.

Megjegyzés: Bizonyos processzorok esetén automatikusan létrejön a LOCK prefixumnak megfelelő hatás az XCHG utasítás végrehajtásakor akkor is, ha nem írtuk ki a LOCK prefixumot.

5.5.2 Feladatok

1. v egy több processzus által módosítható és olvasható változó. Írjuk meg a módosító és olvasó eljárásokat szemafor segítségével úgy, hogy módosítás közben más processzus számára v se módosítható se olvasható ne legyen!

6. PSZEUDO UTASÍTÁSOK

A pszeudo utasítások nem valódi utasítások abban az értelemben, hogy ezeket a lefordított program a futás során nem fogja végrehajtani (ilyenek pl. az adat definíciós utasítások), bizonyos esetekben nem is kerülnek bele a lefordított programba (pl. struktúra definíció). A pszeudo utasításokat a fordítóprogram hajtja végre. Ez a végrehajtás fordítás közbeni tevékenységet vagy a fordításhoz szükséges információ gyűjtést jelenthet.

6.1 Adat definíciós utasítások

Az adatokat általában külön szegmensben szokás és javasolt definiálni. A definiálás történhet iniciálással vagy anélkül. A kezdőértékkel ellátott változók a program betöltésekor a megadott értékkel rendelkeznek, de a program futása során természetesen változhatnak. Erre különösen akkor kell tekintettel lennünk, ha a programot egyszeri betöltés után többször is el akarjuk indítani. Ha a változók kezdőértéke módosult, akkor elképzelhető, hogy az ismételt indítások után eltérően fog viselkedni.

Az adat definíciós utasítások elé általában azonosítót (változó név) írunk, hogy hivatkozhatunk az illető adatra. Egy-egy adat definíciós utasítással – vesszővel elválasztva – több azonos típusú adatot is definiálhatunk. A kezdőérték – megfelelő típusú – tetszőleges konstans (szám, szöveg, cím, ...) kifejezés lehet. Ha nem akarunk kezdőértéket adni, akkor ? -et kell írunk. A szöveg és karakter konstansokat aposztrofok (') vagy idézőjelek (") között adhatjuk meg.

A kifejezés fogalmával a későbbiek során fogunk megismerkedni, itt elégedjünk meg azzal, hogy a kifejezés értékét fordítási időben kell meghatározni, és a fordítóprogram a kifejezés helyére a kifejezés értékét fogja beírni. (A végső, végrehajtáskori értéket címek esetében sokszor csak a szerkesztő vagy a betöltő tudja kiszámítani és elhelyezni.)

Az operandus részen a

kifejezés DUP (adat)

szerkezettel érhető el, hogy adat kifejezés-szer kerüljön elhelyezésre. adat lehet egy kifejezés, vagy vesszővel elválasztott több kifejezés.

6.1.1 Egyszerű adat definíciós utasítások

Define Byte (DB): Az operandus lehet numerikus, karakter vagy szöveg. Pl.:

```
Adat1      db      25      ; 1 byte, kezdőértéke decimális 25
Adat2      db      25H     ; 1 byte, kezdőértéke hexadec. 25
Adat2      db      1,2     ; 2 byte (nem egy szó!)
Adat3      db      5 dup (?) ; 5 inicializálatlan byte
Kar        db      'a','b','c' ; 3 ASCII kódú karakter
Szoveg     db      "Ez egy szöveg",13,0AH ; ASCII kódú szöveg
                                           ; és 2 szám
Szoveg1    db      'Ez is "szöveg"', "és ez is 'szöveg'"
```

Természetesen a fenti adatmegadási módok vegyesen is előfordulhatnak egy utasításban.

Define Word (DW): Az operandus lehet numerikus, cím, karakter vagy 2 karakteres szöveg. Pl.:

```
Szo        dw      0742H,452
Szo_cime   dw      szo    ; Szo offset címe
```

Define Double (DD): Az operandus lehet numerikus, távoli cím (segment és offset cím), karakter vagy maximum 4 karakteres szöveg. Tipikus alkalmazásai: Távoli címek tárolására a string kezelő utasítások előkészítéséhez az LDS és az LES utasítás operandusaként, vagy távoli eljárások indirekt hívásához. Pl.:

```
Szo_f      dd      szo     ; Szo távoli (segment + offset) címe
```

Define Quadword (DQ): dupla pontosságú (4 szavas) lebegőpontos számok, változók számára történő helyfoglalásra használatos.

Define Ten bytes (DT): 10 byte-on tárolt 10 jegyű pakolatlan vagy 20 jegyű pakolt decimális számok, változók számára történő helyfoglalásra használatos.

6.1.2 Összetett adat definíciós utasítások

Programozás során sokszor előnyös, ha több – esetleg eltérő típusú – adatot egységesen tudunk kezelni. Assembly-ben ezt a lehetőséget a struktúra és a rekord biztosítja. E két összetett adat típusban közös, hogy először magát a típust kell definiálni. Maga a típus definíció nem jelent helyfoglalást. A struktúra illetve rekord konkrét példányai struktúra illetve rekord hívással definiálhatók. A struktúra illetve rekord elemi részeit mezőknek (field) nevezzük.

6.1.2.1 Struktúra

Struktúra definíció: Csak a struktúra típusát definiálja a későbbi struktúra hívások számára, ezért a memóriában nem jár helyfoglalással.

```
Str_típus   STRUC ; struktúra (típus) definíció
...         ; mező (field) definíciók:
            ; egyszerű adat definíciós utasítások
Str_típus   ENDS ; struktúra definíció vége
```

Hangsúlyozzuk, hogy a mező (field) definíció csak egyszerű adat definíciós utasítással történhet, ezért – a magasabb szintű programozási nyelveknél szokásostól eltérően – struktúra mező nem lehet másik struktúra vagy rekord.

A mezők definiálásakor megadott értékek kezdőértékül szolgálnak a későbbiekben történő struktúra hívásokhoz. A definícióban megadott kezdőértékek közül azoknak a mezőknek a kezdőértéke híváskor felülbírálható, amelyek csak egyetlen adatot tartalmaznak (ilyen értelemben a szöveg konstans egyetlen adatnak minősül). Pl.:

```
S          STRUC ; struktúra (típus) definíció
F1         db     1,2          ; nem lehet felülírni
F2         db     10 dup (?)    ; nem lehet felülírni
F3         db     5            ; felülírható
F4         db     'a','b','c'   ; nem lehet felülírni, de
F5         db     'abc'         ; felülírható
S          ENDS
```


Struktúra hívás: A struktúra definíciójánál megadott `str_típus` névnek a műveleti kód részen történő szerepeltetésével hozhatunk létre a definíciónak megfelelő típusú struktúra változókat. A kezdőértékek fölülbírása a kívánt értékek $\diamond$ közötti felsorolásával történik. Az egyes mezők kezdőértékeit vesszővel választjuk el. Ha egy mezőnek a kezdőértékét nem kívánjuk megadni, vagy nem adhatjuk meg, akkor is ki kell írjuk a vesszőt, kivéve, ha a további mezők egyikének sem kívánunk a definícióban megadottól eltérő kezdőértéket adni. Az alábbi struktúra hívások `s` típusú struktúra példányokat állítanak elő:

```
S1          S          ; kezdőértékek a definícióból
S2          S <,,7,, 'FG'> ; F3 kezdőértéke 7, F5-é 'FG'
S3          S <,, 'A'>    ; F3 kezdőértéke 'A' ,
                        ; F5-é a definícióból
```

Struktúrából vektort is előállíthatunk, pl.:

```
S_vektor    S 8 dup (<,, 'A'>) ; 8 elemű struktúra vektor
```

Struktúra mezőre hivatkozás: A struktúra változó névhez tartozó `OFFSET` cím – értelemszerűen – a struktúra `OFFSET` címét, míg a mező név a struktúrán belüli címet jelenti. A struktúra adott mezejére úgy hivatkozhatunk, hogy a struktúra és mező név közé `.`-ot írunk, pl.:

```
MOV  AL, S1.F3
```

A `.` bármely oldalán lehet másfajta cím is, pl.

```
MOV  BX, OFFSET S1
```

után az alábbi utasítások mind ekvivalensek az előzővel:

```
MOV  AL, [BX].F3
MOV  AL, [BX]+F3
MOV  AL, F3.[BX]
MOV  AL, F3[BX]
```

A fentiekből az is következik, hogy a mező nevek – ellentétben a magasabb szintű programozási nyelvekkel – szükségképpen egyedi nevek, tehát sem másik struktúra definícióban, sem közönséges változóként nem szerepelhet.

A struktúra vektorokat a hagyományos módon még akkor sem indexezhetjük, ha az index konstans. Ha pl. 5, akkor

```
MOV    AL,S_vektor[5].F3      ; szintaktikusan helyes
```

de [5] nem a vektor ötödik elemére mutató címet fogja eredményezni, csupán 5 byte-tal magasabb címet, mint S_vektor.F3. Ha i változó, akkor

```
MOV    AL,S_vektor[i].F3      ; szintaktikusan is HIBÁS!
```

Mindkét esetben programmal kell kiszámíttatni az elem offset-jét, pl.:

```
MOV    AX,TYPE S      ; S hossza byte-okban (l. később)
MUL    i              ; ha i word. Az indexet 0-tól számoljuk
MOV    BX,AX           ; az adat nem „lőghat ki” a szegmensből
MOV    AL,S_vektor.F3[BX] ; AL←az i-dik elem F3 mezeje.
```

6.1.2.2 Rekord

Rekordon olyan 1 vagy 2 byte-os összetett adatot értünk, melynek meghatározott szomszédos bit sorozatai logikai egységet alkotnak. Ezeket a logikai egységeket mezőknek (field) nevezzük. Ez a fogalom eltér pl. a PL/I rekord fogalmától, ahol a rekord az assembly struktúra fogalmának általánosítása. Minden olyan esetben hasznunkra válhat a rekord, amikor egy vagy két byte-os adatokat bit csoportonként kell kezelnünk, ami leggyakrabban hardver közeli programok – pl. driver-ek – írásakor fordul elő.

Rekord definíció: Csak a rekord típusát definiálja a későbbi rekord hívások számára, ezért a memóriában nem jár helyfoglalással.

```
Rec_típus    RECORD      mező specifikációk
```

ahol az egyes mező specifikációkat , -vel választjuk el egymástól. Maga a mező specifikáció:

```
mező_név:    szélesség=kezdőérték
```

a szélesség a mező bit-jeinek száma. Az =kezdőérték el is maradhat, pl.:

```
R          RECORD      X:3,Y:4=15,Z:5
```

Az R rekord szavas (12 bit), a következőképpen helyezkedik el egy szóban:

				X	X	X	Y	Y	Y	Y	Z	Z	Z	Z	Z
--	--	--	--	---	---	---	---	---	---	---	---	---	---	---	---

Rekord hívás: A rekord definíciójánál megadott Rec_típus névnek a műveleti kód részen történő szerepeltetésével hozhatunk létre a definíciónak megfelelő típusú rekord változókat. A kezdőértékek fölülbírálása a kívánt értékek <> közötti felsorolásával történik. Az egyes mezők kezdőértékeit vesszővel választjuk el. Ha egy mezőnek a kezdőértékét nem kívánjuk megadni, akkor is ki kell írjuk a vesszőt, kivéve, ha a további mezők egyikének sem kívánunk a definícióban megadottól eltérő kezdőértéket adni. Az alábbi rekord hívások R típusú rekord példányokat állítanak elő:

```

R1    R    < >          ; 01E0H, kezdőértékek a definícióból
R2    R    <,,7>        ; 01E7H, X, Y kezdőértéke a
                        ; definícióból, Z-é 7
R3    R    <1,2>        ; 0240H, X kezdőértéke 1, Y-é 2,
                        ; Z-é a definícióból

```

Rekordból vektort is előállíthatunk, pl.:

```

R_vektor    R 5 dup (<1,2,3>) ; 0243H, 5 elemű rekord vektor

```

Rekord mezőre hivatkozás: A hardver nem támogatja a mezők elérését, de az assembler nyújt némi segítséget. A mező név olyan konstansként használható, amely azt mondja meg, hány bittel kell jobbra léptetnünk a rekordot, hogy a kérdéses mező az 1-es helyértékre kerüljön. A fenti példánknál maradva: X=9, Y=5 és Z=0. További segítség a MASK operátor, melyet a mező név elé írva olyan konstans kapunk, amely az illető mezőhöz tartozó biteken 1, a többi biten 0, NOT MASK ennek a bitenkénti negáltja. Ezek segítségével az R3 rekord Y mezejét a következő program részlettel tudjuk AX-be írni:

```

MOV    AX,R3          ; R3 szavas rekord!
AND    AX,MASK Y      ; Y mezőhöz tartozó bitek maszkolása
MOV    CL,Y           ; léptetés előkészítése
SHR    AX,CL          ; kész vagyunk. SAR nem lenne korrekt

```

6.1.3 Feladatok

1. Definiáljunk egy struktúrát, amely név, születési év, stb. adatokat tartalmaz. Legyen egy n elemű struktúra vektorunk. Határozzuk meg a vektorban szereplő személyek átlag életkorát!
2. Írjunk program részletet, amely egy rekord adott mezejének tartalmát eggyel növeli (ha a megnövelt érték nem ábrázolható a mezőben, akkor 0-t írunk bele)!
3. Írjunk program részletet, amely egy rekord adott mezejének tartalmát bitenként negálja!

6.2 Kifejezés

Egy művelet operandusa lehet konstans, szimbólum vagy kifejezés.

6.2.1 Konstans

A konstans lehet numerikus vagy szöveg konstans.

A numerikus konstansok decimális, hexadecimális, oktális és bináris számrendszerben adhatók meg. A számrendszert a szám végére írt D, H, O illetve B betűvel választhatjuk ki. Ha nem írunk betűt a szám végére, akkor a számot az alapértelmezésnek megfelelő számrendszerben kell érteni. Az alapértelmezést (ami általában a decimális) a

.RADIX n

pszeudo utasítással állíthatjuk be, ahol n a beállítani kívánt számrendszer alapszáma.

Hexadecimális számok esetén ügyelnünk kell arra, hogy a szám 0 és 9 közé eső számjeggyel kezdődjön, különben a fordító program azonosítónak fogja tekinteni.

A szöveg konstansokat a DB utasításban " vagy ' jelek között adhatjuk meg.

6.2.2 Szimbólum

A szimbólum lehet szimbolikus konstans, változó név vagy címke.

Szimbolikus konstans: Az = vagy az EQU pszeudo utasítással definiálható. Az = illetve EQU jobb oldalán kifejezés állhat. Az =-vel definiált konstans értéke másik =-vel megváltoztatható, az EQU-val definiálté nem. Szimbolikus szöveg konstans csak EQU-val definiálható. A szimbolikus konstans a program szövegnek a definíciót követő részében használható, értékét a használat helyét megelőző utolsó definíciója határozza meg. Pl.:

```
S      =      1      ; S értéke 1
N      EQU    14      ; N értéke 14
      MOV     CX,N     ; CX ← 14
ISM:
S      =      S+1     ; S értéke ezután 2, függetlenül attól,
                      ; hogy hányadszor fut a ciklus
      MOV     AX,S     ; AX ← 2
      LOOP    ISM
N      =      5      ; hibás
N      EQU    5      ; hibás
S      =      5      ; helyes
S      EQU    5      ; hibás
```

Szimbolikus konstansként használhatjuk a \$ jelet (helyszámláló), melynek az értéke mindenkor a program adott sorának megfelelő OFFSET cím. A helyszámláló értékének módosítására az ORG utasítás szolgál, pl.:

```
ORG    $+100H      ; 100H byte kihagyása a memóriában
```

Címke: Leggyakoribb definíciója, hogy valamelyik utasítás előtt a sor első pozíciójától :-tal lezárt azonosítót írunk. Az így definiált címke NEAR típusú. Címke definícióra további lehetőséget nyújt a LABEL és a PROC pszeudo utasítás:

```
ALFA:      ...      ; NEAR típusú
BETA      LABEL FAR ; FAR típusú
GAMMA:     ...      ; BETA is ezt az utasítást címkézi,
                      ; de GAMMA NEAR típusú
```

Az eljárás deklarációt a PROC pszeudo utasítással nyitjuk meg. A címke rovatba írt azonosító az eljárás neve és egyben a belépési pontjának címkéje. Az eljárás végén az eljárás végét jelző ENDP pszeudo utasítás előtt meg kell ismételniünk ezt az azonosítót, de az ismétlés nem minősül címkének. Az

eljárás címkéje aszerint NEAR vagy FAR típusú, hogy maga az eljárás NEAR vagy FAR. Pl.:

```
A      PROC          ; NEAR típusú
      ...
B      PROC NEAR    ; NEAR típusú
      ...
C      PROC FAR     ; FAR típusú
      ...
```

Címkére vezérlés átadó utasítással hivatkozhatunk, NEAR típusúra csak az adott szegmensből, FAR típusúra más szegmensekből is.

Változó: Definíciója adat definíciós utasításokkal történik. Néha (adat) címkének is nevezik.

6.2.3 Kifejezés

A kifejezés szimbólumokból és konstansokból épül fel az alább ismertetendő műveletek segítségével. Kifejezés az operátorok, pseudo operátorok operandus részére írható. Értékét a fordítóprogram határozza meg, és a kiszámított értéket alkalmazza operandusként. Szimbólumok értékén konstansok esetében természetesen a konstans értékét, címkék, változók esetében a hozzájuk tartozó címet – és nem a tartalmat – értjük. Az érték nemcsak számérték lehet, hanem minden, ami az utasításokban megengedett címezési módok valamelyikének megfelel. Pl. [BX] is kifejezés és értéke a BX regiszterrel történő indirekt hivatkozás, és ehhez természetesen a fordító programnak nem kell ismernie BX értékét. Természetesen előfordulhat, hogy egy kifejezés egyik szintaktikus helyzetben megengedett, a másikban nem, pl.:

```
mov ax, [BX] ; [BX] megengedett
mul [BX]     ; [BX] hibás, de WORD PTR [BX] megengedett
```

Egy kifejezés akkor megengedett, ha az értéke fordítási időben meghatározható és az adott szintaktikus helyzetben alkalmazható, pl. az adott utasítás lehetséges címezési módja megengedi. A megengedett kifejezés értékeit az egyes utasítások ismertetése során megadtuk.

A műveletek, csökkenő precedencia szerinti sorrendben:

1.

- A `()` és `[]` (zárójelek): míg a `()` zárójel pár a kifejezés kiértékelésében csupán a műveletek sorrendjét befolyásolja, addig a `[]` az indirekció előírására is szolgál. Ha a `[]`-en belüli kifejezésre nem alkalmazható indirekció, akkor a `()`-lel egyenértékű;
- `.` (pont): struktúra mezőre hivatkozásnál használatos;
- `LENGTH` változó: a változó-hoz tartozó adat terület elemeinek száma;
- `SIZE` változó: a változó-hoz tartozó adat terület hossza byte-okban;
- `WIDTH R/F`: az `R` rekord vagy az `F` (rekord) mező szélessége bitekben;
- `MASK F`: az `F` (rekord) mező bitjein 1, másutt 0;

Pl.:

```
v dw      20 dup (?)
rec      record  x:3,y:4
```

esetén:

```
mov      ax,LENGTH v      ; ax ← 20
mov      ax,SIZE v        ; ax ← 40
mov      ax,WIDTH rec     ; ax ← 7
mov      ax,WIDTH x       ; ax ← 3
mov      ax,SIZE rec      ; ax ← 1
mov      ax,MASK x        ; ax ← 70H
```

2. Explicit szegmens megadás (segment override prefix). Az automatikus szegmens regiszter helyett más szegmens regiszter használatát írja elő, pl.:

```
mov      ax,ES: [BX] ; ax ← (ES:BX) címen lévő szó
```

Nem írható felül az automatikus szegmens regiszter az alábbi esetekben:

- CS program memória címezésnél,
- SS stack referens utasításokban (PUSH, POP, ...),
- ES string utasításban DI mellett, de az SI-hez tartozó DS átírható.

3.

- típus PTR cím: (típus átdefiniálás) ahol típus lehet BYTE, WORD, DWORD, QWORD, TBYTE, illetve NEAR, és FAR. Pl.:

```
MUL          BYTE PTR [BX]      ; a [BX] címet
                                   ; byte-osan kell kezelni
```

- OFFSET szimbólum: a szimbólum OFFSET címe (a szegmens kezdetétől számított távolsága byte-okban);
- SEG szimbólum: a szimbólum szegmens címe (abban az értelemben, ahogy a szegmens regiszterben szokásos tárolni, tehát valós üzemmódban a szegmens tényleges kezdőcímének 16-oda);
- TYPE változó: a változó elemeinek hossza byte-okban;
- ... THIS típus: a program szöveg adott pontján adott típusú szimbólum létrehozása;

Pl. az

```
ADATB  EQU      THIS BYTE      ; BYTE típusú változó,
                                   ; helyfoglalás nélkül
ADATW  dw       1234H          ; ez az adat ADATB-vel
                                   ; byte-osan érhető el
```

definíció esetén:

```
mov     al, BYTE PTR ADATW ; al ← 34H, helyes
mov     al, ADATB          ; al ← 34H
mov     ah, ADATB+1        ; ah ← 12H
```

Emlékeztetünk arra, hogy szavak tárolásakor az alacsonyabb helyértékű byte kerül az alacsonyabb címre!

4.

- LOW változó: egy szó alsó (alacsonyabb helyértékű) byte-ja
- HIGH változó: egy szó felső (magasabb helyértékű) byte-ja

5. Multiplikatív műveletek:

- * : szorzás
- / : osztás
- MOD: (modulo) a legkisebb nem negatív maradék, pl.:

```
mov     al, 20 MOD 16        ; al ← 4
```


- kifejezés SHL lépés: kifejezés bitenkénti léptetése balra lépés bittel
- kifejezés SHR lépés: kifejezés bitenkénti léptetése jobbra lépés bittel

Megjegyezzük, hogy lépés is lehet kifejezés.

A kifejezésben előforduló műveleti jelek (SHL, SHR, és az alacsonyabb prioritású NOT, AND, OR, és XOR) nem tévesztendőek össze a velük azonos alakú műveleti kódokkal: az előbbieket a fordító program, az utóbbiakat a futó program hajtja végre!

6. Additív műveletek:

- + : összeadás
- - : kivonás vagy negatív előjel

7. Relációs operátorok: általában feltételes fordítással kapcsolatban fordulnak elő

- EQ : =
- NE : ≠
- LT : <
- LE : ≤
- GT : >
- GE : ≥

8. NOT : bitenkénti negálás

9. AND : bitenkénti és művelet

10. Bitenkénti vagy és kizáró vagy művelet

- OR : bitenkénti vagy művelet
- XOR : bitenkénti kizáró vagy művelet

11. SHORT : 8 bites IP relatív címzés kikényszerítése

6.2.4 Feladatok

1. Milyen szimbólum lehet A és B , ha $A+B$ nem, viszont $A-B$ helyes kifejezés?
2. Lehet-e megengedett kifejezés $A+B-C$, ha $A+B$ nem megengedett?
3. Mikor lehet $A*B$ helyes kifejezés?

6.3 Feltételes fordítás

Általában előnyös, ha olyan programot tudunk írni, amely eltérő körülmények között képes dolgozni. Pl. a program fejlesztését megkönnyíti, ha a kritikus pontokon ellenőrző nyomtatásokat helyezünk el. Ez persze később, a program rendeltetésszerű használatánál nem kívánatos. Egyszerű azt mondani, hogy ki kell törölni a kész programból ezeket a nyomtatásokat, de mi van akkor, ha később a program mégis hibásnak bizonyul? Ilyenkor persze az lenne a szerencsés, ha nem töröltük volna ki! Jobb az a megoldás, ha csak kommentáljuk az ellenőrző nyomtatásokat, de még így is sok munkát jelent az ellenőrzések ki- és bekapcsolása. Másik lehetőség, hogy az ellenőrzéseket feltételesen hajtjuk végre. A program elején pl. 0-ra vagy 1-re állítjuk egy változó értékét, és csak akkor végezzük el a nyomtatást, ha a változó értéke 0. Ennek a megoldásnak az a hátránya, hogy az ellenőrzések miatt a program memória igénye jelentősen megnőhet.

A fordító programok általában – így az assembler is – feltételes fordítási lehetőséget biztosít. Ez azt jelenti, hogy a program bizonyos részeit csak abban az esetben fordítja le, ha – a fordítóprogram számára ellenőrizhető – feltétel igaz illetve hamis. A feltételes fordítás általános szerkezete a következő:

```
IFxx feltétel
...                ; lefordul, ha a feltétel igaz
ELSE               ; el is maradhat
...                ; lefordul, ha a feltétel hamis
ENDIF
```

Ahol IFxx feltétel :

```
IF      kifejezés    ; igaz, ha kifejezés≠0
IFE     kifejezés    ; igaz, ha kifejezés=0
```

IF1		; igaz a fordítás első menetében
IF2		; igaz a fordítás második menetében
IFDEF	Szimbólum	; igaz, ha Szimbólum definiált
IFNDEF	Szimbólum	; igaz, ha Szimbólum nem definiált
IFB	<arg>	; igaz, ha arg üres (blank)
IFNB	<arg>	; igaz, ha arg nem üres
IFIDN	<arg1>, <arg2>	; igaz, ha arg1=arg2
IFDIF	<arg1>, <arg2>	; igaz, ha arg1=arg2 nem teljesül

A feltételes fordítási utasításokat makró definíciókban kiterjedten fogjuk használni. Némelyikük szerepe csak ott válik világossá.

6.3.1 Feladatok

1. Írjunk program részletet, amely egy rekord adott mezőjének tartalmát eggyel növeli (ha a megnövelt érték nem ábrázolható a mezőben, akkor 0-t írunk bele)!

6.4 Makró és blokk ismétlés

6.4.1 Makró

A programban többször előforduló részfeladatok megoldására szolgáló eszközök közül eddig az eljárásokkal ismerkedtünk meg. Az eljárások előnye, hogy csak egyszer kell megírunk az illető részfeladatot megoldó eljárást, a továbbiak során elegendő a paraméterek megfelelő beállítása után felhívunk ezt az eljárást. Maga az eljárás csak egyszer szerepel a programban.

Kimutatkozik az eljárások alkalmazásának hátránya, ha az eljárás maga egyszerű, de sok paramétert használ, mert ilyenkor a paraméterek átadása összemérhető feladat magával az eljárás végrehajtásával. A paraméterek átadása minden felhíváskor terheli a memóriát és a futási időt is. Többek között ilyen esetekben nyújt hatékony megoldást a makró.

Makrók használatával kapcsolatban meg kell különböztessük a makró definíciót és a makró hívást.

Makró definíció:

M_név	MACRO [fpar1] [,fpar2] ...	; makró fej (kezdet)
...		; makró törzs
	ENDM	; makró vége

A makró fejben megadott [fpar1] [,fpar2] ...-t formális paramétereknek, vagy egyszerűen paramétereknek szokás nevezni.

A makró definíció nem lesz része a lefordított programnak, csupán azt határozza meg, hogy később mit kell a makró hívás helyére beírni (makró kifejtés, helyettesítés).

A makró törzsön belül előfordulhat makró hívás és másik makró definíció is.

Makró hívás:

M_név [apar1] [,apar2] ...

A makró hívásban megadott [apar1] [,apar2] ...-t aktuális paramétereknek vagy argumentumoknak szokás nevezni.

A műveleti kód helyére írt M_név hatására a korábban megadott definíció szerint megtörténik a makró helyettesítés, más néven makró kifejtés. Ez a makró törzs bemásolását jelenti, miközben az összes paraméter összes előfordulása a megfelelő argumentummal helyettesítődik. A helyettesítés szövegesen történik, azaz minden paraméter – mint szöveg – helyére a megfelelő argumentum – mint szöveg – kerül. A helyettesítés nem rekurzív, azaz az egyszeri helyettesítés után előállott szövegben esetleg előforduló olyan szavak helyettesítése már nem hajtodik végre, amelyek – mint szöveg – megegyeznek valamely formális paraméterrel. Makró hívás argumentuma sem lehet makró hívás, csak egyszerű szöveg, de az argumentumnak megfelelő formális paraméternek lehet olyan előfordulása, amely a későbbiek során makró hívást eredményez. A kérdéskör jobban érthetővé válik a makró generátor ismertetésénél (Lásd Makró generátor).

Példaként legyen a programban gyakran előforduló részfeladat a dupla szavas összeadás. Hasonlítsuk össze az eljáráson és a makrón alapuló megoldásokat. Az egyszerűség kedvéért először tegyük fel, hogy mindig a $(DX:AX) \leftarrow (DX:AX) + (CX:BX)$ feladatot kell végrehajtani.

Eljárás deklaráció:

```
EDADD      PROC    NEAR
            ADD     AX, BX
            ADC     DX, CX
            RET
EDADD      ENDP
```

Makró definíció:

```
MDADD      MACRO
            ADD     AX, BX
            ADC     DX, CX
            ENDM
```

Az eddigi különbség, hogy az eljárás deklaráció része lesz a lefordított programnak, a makró definíció nem.

Ha a programban valahol meg kell oldanunk a részfeladatot, akkor hívunk kell az eljárást illetve a makrót:

Eljárás hívás:

```
CALL EDADD
```

Makró hívás:

```
MDADD
```

Ennek hatása:

Futás közben felhívásra kerül az EDADD eljárás

Fordítás közben MDADD definíciója szerint, megtörténik a makró helyettesítés, és a program szöveg részévé válik:

```
ADD     AX, BX
ADC     DX, CX
```

Futás közben ez a két utasítás kerül csak végrehajtásra.

Látható, hogy eljárás esetén kettővel több utasítást kell végrehajtanunk, mint makró esetében (CALL EDADD és RET).

Még nagyobb különbséget tapasztalunk, ha (CX:BX) helyett paraméterként kívánjuk megadni az egyik összeadandót:

Eljárás deklaráció:

```
EDADD2     PROC    NEAR
            PUSH    BP
```

Makró definíció:

```
MDADD2     MACRO P
            IFB    <P>
```

	MOV	BP, SP		ADD	AX, BX
	ADD	AX, 4 [BP]		ADC	DX, CX
	ADC	DX, 6 [BP]	ELSE		
	POP	BP		ADD	AX, P
	RET	4		ADC	DX, P+2
EDADD	ENDP		ENDIF		
				ENDM	

Természetesen most sem lesz része a makró definíció a lefordított programnak.

Ha SI az összeadandónk címét tartalmazza, akkor a felhívások a következőképpen alakulnak:

PUSH	2 [SI]	MDADD2	[SI]
PUSH	[SI]		
CALL	EDADD2		

Ennek hatása:

Futás közben végrehajtásra kerül a paraméter átadás, az eljárás hívás, az eljárás: összesen 9 utasítás

Fordítás közben a makró hívás az

ADD	AX, [SI]
ADC	DX, [SI] + 2

utasításokra cserélődik, futás közben csak ez a két utasítás kerül végrehajtásra.

Megjegyzés: Az MDADD2 makrót úgy írtuk meg, hogy ha nem adunk meg paramétert, akkor az eredeti feladatot oldja meg, ekkor ugyanis az

IFB <P>

feltétel teljesül.

Paraméter másutt is előfordulhat a makró törzsben, nemcsak az operandus részen, pl.:

PL	MACRO	P1, P2
	MOV	AX, P1
	P2	P1
	ENDM	

makró definíció esetén a

```
PL      Adat, INC
```

makró hívás a

```
MOV     AX, Adat
INC     Adat
```

utasításokra cserélődik.

A &, %, ! karakterek továbbá a < és ; speciális szerepet töltenek be makró kifejtéskor.

&: Előfordulhat, hogy azt szeretnénk, hogy makró kifejtéskor a paraméter – helyettesített – értéke része legyen egy szónak. Ezt az & karakter segítségével érhetjük el. A makró generátor az & jel mindkét oldalát megvizsgálja, hogy ott paraméter szerepel-e. Ha igen, akkor elvégzi a paraméter helyettesítését, és mindenképpen elhagyja az & jelet. Pl.:

```
UGRIK      MACRO      Felt, Cimke
             J&Felt     Felt&Cimke
             ENDM
```

A hívás pl.:

```
UGRIK      Z, Alfa
```

Ennek a kifejtése:

```
JZ         Zalfa
```

?: Az utána szereplő paraméter értékének számjegyek sorozatává alakítását írja elő. Az alábbi példa a % használatán kívül a makró törzsön belüli makró hívást is bemutatja. Makró definíciók:

```
S          =      0
ErrMsg     MACRO  text
S          =      s+1
Msg        %s, text
ENDM
```

```
Msg        MACRO  sz, str
msg&sz     db     str
ENDM
```


Az

```
ErrMsg 'syntax error'
```

makró hívás hatására bemásolásra kerül az

```
s          =      s+1
Msg      %s, 'syntax error'
```

szöveg. *s* értéke itt 1-re változik. Ez a szöveg újabb makró hívást tartalmaz (*Msg*). A *%s* paraméter az értékére (1) cserélődik, majd kifejtésre kerül ez a makró is, ebből kialakul az

```
msg1      db      'syntax error'
```

adat terület foglaló utasítás. Megjegyezzük, hogy a program listában csak ez az utasítás fog szerepelni, az első helyettesítés utáni közbülső állapot nem. Egy újabb, pl.:

```
ErrMsg 'invalid operand'
```

hatása:

```
msg2      db      'invalid operand'
```

! : Az utána következő karakter makró kifejtéskor változatlanul kerül bemásolásra. Ez ad lehetőséget arra, hogy pl. az & jel a kifejtett makró szövegében szerepelhessen.

< : Ha aktuális paraméterként olyan szöveget kívánunk megadni, amely pl. , -t is tartalmaz, akkor ezt < jelek között tehetjük meg. Pl. az

```
adat      macro p
          db      p
          endm
```

makró definíció esetén az

```
adat      <'abc', 13, 10, 0>
```

hívás kifejtve:

```
db      'abc',13,10,0
```

míg ezzel szemben az

```
adat    'abc',13,10,0
```

makró hívás kifejtve:

```
db      'abc'
```

;;; A makró definíció megjegyzéseinek kezdetét jelzi. A ;; utáni megjegyzés a program listában nem jelenik meg.

Szükségünk lehet arra, hogy a makró törzs bizonyos utasításait címkézzük. Ha egy ilyen makrót többször is meghívunk, akkor a makró törzsben előforduló címkék többször fognak szerepelni a makró kifejtés után a programban, ez pedig hibajelzést eredményez. Az ilyen hiba megelőzésére szolgál a LOCAL utasítás (pszeudo utasítás, operátor).

```
LOCAL c1[,c2...]
```

c1[,c2...] minden makró híváskor más, ??xxxx alakú szimbólumra cserélődik, ahol xxxx a makró generátor által meghatározott hexadecimális szám. A LOCAL operátort közvetlenül a makró fej utáni sorba kell írni.

Lássuk példaként az alábbi kopogtató makrót! Tételezzük fel, hogy már definiáltunk egy KOPP nevű makrót, amely egy koppanást előidéző programot generál, ekkor:

```
KOPOG      macro n
            LOCAL ujra
            mov    cx,n
ujra:      KOPP
            loop   ujra
            endm
```

Ha a programban többször hívnánk a KOPOG makrót, akkor a LOCAL operátor nélkül az ujra címke többször lenne definiálva.

Makró definíció belsejében meghívható az éppen definíálás alatt lévő makró is (a makró hívás ezáltal rekurzívvá válik). Pl. a

```

PUSHALL    macro      reg1,reg2,reg3,reg4,reg5,reg6
IFNB <reg1>                ;;    ha a paraméter nem üres
    push    reg1          ;; az első regiszter mentése
    PUSHALL    reg2,reg3,reg4,reg5,reg6 ;; rekurzió
ENDIF
                ENDM

```

makró definíció lehetővé teszi, hogy egy felhívással akárhány (de legfeljebb 6) regiszter tartalmát a verembe mentjük, pl: a

```

PUSHALL    ax, bx, cx

```

makró hívás hatására a

```

PUSH    ax
PUSH    bx
PUSH    cx

```

utasítás sorozat generálódik. Ebben a példában a PUSHALL makró mindaddig generál egy PUSH utasítást az első aktuális paraméterrel, és felhívja önmagát a többi paraméterrel, ameddig a soron következő paraméter nem üres.

Makró definíció belsejében lehet másik makró definíció is. A belső makró definíciók csak a külső makró meghívása után jut érvényre, válik láthatóvá. Pl.:

```

shifts      macro      OPNAME      ; makró definíáló makró
OPNAME&S    MACRO      OPERANDUS,N
    mov     c1, N
    OPNAME  OPERANDUS,c1
ENDM
ENDM

```

Ha ezt a makró felhívjuk pl.:

```

shifts ROR

```

akkor a

```

RORS      MACRO      OPERANDUS,N
    mov    c1, N
    ROR    OPERANDUS,c1
ENDM

```

makró definíció generálódik. Mostantól meghívható a RORS makró is, pl.:

```
RORS      AX, 5
```

aminek a hatása a

```
mov       cl, 5
ROR       AX, cl
```

utasítás sorozat (a makrónk elrontja `cl` tartalmát, de ez könnyen javítható). Természetesen a többi bit léptető és forgató utasítás „kiterjesztését” ugyanígy megtehetjük a `shifts` makró segítségével.

A következő példánk azt mutatja be, hogy hogyan készíthetünk olyan makrót, amely minden felhíváskor egy eljárás hívásra cserélődik, az első hívásakor ezenkívül még a hívott eljárás is bemásolódik a program szövegbe. A makró definícióban használni fogjuk, az `EXITM` utasítást, melynek a hatására az adott makró helyettesítés befejeződik. Az `EXITM` utasítás a feltételes fordítást vezérlő utasításokat tartalmazó program írását teszi kényelmesebbé, áttekinthetőbbé.

```
FL_CALLELJ = 0

CALLELJ macro
    LOCAL FIRST ;; nem lenne fontos
IF      FL_CALLELJ    ;; a 2. hívástól igaz
    call Elj        ;; elég felhívni az eljárást
    EXITM           ;; makró helyettesítés vége
ENDIF

FL_CALLELJ = 1      ;; csak az első híváskor jut érvényre
JMP FIRST
Elj      proc        ;; eljárás deklaráció
    ...
    ret
Elj      endp
FIRST:   call Elj    ;; az eljárás felhívása
        endm
```

Az első hívás hatására a

```
Elj      JMP  ??0000
        proc
        ...
        ret
Elj      endp
??0000:  call elj
```

utasítások generálódnak (??0000 a FIRST-ből keletkezett), a további hívások esetén csak egyetlen utasítás, a

```
call elj
```

utasítás generálódik.

A megoldás előnye, hogy az eljárás csak akkor része a programnak, ha a program tartalmazza az eljárás felhívását is, és mégsem kell törődjünk azzal, hogy hozzá kell-e szerkeszteni a programhoz vagy se.

Megváltoztathatjuk egy már korábban definiált makró definícióját azáltal, hogy újra definiáljuk. Mint korábban említettük, makró definíción belül előfordulhat másik makró definíció is. E két lehetőség kombinációjából adódik, hogy egy makró definíción belül megadhatunk ugyanarra a makró névre egy másik definíciót, ezáltal készíthető olyan makró, amely „átdefiniálja” önmagát. Az önmagát „átdefiniáló” makrók esetében a belső és külső definíciót lezáró ENDM utasítások között egyetlen utasítás sem szerepelhet – még kommentár sem!

Az előző feladatot ennek segítségével is megoldhatjuk:

```
CALLELJ2    macro           ; külső makró definíció
             jmp    FIRST
Elj2        proc           ; eljárás deklaráció
             ...
             ret
Elj2        endp
FIRST:      call    Elj2    ; eljárás hívás

CALLELJ2    macro           ; belső makró definíció
             call    Elj2
             ENDM        ; belső makró definíció vége
             ENDM        ; külső makró definíció vége
```

CALLELJ2 első hívásakor a kifejtés eredménye:

```
Elj2        jmp    FIRST
             proc           ; eljárás deklaráció
             ...
             ret
Elj2        endp
FIRST:      call    Elj2    ; eljárás hívás

CALLELJ2    macro           ; belső makró definíció
             call    Elj2
             ENDM        ; belső makró definíció vége
```

A kifejtés CALLELJ2 újabb definícióját tartalmazza, ez felülírja az eredeti definíciót, és a továbbiak során ez a definíció érvényes. Ez alapján a későbbi hívások esetén

```
call Elj2
```

a kifejtés eredménye.

Megjegyezzük, hogy most is szerencsésebb lett volna a FIRST címkét lokálissá tenni. Igaz, hogy csak egyszer generálódik, de így annak, aki használni akarja a CALLELJ2 makrót, annak tudnia kell, hogy a FIRST címke már „foglalt”!

Ha egy M_név makró definíciójára nincs szükség a továbbiak során, akkor a

```
PURGE M_név
```

pszeudo utasítással kitörölhetjük.

6.4.2 Feladatok

1. Írjunk makrót, amely egy szó hosszúságú rekord adott mezejének értékét bitenként az ellenkezőjére váltja!
2. Írjunk makrót egy rekord adott mezejének kiolvasására (feltételes fordítással döntsük el, hogy a rekord byte-os vagy szavas)!
3. Írjunk makrót, amely egy rekord adott mezejét adott tartalommal tölti fel (feltételes fordítással döntsük el, hogy a rekord byte-os vagy szavas)!
4. Írjunk makrót, amely egy rekord adott mezejének tartalmát eggyel növeli (ha a megnövelt érték nem ábrázolható a mezőben, akkor 0-t írunk bele. Feltételes fordítással döntsük el, hogy a rekord byte-os vagy szavas)!
5. Írjunk olyan makrót, amelynek hívásakor a

```
db    "speciális karakterek: &, %, !, <, > és ;;"
```

adat definíciós utasítás generálódik!

6. Alakítsuk át a shifts makrót feltételes fordítás felhasználásával úgy, hogy a bit léptető és forgató utasítások kiterjesztése $N < 5$ esetén a megfelelő utasítás N -szeri ismétlését generálja!

6.4.3 Blokk ismétlés

A makrókhoz hasonló lehetőséget biztosítanak: a blokk egy definíciója alapján a blokk többszöri kifejtését. Lényeges különbség azonban, hogy ezek az operátorok nemcsak a blokk definíciójának kezdetét jelölik ki, hanem a kifejtést (hívást) is előírják. A program más részéről nem is hívhatók ezek a definíciók.

Blokk ismétlés kifejezés-szer:

```

REPT      kifejezés
...      ; ez a rész ismétlődik
ENDM

```

Pl. a korábban ismertetett kopogást így is megoldhattuk volna:

```

REPT      N
KOPP
ENDM

```

Ha pl. $N=3$, akkor ennek a hatására a

```

KOPP
KOPP
KOPP

```

makró hívások generálódnak.

Megjegyezzük, hogy – a korábbi megoldással ellentétben – most N nem lehet változó (fordítási időben ismert kell legyen az értéke!).

Blokk ismétlés argumentum lista szerint:

```

IRP      par,<arg1[,arg2...]>
...      ; ez a rész kerül többször bemásolásra úgy,
...      ; hogy par rendre felveszi az
...      ; arg1[,arg2...] értéket
ENDM

```

Pl.:


```
IRP    x,<1,2,3>
db     x
ENDM
```

hatása:

```
db     1
db     2
db     3
```

Blokk ismétlés string alapján:

```
IRPC   par,string
...    ; ez a rész kerül többször bemásolásra úgy,
...    ; hogy par rendre felveszi
...    ; a string karaktereit
ENDM
```

Megjegyezzük, hogy ezt a string-et nem kell idézőjelek közé tenni, az idézőjel a blokk újabb ismétlését jelentené. Ez a string az assembler-nek, pontosabban a makró generátornak szól! Az előző feladatot így is megoldhattuk volna:

```
IRPC   x,123
db     x
ENDM
```

Csak azért tudtuk így is megoldani az előző feladatot, mert ott minden argumentumunk egy karakteres volt.

Természetesen makró definíció tartalmazhat blokk ismétlést és blokk ismétlés is tartalmazhat makró definíciót vagy makró hívást. Példaként bemutatjuk a Makró című fejezetben látott bit léptető és forgató utasítás kiterjesztésnek egy újabb megoldását.

; makró definíáló blokkismétlés

```
OPNAME&S  IRP    OPNAME,<RCR,RCL,ROR,ROL,SAR,SAL,SHR,SHL>
MACRO      OPERANDUS,N
mov        c1,N
OPNAME     OPERANDUS,c1
ENDM
ENDM
```

Ennek a megoldásnak előnye, hogy nem kell külön meghívunk a belső makrót az egyes utasításokkal, mert ezt elvégzi helyettünk az `IRP` blokk ismétlés.

6.4.4 Feladatok

1. Határozzuk meg az előjeles word típusú `A` változó n -edik hatványát blokk ismétléssel! Mi a feltétele, hogy megtehesük? (Tegyük fel, hogy az $n-1$ -dik hatvány elfér egy szóban és $n > 0$!)
2. Határozzuk meg az előjeles word típusú `A` változó n -edik hatványát blokk ismétléssel a gyors hatványozás algoritmusát felhasználva!
3. Alakítsuk át a `SHIFTS` blokk ismétlést az előző fejezet utolsó feladatának mintájára.

6.5 Szegmens, szegmens csoport

A szegmens regiszterek nyújtotta lehetőségek és a fordító programokhoz való jobb illeszkedés egyaránt indokolja, hogy az assembly programokat logikai egységekre bontsuk. Az egyik felbontási szempont a gépi utasítások, az adatok és a verem elkülönítését szolgáló szegmens szerkezet, a másik – amivel később részletesebben is fogunk foglalkozni – a moduláris szerkezet (hogy egy programot több file-ban – modul – is megírhatunk).

Szegmens a program egy olyan része, melynek elemeit ugyanazzal a szegmens regiszterrel kívánjuk címezni, anélkül, hogy a szegmens regiszter tartalmát módosítanánk. Lehetőségünk van arra, hogy egy szegmenst több részletben írjunk meg. Erre feltétlenül szükségünk is van, ha a szegmenst több modulban kívánjuk megírni. A szegmens egyes szeleteit a

```
sz_név      SEGMENT      align_type combine_type 'osztály'
```

szegmens kezdő és a

```
sz_név      ENDS
```

szegmens vége utasítások között írhatjuk meg.

sz_név a szegmens (szelet) neve. Ez lehet egyedi név is, de több *SEGMENT* utasítás előtt is szerepelhet ugyanaz a szegmens név. A fordító az azonos nevű szegmens szeleteket úgy tekinti, mintha folyamatosan, egyetlen szegmens szeletbe írtuk volna. Ez lehetővé teszi pl. azt, hogy a kód szegmens írása közben megírjuk az adat szegmensnek azt a részét, amely logikailag a kód szegmens aktuális részéhez kapcsolódik.

Az azonos nevű szegmens szeletek paraméterei egy modulon belül nem változhatnak.

A szerkesztő egy memória szegmensbe szerkeszti az azonos nevű szegmenseket. A szegmens név és osztály tetszésszerűen lehet, de figyelemmel kell lennünk a magasabb szintű programozási nyelvek fordító programjai által használt elnevezésekre, ha az assembly modult más programozási nyelven írt modullal kívánjuk összeszerkeszteni. A szegmens osztály legtöbbször *CODE*, *DATA*, *CONSTANT*, *STACK*, *MEMORY*.

Az *align_type* (illesztés típusa) azt mondja meg, hogy – amennyiben a szegmens szelet egy már megkezdett szegmens folytatása, akkor – milyen „határon” kezdődjön a szegmens szelet. Az *align_type* lehet *BYTE*, *WORD*, *DWORD*, *PARA*, *PAGE*, ami rendre 1-gyel, 2-vel, 4-gyel, 16-tal és 256-tal osztható címet jelent.

A *combine_type* (kombinációs típus) is a szerkesztőnek szóló üzenetet jelent, ami az azonos nevű szegmensek kezelésére vonatkozik. Lehet:

PUBLIC: (alapértelmezés) az azonos nevű szegmens szeletek egymás folytatásként szerkesztendőek össze. Ez a leggyakrabban használt kombinációs típus.

STACK: a stack részeként szerkesztendő a szegmens szelet, egyebekben megegyezik a **PUBLIC**-kal. Amennyiben van **STACK** kombinációs típusú szegmens a programban, akkor *ss* és *sp* úgy inicializálódik, hogy *ss* az utolsó **STACK** kombinációs típusú szegmensre mutat, *sp* értéke pedig ennek a szegmensnek a hossza.

COMMON: az azonos nevű szegmens szeletek azonos címre szerkesztendőek, tehát ezek a szegmens szeletek ugyanarra a memória tartományra kerülnek. Az így keletkező terület hossza megegyezik a leghosszabb szegmens szelet hosszával. Segítségével elérhető, hogy ugyanazt a memória területet eltérő

beosztás szerint tudjuk kényelmesen címezni. Talán a leggyakoribb alkalmazása, hogy a program futásának különböző fázisaiban más-más szerkezetű átmeneti tárolóra van szükség – pl. nagy input terület a futás elején, nagy output terület a futás végén – de nincs szükség egyidejűleg mindkét területre, ilyenkor ugyanis a COMMON illesztési típussal elérhető, hogy ugyanaz a terület mindkét igényt kiszolgálja.

Megjegyezzük, hogy egy modulon belül az azonos nevű szegmens szeletek akkor is egymás után szerkesztődnek, ha COMMON-nak deklaráltuk. A COMMON hatása csak különböző modulokban megírt szegmens szeletekre érvényesül.

MEMORY: a szerkesztő ezt a szegmenst az összes többi szegmens fölé (magasabb címre) fogja szerkeszteni, mindig a program legmagasabb címre kerülő része (megjegyezzük, hogy a Microsoft LINK programja ezt nem támogatja, a MEMORY típust ugyanúgy kezeli, mint a PUBLIC-ot).

AT kif: a kif sorszámú paragrafusra kerül a szegmens szelet. Alkalmas lehet pl. a port-okhoz kapcsolódó memória címek szimbolikus definiálására.

Az assembler-t informálnunk kell arról, hogy a címzésekhez a szegmens regisztereket milyen tartalommal használhatja, más szóval, hogy melyik szegmens regiszter melyik szegmensnek a szegmens címét tartalmazza (melyik szegmensre mutat). Erre a célra szolgál az ASSUME pszeudo utasítás:

```
ASSUME      sz_reg1:sz_név1[, sz_reg2:sz_név2 ...]
```

Az ASSUME utasításban felsorolt szegmenseket „aktívak”-nak nevezzük. Meg kell jegyezzük, hogy az ASSUME utasítás nem gondoskodik a szegmens regiszterek megfelelő tartalommal történő feltöltéséről! Ez a programozó feladata!

Az ASSUME utasítás hatása egy-egy szegmens regiszterre vonatkozóan mindaddig érvényes, amíg egy másik ASSUME utasítással mást nem mondunk az illető regiszterről.

A GROUP utasítással csoportosíthatjuk a szegmenseinket:

```
G_név      GROUP S_név1[, S_név2] ...
```

Az egy csoportba sorolt szegmensek a memória egy szegmensébe kerülnek. Ha ilyenkor az ASSUME utasításban a csoport nevét adjuk meg, és ennek megfelelően állítjuk be a bázis regisztert, akkor a csoport minden szegmensének minden elemére tudunk hivatkozni. Vigyáznunk kell azonban arra, hogy pl. egy változó OFFSET-je továbbra is annak a szegmensnek az elejétől számított távolsága, amelyben a változót definiáltuk, az effektív címe (Effective Address, EA) viszont a változónak a szegmens csoport elejétől (a memória szegmens elejétől) számított távolsága. Pl.:

```
GRP      GROUP      ADAT1,ADAT2
ADAT1    segment    para public 'data'
A         dw         1111h
...
ADAT1    ends
ADAT2    segment    para public 'data'
W         dw         2222h
...
ADAT2    ends
code     segment    para public 'code'
ASSUME   CS:code, DS:GRP, SS:stack, ES:nothing
...
MOV      SI,OFFSET W ; SI ← W offset-je: 0
; az ADAT2 szegmens elejétől mért távolság
MOV      AX,[SI]      ; AX ← 1111h,
; de!!!
LEA      SI, W         ; SI ← effektív címe:
; a GRP szegmens csoport elejétől mért távolság
MOV      AX,[SI]      ; AX ← 2222h.
...
```

6.5.1 Feladatok

1. Egészítsük ki a fenti program részletet az offset és az effective address közötti különbség demonstrálására!

6.6 Globális szimbólumok

Gyakran előfordul, hogy egy programot több modulban írunk meg, és ugyanazt a szimbólumot, pl. változót, még gyakrabban címkét – pl. eljárást – több modulból is el akarjuk érni. A több modulból is elérhető szimbólumokat globális szimbólumoknak nevezzük.

Ha egy szegmens valamely szimbólumát globálissá kívánjuk tenni, akkor **PUBLIC**-ká kell nyilvánítanunk annak a modulnak az elején, amelyben a szimbólumot definiáltuk. Ez a

```
PUBLIC      szimb1[, szimb2]...
```

pszeudo utasítással lehetséges. (A **PUBLIC** utasítás nem tévesztendő össze a **SEGMENT** utasítás **PUBLIC** paraméterével!)

Azokban a modulokban, amelyekben más modulban definiált szimbólumokat is használni szeretnénk, az ilyen szimbólumokat **EXTRN**-né kell nyilvánítanunk. Mivel az assemblernek a szimbólum típusát is ismernie kell, ezért az **EXTRN** pszeudo utasítás a következő alakú:

```
EXTRN      szimb1:típus1[, szimb2:típus2]...
```

A globális szimbólumok lehetővé teszik, hogy a programjainkat modulokra bontva készítsük el. Az egyes blokkok közötti kapcsolatot a globális szimbólumok jelentik. Törekedjünk arra, hogy viszonylag kis modulokat készítsünk, és hogy a moduloknak kevés csatlakozási pontja legyen. Ne használjunk fölöslegesen globális változókat. Azok a változók, melyeket nem adunk meg a **PUBLIC** utasításban, a modul lokális változói, ezeket a többi modul nem ismeri, nem tudja használni, így védve vannak a többi modul elől. Ezáltal a programjaink megbízhatóbbak lesznek.

6.7 **INCLUDE** utasítás

A moduláris program fejlesztés gyakran megköveteli, hogy több modulban is használjuk ugyanazt az adat szerkezetet. Komoly hibalehetőség lenne, ha pl. egy több modulban is használt struktúra definícióját minden modulban megismételnénk. A program írásakor ez még viszonylag egyszerűen megoldható a szövegszerkesztők másoló funkciója segítségével, de a struktúra esetleges módosításakor már sokkal nehezebb dolgunk lenne. Ennek a problémának a kezelésére szolgál az assemblernek szóló **INCLUDE** utasítás:

```
INCLUDE     File_Specifikáció
```

Ennek az utasításnak a hatására az assembler az **INCLUDE** utasítás helyére bemásolja az utasítás paraméterében specifikált szöveg file-t. Így elegendő –

hogy a fenti példánál maradjunk – egy külön file-ban megadni a struktúra definícióját, és a program moduljainkban az INCLUDE utasítást alkalmaznunk erre a file-ra (INCLUDE-oljuk a file-t).

Az INCLUDE-olt file-ok is tartalmazhatnak INCLUDE utasítást, természetesen az így INCLUDE-olt file-okat is bemásolja a fordítóprogram a megfelelő helyre. Könnyű látni, hogy az INCLUDE-ok nem kellően gondos alkalmazása azt eredményezheti, hogy egy file-t többször is INCLUDE-olunk (általában végtelen sokszor), ami a programot hibássá teszi (teheti). Ez ellen a feltételes fordítás segítségével védekezhetünk. Tegyük fel, hogy a makró definícióinkat egy file-ba kívánjuk (MyMacros.i) összegyűjteni, ekkor célszerű ezt a file-t így elkészítenünk:

```
IFDEF MyMacros_i
MyMacros_i = 1
...           ;; makró, struktúra definíciók, EXTRN szimbólumok
ENDIF
```

Ebben az esetben a MyMacros.i file legfeljebb egyszer kerül bemásolásra, mert az összes további esetben a feltételes fordítás feltétele már nem teljesül. Figyeljük meg, hogy a file névben szereplő .-ot _-sal helyettesítettük (az eredeti file név nem szimbólum)! Megjegyezzük, hogy a legtöbb include file-ban ezt a konvenciót alkalmazzák.

6.7.1 Feladatok

1. Készítsünk INCLUDE file-t a makróink számára.

6.8 Lista vezérlési utasítások

TITLE cím

A fordítás során keletkező lista minden oldalán megjelenik ez a cím. Egy modulon belül csak egyszer alkalmazhatjuk.

SUBTITLE alcím

Többször is előfordulhat egy modulon belül. A program lista minden oldalán – a cím alatt – megjelenik az utolsó SUBTITLE utasításban megadott alcím.

PAGE [op1] [,op2]

A paraméterek számától függően több funkciót is betölt. Paraméter nélkül lapdobást jelent. Ha egyetlen paramétere van és az egy + jel, akkor a fejezet sorszámát növeli eggyel, és a lapszámot 1-re állítja. Ettől eltérő esetekben op1 az egy lapra írható sorok ($10 \leq op1 \leq 255$), op2 az egy sorba írható karakterek számát jelenti ($60 \leq op2 \leq 132$). Ha valamelyik paramétert nem adjuk meg, akkor természetesen nem változik a korábban beállított érték. A sorok száma kezdetben 66, a karaktereké 80.

A TITLE, a SUBTITLE és a PAGE egy elkészült programcsoport végső papírdokumentációjának jól olvashatóságát segíti. A programfejlesztés során ritkán készítünk program listákat.

NAME név

A modul nevét definiálhatjuk vele. A szerkesztő ezt a nevet fogja használni. Ha nem szerepel NAME utasítás a modulban, akkor a TITLE utasítással megadott cím a modul neve. Ha ez sincs, akkor a file nevének első 6 karaktere lesz a modul neve.

COMMENT határoló_jel szöveg határoló_jel

Segítségével több soros kommentárokat írhatunk. Az assembler a COMMENT utáni első látható karaktert tekinti határoló_jel-nek, és ennek a jelnek az újabb előfordulásáig minden kommentár. Nyilvánvaló, hogy belsejében nem szerepelhet határoló_jel.

%OUT szöveg

Amikor ehhez az utasításhoz ér a fordítóprogram, akkor a paraméterként megadott szöveg-et kiírja a képernyőre.

.RADIX számrendszer_alapja

Ha programban egy szám nem tartalmaz számrendszer jelölést, akkor az illető számot ebben a számrendszerben kell érteni.

.LIST

Engedélyezi a forrás- és tárgykódú sorok bekerülését a lista file-ba (alapértelmezés).

.XLIST

Tiltja a forrás- és tárgykódú sorok bekerülését a lista file-ba. Jól használható arra, hogy **INCLUDE** előtt tiltsuk a listázást, utána **.LIST** -el újra engedélyezzük, és ezzel az **INCLUDE** file-ok többszöri listázását elkerüljük.

.LFCOND

Minden feltételes blokk kerüljön listára.

.SFCOND

Csak a teljesülő feltételes blokkok kerüljenek listára (alapértelmezés).

.TFCOND

Vált a két előző listázási mód között.

.CREF

Készüljön kereszthivatkozási (cross-reference) tábla (alapértelmezés). Ez a tábla azt a célt szolgálja, hogy könnyen megtaláljuk az egyes változókra történő hivatkozásokat a programban.

.XCREF

Ne készüljön kereszthivatkozási tábla.

.LALL

Kerüljön listára a makró hívások kifejtése.

.SALL

Ne kerüljön listára a makró hívások kifejtése.

.XALL

A makró hívások kifejtéséből csak a kódot generáló rész kerüljön listára (alapértelmezés).

END kifejezés

A modul végét jelzi, kifejezés a program indítási címe. Ha a programunk több modulból áll, akkor természetesen csak egy modul végén adhatunk meg kezdő címet.

6.8.1 Feladatok

1. Lássuk el valamelyik hosszabb programunkat lista vezérlő operátorokkal! Ellenőrizzük az egyes operátorok hatását!

7. ASSEMBLER

Az assembly nyelvre bizonyos közelítéssel igaz, hogy egy sorból egy gépi utasítás lesz. Ez alapján kézenfekvőnek látszik, hogy a fordítóprogram nagy vonalakban az

egy sor olvasása
a beolvasott sor fordítása
a lefordított utasítás az object file-ba
a sor és a lefordított utasítás a lista file-ba

ciklus mag ismétléséből álljon.

Sajnos ez így nem működik, mert számos olyan szimbólumra hivatkozhat a program, amely csak a program szöveg későbbi részében lesz definiálva. Ezt nevezik az előre hivatkozási problémának. A probléma megoldására két lehetőség kínálkozik:

- Az assembler kétszer olvassa a program szövegét (két menet). Az első menet célja összegyűjteni, táblázatba foglalni a szimbólum definíciókat, így a második menet idején már minden (a programban definiált) szimbólum ismert, tehát nem jelentkezik az előre hivatkozás problémája.
- Valahogy megpróbálni a fordítást egy menetben. Késleltetni a fordítást ott, ahol előre hivatkozás van, pl. táblázatba tenni a még le nem fordított részeket. A menet végén már minden szimbólum ismert, ekkor feldolgozni a táblázatot. Esetleg minden szimbólum definíciót követően azonnal feldolgozni a szimbólumra vonatkozó korábbi hivatkozásokat.

Mindkét esetben szükség van szimbólum tábla készítésére, de az utóbbi megoldásban a még le nem fordított utasítások miatt is szükség van táblázatra. További nehézséget jelent, hogy nem sorban készülnek el a tárgy kód (object code) utasításai, ezért ezeket pl. listába kell helyezni, majd rendezni a listát, és csak ezután történhet meg az object és a lista file elkészítése. Ezeknek a nehézségeknek – továbbá annak a ténynek, hogy ma már nem jelent komoly idővesztést egy legfeljebb néhány ezer soros szöveg többszöri végigolvasása – tulajdonítható, hogy manapság a legtöbb assembler két menetben működik.

7.1 Két menetes assembler, első menet

A két menetes assembler első menetének legfontosabb feladata a szimbólum tábla felépítése. Ehhez meg kell keresni a modulban előforduló szimbólum definíciókat, pl.:

```
hossz      equ    100
c          =      c+1
```

de a szimbólum definíció leggyakoribb módja a címke és változó definíció, az ezekre való hivatkozás a definíciónak a programban való előfordulási helyétől függ. A táblázat kitöltéséhez ismernünk kell a definíció előfordulási helyét, a helyszámláló (a már említett \$ assembler változó) aktuális értékét. A helyszámláló értéke általában soronként – utasításonként, pszeudo utasításonként az utasítás hosszával nő (kivéve pl. az ORG utasítást). A szimbólum tábla összeállításához tehát az első menetnek részletesen kell elemeznie a forrás programot.

A szimbólum tábla:

A szimbólum neve	értéke	egyéb információk
...	...	...

érték: – címke címe
– változó címe
– szimbolikus konstans értéke

egyéb információk:
– típus,
– méret,
– szegmens neve, amelyben a szimbólum definiálva van,
– relokációs flag,
– ...

A relokációs flag azt jelzi, hogy a szimbólum értéke függ-e a definíciójának szegmensbeli helyzetétől vagy sem. Erre azért van szükség, mert az assembler modulonként fordítja le a programunkat, és több modulban

történhet egy-egy szegmens definíciója. Az assemblernek tehát nincs tudomása a tényleges OFFSET értékről, csak a modulon belüli helyzetről. A tényleges érték, csak akkor derül ki, amikor az egész szegmens definíció ismertté válik, tehát szerkesztés közben. Ekkor a modulon belüli cím, és a szegmens modulon belüli részének (szegmens szelet) a teljes szegmens kezdetéhez viszonyított kezdő címét összeadva (relokáció) kapjuk meg a szimbólum tényleges (relokált) OFFSET címét.

Sok assembly operandusként megenged konstansokat még akkor is, ha nincs lehetőség közvetlen operandus megadásra, pl. az IBM 370-es gépcsaldon:

```
L      14,=F'5'      ; Load register 14 az 5-ös
                        ; Full Word konstansal
```

Az assembler „magára vállalja”, hogy definiál egy változót, amelynek a tartalma 5. Ezenkívül arra is vigyáz, hogy ne hogy megváltoztassuk az értékét. Az ilyen automatikusan deklarált változókat literáloknak nevezzük. A program a literálok használatával olvashatóbbá válik. Többek között a literálok gyakori használata vezetett a közvetlen operandus megadás kialakulásához és elterjedéséhez.

Az assemblereket általában úgy készítik, hogy az operációs kódokkal kapcsolatos legfontosabb információkat táblázatba, az úgynevezett operációs kód táblázatba helyezik el. Egy lehetséges operációs kód tábla egy részlete:

mnemonic	op1	op2	kód	hossz	osztály
AAA	-	-	37	1	6
ADD	reg8	reg8	02	2	10
ADD	reg16	reg16	03	2	11
...	...	...	...	...	...
AND	reg8	reg8	22	2	10
AND	reg16	reg16	23	2	11
...	...	...	...	...	...

Látható, hogy ugyanaz a mnemonic több sorban is előfordulhat, ez annak felel meg, hogy az operandusoktól függően ezek a mnemonic-ok más és más gépi utasítás kódot jelölnek (a mnemonic nem az utasítás kód megjelölése!

Az összeadást mindig ADD jelöli, függetlenül attól, milyen kódú utasítás fogja végrehajtani).

Az osztály arra utal, hogy milyen operandusok tartozhatnak az adott mnemonic-hoz.

Az operációs kód tábla az assembler készítésekor kerül kitöltésre. Kellően ügyes fordító esetén a processzor kisebb módosításainak, fejlesztéseinek figyelembevételéhez elegendő ezt a táblát módosítani.

Az alábbiakban bemutatjuk egy két menetes assembler Pascal nyelvű vázlatát. Finomítás nélkül nem alkalmas az eddig tárgyalt assembly nyelv fordítására, pl. az eljárás nevének az eljárás végén történő megismétlését is új szimbólumnak tekintené. A célunk nem egy tökéletes program elkészítése volt – a jelen keretek közt amúgy is lehetetlen! – hanem egy jól áttekinthető vázlat ismertetése.

```

procedure ElsőMenet;           {Egyszerű assembler 1. menet, vázlat}

const méret = 8; EndUtasítás = 99;

var
  HelySzámláló, osztály, hossz, kód: integer;
  VanInput: boolean;
  szimbólum, literál, mnemo: array[1..méret] of char;
  sor: array[1..80] of char;

begin
  Előkészítés;
  TáblákInicializálása;
  HelySzámláló := 0;
  VanInput = true;

  while VanInput do begin           {sorok feldolgozása}
    SorOlvasás(sor);
    Megőrzés(sor);
    if NemKomment(sor) then begin           {nem kommentár sor}
      SzimbólumDef(sor, szimbólum);
      if szimbólum[1] <> ' ' then           {szimbólum definíció}
        ÚjSzimbólum(sor, szimbólum, HelySzámláló);
      LiterálKeresés(sor, literál);
      if literál[1] <> ' ' then
        ÚjLiterál(literál);
      Hossz := 0;
      OpKódKeresés(sor, mnemo);
    end
  end

```



```

OpKódTáblában(sor, mnemo, osztály, kód);
if osztály < 0 then {nem létező utasítás}
  PszeudoTáblában(sor, mnemo, osztály, kód);
if osztály < 0 then HibásOpKód;
hossz := típus(osztály); {az utasítás hossza}
HelySzámláló := HelySzámláló + hossz;

if osztály = EndUtasítás then begin
  VanInput := false;
  LiterálTáblaRendezés;
  DuplikátumokKiszűrése;
  Lezárások;
end;
end; {nem kommentár}
end; {while VanInput}
end; {első menet}

```

Az eljárásaink elnevezése magáért beszél, némelyik nagyon egyszerű – pl. az *OpKódKeresés* eljárásnak mindössze az a feladata, hogy a *sor*-ban az első szóköz után a látható karaktereket a következő szóközig terjedően *mnemo*-ba másolja –, mások pedig meglehetősen bonyolultak – pl. az *OpKódTáblában* eljárásnak az operandusok elemzésével kell megállapítania, hogy az utasítás melyik *osztály*-ba tartozik. Látszólag feleslegesen határozza meg a *kód*-ot, de a többi feladata mellett ez már nagyon egyszerű, és így ez a függvény a második menetben változtatás nélkül alkalmazható. Ugyanakkor az *OpKódTáblában* eljárás nem alkalmas pl. az *ORG* pszeudo utasítás feldolgozására, ehhez ismernie kellene ugyanis a *HelySzámláló*-t.

A *SorOlvasás(sor)*; *Megőrzés(sor)*; arra utal, hogy a második menetben olvashatjuk az első menet eredményét. Pl. az első menet folyamán szokás elvégezni az *INCLUDE* utasításokhoz, a makró definíciókhoz és makró hívásokhoz tartozó feladatokat. Megtehetjük, hogy a második menetben újra elvégezzük ezt a feladatot, de ha nincs valami különleges indoka, akkor célszerűbb, ha a második menet az első menet által előállított szöveget olvassa.

Az *Előkészítés* valami ilyesmi lehet:

```

Push(NIL); {sehova mutató pointer a verembe. Ez a verem nem
           az, amit az assembly a PUSH/POP utasításokkal
           kezel}
InputFileNyitás;
p = ProgramSzövegKezdet;
...

```

A továbbiak során p mutatja a következő feldolgozandó karaktert. A *SorOlvasás* eljárás:

```
begin
  while  $p \neq \text{EOF}$  do begin
    Pop( $p$ );
    if  $p = \text{NIL}$  then ENDHiba;           {nincs END utasítás}
  end;
  EgySorOlvasás(sor);
end;
```

Ha *sor* történetesen INCLUDE utasítás, akkor az *EgySorOlvasás* eljárás ezt a következőképpen dolgozhatja fel:

```
Push( $p$ );
IncludeFileNyitás;
 $p = \text{IncludeSzövegKezdete}$ ;
```

7.1.1 Feladatok

1. Írjuk meg assembly-ben a *SzimbólumDef* eljárást!
2. Írjuk meg assembly-ben az *ÚjSzimbólum* eljárást!
3. Módosítsuk az *ElsőMenet* eljárást úgy, hogy az eljárás nevének az eljárás végén történő ismétlését ne tekintse címkének!

7.2 Két menetes assembler, második menet

A második menet legfontosabb feladata, hogy generálja az object file-t és esetleg listát készítsen.

A második menet elején természetesen nem szabad inicializálnunk a szimbólum és a literál táblát! Nem szükséges a

```
SzimbólumDef(sor, szimbólum) ;
if szimbólum[1] <> ' ' then      {szimbólum definíció}
  ÚjSzimbólum(sor, szimbólum, HelySzámláló) ;
```

programrész sem, hiszen ennek volt a feladata a szimbólum tábla felépítése. Megjegyezzük, hogy valami hasonlóra mégis szükség van az = jellel

definiált konstansok miatt, hiszen azok igazából nem is konstansok, hanem a program szöveg különböző területein esetleg eltérő konstans értékkel bíró assembler változók. Az is hasznos, ha a második menet ellenőrzi, hogy most is ugyanott találhatók-e a címkék és változók, ahol az első menetben (elképzeltető, hogy pl. feltételes fordítás miatt ez nem teljesül, ilyenkor az assembler nem tudja befejezni a fordítást és „fázis hibá”-val leáll).

A literálokkal a második menetnek nincs dolga, közönséges változóknak tekintheti, csak a jellemzőiket a literál táblában találja a fordító.

```

procedure MásodikMenet; {Egyszerű assembler 2. menet, vázlat}

const méret = 8; EndUtasítás = 99;

var
  HelySzámláló, osztály, hossz, kód, object: integer;
  VanInput: boolean;
  szimbólum, mnemo: array[1..méret] of char;
  sor: array[1..80] of char;
  operandusok[1..3] of integer; {op1, op2, címezési mód
byte}
  object: [1..10] of byte;

begin
  Előkészítés2;
  HelySzámláló := 0;
  VanInput = true;

  while VanInput do begin                                {sorok feldolgozása}
    SorOlvasás2(sor) ;
    if NemKomment(sor) then begin                            {nem kommentár sor}
      SzimbólumDef(sor, szimbólum);
      if szimbólum[1] <> ' ' then                            {szimbólum definíció}
        SzimbólumEllenőrzés(sor, szimbólum, HelySzámláló);
      hossz := 0;
      OpKódKeresés(sor, mnemo);
      OpKódTáblában(sor, mnemo, osztály, kód);
      if osztály < 0 then                                       {nem létező utasítás}
        PszeudoTáblában(sor, mnemo, osztály, kód);
      if osztály < 0 then HibásOpKód2; {Most készül a lista!}
      case osztály of
        0: hossz := fordító(sor, operandusok);
        1: hossz := fordító1(sor, operandusok);
        ...
      end;
      Összeállítás(kód, osztály, operandusok, object);
  end;

```

```

ObjectKiírás(object);
Listázás(HelySzámLáló, object, sor);
HelySzámLáló := HelySzámLáló + hossz;

if osztály = EndUtasítás then begin
    VanInput := false;
    Lezárások2;
end;
end; {nem kommentár}
end; {while VanInput}
end; {második menet}

```

Az *Összeállítás* eljárás angol elnevezése *assemble*. Az *assembly*, *assembler* innen kapta az elnevezését. Az *object file* nemcsak a lefordított utasításokat tartalmazza, hanem további – a szerkesztőnek szóló – információkat is.

Az assembler számos hibát ismerhet fel:

- használt szimbólum nincs definiálva,
- egy szimbólum többször van definiálva,
- nem létező operációs kód,
- nincs elegendő operandus,
- túl sok operandus van,
- hibás kifejezés (pl. 9 egy oktális számban),
- Illegális regiszter használat,
- típus hiba,
- nincs END utasítás,
- ...

Számos olyan hibát azonban, melyet a magasabb szintű nyelvek fordítói könnyen felismernek – vagy egyáltalán elő se fordulhatnak – az assembler nem tud felderíteni:

- az eljárás hívás paramétereinek típusa nem megfelelő.
- a regiszter mentések és helyreállítások nem állnak „párban”,
- hibás vagy hiányzik a paraméterek vagy a lokális változó terület ürítése a veremből,
- A hívás és a hívott eljárás helyén érvényes ASSUME-ok ellentmondásosak (nem feltétlenül hiba, de az lehet),
- ...

7.2.1 Feladatok

1. Módosítsuk az *ElsőMenet* eljárást úgy, hogy fel tudja dolgozni az
IFDEF ... ELSE ... ENDIF feltételes fordítási szerkezetet!

8. MAKRÓ GENERÁTOR

A makró generátor feladata a makró definíciók megjegyzése (pl. makró táblába helyezése) és a makró hívások kifejtése. Általában az assembler első menetében működik.

Makró definíciók felismerése

Amennyiben az assembler a forrás szöveg olvasása közben makró definíciót talál (ezt könnyű felismerni a műveleti kód részen lévő `MACRO` szó alapján), akkor a makró definíció teljes szövegét – az `ENDM` műveleti kódot tartalmazó sor végéig – elhelyezi a makró táblában. A felismerést és a tárolást a *SorOlvasás* vagy a *PszeudoTáblában* eljárás végezheti.

Makró hívások kifejtése

Az

```
OpKódTáblában(sor, mnemo, osztály, kód);  
if osztály < 0 then {nem létező utasítás}  
    PszeudoTáblában(sor, mnemo, osztály, kód);
```

programrész után be kell szűrni az

```
if osztály < 0 then {nem létező pszeudo utasítás}  
    MakróTáblában(sor, mnemo, osztály, kód);
```

sorokat. A eljárás feladata a makró hívás felismerése és a makró helyettesítés is. A kifejtett makró egy pufferbe kerül, a puffer tartalma az `INCLUDE` utasításnál látottakhoz hasonlóan illeszthető a program szövegébe. A makró kifejtés a következő programrészt hajtja végre ciklikusan:

```
EgySzóOlvasásaAMakróTörzsből;  
if FormálisParaméter then  
    AMegfelelőSorszámúAktuálisParaméterÁtmásolása;  
else  
    ASzóÁtmásolása;  
ElválasztójelFeldolgozása;
```

Az *ElválasztójelFeldolgozása* legtöbbször az elválasztójel másolását jelenti, de a makró definícióban különleges szerepet játszó karakterek esetén ettől eltérő – magától értetődő – speciális feladatot kell végrehajtani.

A *LOCAL* utasítás feldolgozásához a makró generátor egy 0 kezdeti értékű változót használ. Makró híváskor a *LOCAL* utasításban szereplő szimbólumot, és az összes előfordulását a makró törzsben *??xxxx* alakú azonosítóval helyettesíti, ahol *xxxx* a változó aktuális értéke hexadecimális számrendszerben. A változó értékét minden a *LOCAL* utasításban szereplő szimbólum feldolgozása után 1-gyel növeli. Ez legegyszerűbben úgy oldható meg, ha a makró generátor a *LOCAL* utasításban szereplő szimbólumokkal kiegészíti a formális paramétereket és a fenti módon generál hozzá aktuális paramétereket.

8.1.1 Feladatok

1. Írjuk meg assembly-ben a makró fej feldolgozását, mutató vektor elemei mutassák az egyes formális paramétereket!
2. Írjuk meg assembly-ben az *ElválasztójelFeldolgozása* eljárást az *&* és a *!* jel esetében!

9. SZERKESZTŐ

Különböző operációs rendszerekben a szerkesztő elnevezései Link, Linker, Linkage Editor, Linkage Loader, ... Az elnevezéstől függően kissé eltérő feladatokat lát el. Alapvető feladata, hogy a programhoz tartozó object file-okat (modulokat) egyetlen, „futtatható” file-ba, a load modulba „szerkessze”. Eközben a következő feladatokat kell megoldania:

- az azonos nevű és osztályú szegmens szeletek egymáshoz illesztése a szegmens szeletek definíciójában megadott módon,
- a GROUP pszeudo utasítással egy csoportba sorolt szegmensek egymás után helyezése,
- a relokáció elvégzése,
- a külső hivatkozások (EXTRN) feloldása.

A relokációs problémáról már tettünk említést a szimbólum tábla ismertetésekor. A külső hivatkozások az EXTRN utasításban felsorolt változókra történő hivatkozások, ezeket a fordító természetesen nem tudja lefordítani, mert – lévén más modulban definiálva a szimbólum – az elhelyezési címét nem ismeri.

A szerkesztő is két menetben dolgozik, az első menetben táblázatokat készít, a második menetben a táblázatokba elhelyezett információk alapján elvégzi a szerkesztést. Az egyik táblázat a szegmens szeletekre vonatkozó úgynevezett térkép (map), a másik a globális szimbólumok táblázata.

A térkép szegmens szeletenként egy sort tartalmaz:

modul név	szegmens név	osztály	illesztés típusa	kombinációs típus	hossz	kezdő- cím	relokációs konstans
...	...	...	...	...	...	...	...

A szerkesztő az első menetben beolvassa az összeszerkesztendő modulokat, és közben kitölti a fenti táblát a kezdőcím és a relokációs konstans kivételével. Az összes modul beolvasása után a kívánt szempontok (szegmens név, osztály, kombinációs típus, és a szerkesztő opcióiban előírt szempontok) szerint rendezi a táblázat sorait, ezután a kezdőcím és a

relokációs konstans rovatot is ki tudja tölteni felülről lefelé haladva az illesztés típusa, a kombinációs típus, az előző hossz és kezdőcím felhasználásával. A legelső kezdőcím az operációs rendszertől függ (betöltési cím). Az egyes szegmens szeleteknek és a szegmens első szeletének a kezdőcíme közötti különbség lesz az a relokációs konstans, amit az illető szegmens szeletben definiált szimbólumokra történő hivatkozások esetén hozzá kell adni a fordító által előállított relokálandó címhez.

A globális szimbólumok kezeléséhez a fordítóprogram minden object modulban információt ad át a modulban PUBLIC-ká és EXTRN-é nyilvánított változokról. A szerkesztő az első menetben a PUBLIC változókból egy közös globális szimbólum táblát készít:

modul név	szegmens név	szimbólum	típus	cím
...	...	...	...	...

A PUBLIC utasítás nem tartalmazza a típust és a szegmens nevét, de az assembler ismeri, és el tudja helyezni az object modulban. A modul nevét a szerkesztő is ismeri. A cím a táblázat összeállításakor még relokálatlan címet jelent. Az első menet végén ebben a táblázatban elvégezhető a relokáció.

Az assembler az EXTRN utasítás alapján a következő információt adja át:

szimbólum	típus	hivatkozás1		hivatkozás2		...
		szegmens név	offset cím	szegmens név	offset cím	
...	...	...	...	...	...	...

A szerkesztő az első menetben kigyűjti a szimbólumokat a hozzájuk tartozó típussal együtt, és ez alapján az első menet végén ellenőrzi, hogy minden szimbólum előfordul-e a közös globális szimbólum táblában. Ha nem, akkor definiálatlan külső hivatkozások maradtak, ilyenkor az első menet további programkönyvtárbeli modulok hozzávételével folytatódhat.

A szerkesztő a második menetben a (már rendezett) térkép sorainak megfelelő sorrendben beolvassa és elhelyezi a szegmens szeleteket, elvégzi a szükséges relokációt és feloldja a külső hivatkozásokat. Ez utóbbi

feladathoz egy-egy modul betöltésének végén a szerkesztő az EXTRN szimbólumokat sorra megkeresi a közös PUBLIC táblában. Ha megtalálta, akkor sorra veszi a hivatkozásokat, és mindenegyest hivatkozáshoz elhelyezi a PUBLIC táblában található címet. Ha az illető szimbólum nem található meg a közös PUBLIC táblában, akkor „feloldatlan címkonstans” hibajelzést ad.

A szerkesztő – az object file-ra, az eljárások paramétereinek átadására és átvételére vonatkozó előírások betartása esetén – lehetővé teszi, hogy egy nagyobb programot több modulban, esetleg különböző programozási nyelveken írjunk meg. Segítségével lehetővé válik, hogy részfeladatok megoldásához már korábban elkészített object modulokat használjuk fel. A szerkesztés paraméterezésének megkönnyítésére object modulokból könyvtárakat szokás létrehozni. A szerkesztéshez ezután nem szükséges az összes standard modult felsorolnunk, hanem elegendő a megfelelő könyvtárakat megadnunk. A szerkesztő feladata, hogy ezekből kiválogassa a szükséges modulokat.

9.1.1 Feladatok

1. Egészítsük ki a két menetes assemblert, hogy helyezze el az object file-ban a PUBLIC és az EXTRN utasítással kapcsolatos információkat!

10. CÍM HOZZÁRENDELÉS (BINDING)

A cím hozzárendelés problémájához vizsgáljuk meg egy időosztásos (time sharing) rendszer működését. Egy ilyen rendszerben (nagyobb léptékben nézve) egyidejűleg több program fut. A program kiválasztó (scheduler) feladata, hogy gondoskodjon arról, hogy mindegyik program elég sűrűn kapjon egy rövid CPU időt. Az idő szerinti megszakítás gondoskodik arról, hogy a program kiválasztó gyakran megkapja a vezérlést. Egy program elindításakor többek között a következő feladatokat kell végrehajtani:

betöltés	indítás	felfüggesztés	kimentés
----------	---------	---------------	----------

a program folytatásakor:

visszamentés	futtatás	felfüggesztés	kimentés
--------------	----------	---------------	----------

A kimentés – visszamentésre (amelyet a program kiválasztó hajt végre) azért van szükség, mert általában nem biztosítható, hogy a program a teljes működési ideje alatt a memóriában tartózkodjon. A jelen témánk szempontjából az a lényeges, hogy általában még az sem biztosítható, hogy a visszamentett program a memóriának ugyanarra a területére kerüljön vissza, ahol korábban futott. A relokációs információk már nem állnak rendelkezésre, és különben is túl drága lenne a használatuk. Ha pl. a programunk egy `JMP L` ugró utasítást tartalmaz, akkor `L`-hez valamikor hozzá kell rendelnünk a konkrét címet – `bindig` = (cím) hozzárendelés.

A cím hozzárendelés különböző időpontokban történhet.

- Program írásakor (gépi kódú programozás): a számítástechnika kialakulásakor ez volt a helyzet, a programot bináris (oktális, hexadecimális) számokkal kellett megadni. Nem is olyan régen még gyakran előfordult, hogy program teszteléskor a memóriába betöltött gépi kódú programot a vezérlő pultról javították. Manapság ez a módszer már ritka, de a lehetősége fellelhető a szegmens definícióban alkalmazható `AT` kifejezés kombinációs típusban.

- Fordításkor: Ma már ezt a módszert is igen ritkán alkalmazzák, pl. szintén az AT kifejezés kombinációs típussal kapcsolatban, ha a szegmensben szimbolikus címezést használnak. Az általános használata arra az időre vezethető vissza, amikor még nem különült el a fordítás és szerkesztés folyamata. A cím hozzárendelés ilyen módjának a legnagyobb hátránya, hogy alkalmazása nehezzé, időnként lehetetlenné teszi az így elkészített program részek különböző programokban való felhasználását.
 - Szerkesztéskor: Ma ez a cím hozzárendelés egyik legelterjedtebb módja. Általában ez valósul meg az assembly programokban is.
 - Betöltéskor: Azt gondolhatnánk, hogy egy assembly programban ezt a cím hozzárendelést használjuk. Valóban használhatnánk ezt is, de akkor a programban nem szerepelhetne pl. egyetlen FAR típusú cím konstans sem, hiszen ennek az értéke a betöltés helyétől függ. A szerkesztő a – feltételezett – betöltési cím alapján ezeket az értékeket kiszámolja és beépíti a betölthető kódba (.EXE file). Az is megvalósítható, hogy az ilyen jellegű feladatok elvégzését a szerkesztő rábízza a betöltőre. Ennek természetesen az az ára, hogy a program betöltése hosszabb időt vesz igénybe, előnye viszont, hogy a betöltési cím tetszőleges lehet.
 - A címezéshez használt bázis regiszter kitöltésekor: Ez a módszer már biztosítja az időosztásos rendszerben futtatható alakú programok elkészítését. Alkalmazása még komolyabb megszorításokat jelent a program megírásával szemben, mert nemcsak a kezdeti, hanem a későbbi – esetleg más címre történő – betöltések esetén is gondoskodni kell a FAR címeket tartalmazó változók helyes értékéről, sőt, távoli eljárás hívás esetén FAR cím kerül a verembe, és természetesen ezzel is foglalkozni kell.
- Megfelelően kialakított hardver sokat segíthet a probléma megoldásában. Van olyan processzor (pl. a Motorola 680x0 processzor család), amely rendelkezik egy „program bázis” regiszterrel. Egy hivatkozás vagy abszolút vagy relatív (a program bázis regiszterrel módosítandó). Minden utasítás tartalmaz egy bitet, hogy a benne szereplő hivatkozás abszolút vagy relatív. Ezeken a gépeken nem okoz problémát a program áthelyezése.
- Az utasítás végrehajtásakor: Ehhez a cím hozzárendelést két lépésben valósítják meg. Először a szimbolikus címezést virtuálissá alakítják, majd ezt fizikai címezéssé. A virtuális cím hozzárendelés történhet pl. szerkesztési időben, a fizikai cím hozzárendelés a lap (page) tábla kitöltésekor, ami valóban gyakran az utasítás végrehajtásának

megkezdése után történik. Amikor ugyanis a hivatkozott lap még nincs a memóriában, akkor azt az utasítás végrehajtása előtt be kell tölteni, a lap táblának a megfelelő elemét módosítani kell (cím hozzárendelés), majd ezután történhet a megkezdett utasítás végrehajtásának befejezése.

10.1 Dinamikus szerkesztés

Előfordulnak olyan nagyméretű programok, amelyekben bizonyos eljárások csak nagyon ritkán szükségesek. Ha ezeket az eljárásokat is hozzászerkesztjük a programhoz, akkor a program memória igénye nagyon nagy lehet. Ennek a problémának a kezelésére szolgál a dinamikus szerkesztés.

A dinamikus szerkesztés egy lehetséges megoldása, hogy a programban létrehozunk egy táblázatot (Linkage Segment), amely minden esetleg szükséges eljáráshoz egy csatoló blokkot tartalmaz. Minden blokk elején egy „virtuális cím”, a további részén az eljárás neve található karakter sorozat formában. A virtuális cím lehet egy csatoló program címe vagy egy lehetetlen cím (Invalid Address). Az utóbbi esetben gondoskodnunk kell arról, hogy a létrejövő megszakítás hatására a csatoló program kapja meg a vezérlést. A csatoló blokk célszerűen egy

```
CSAT_STR    STRUCT
CIM         DD     FAR PTR CSATOLO    ; csatoló program FAR címe
NEV         DB     '                  ; 8 szóköz karakter
CSAT_STR    ENDS
```

típusú struktúra változó lehet. Ez alapján pl. az ALFA eljáráshoz az:

```
ALFA        CSAT_STR    <, 'ALFA'>
```

csatoló blokk tartozhat. A dinamikusan szerkesztendő ALFA eljárás hívása pl. a következő módon történhet:

```
MOV    BX, OFFSET ALFA
CALL   [BX].CIM
```

A CALL utasítás hatására a CSATOLO kapja meg a vezérlést. A BX regiszterben van az ALFA eljáráshoz tartozó csatoló blokk címe, így a csatolandó eljárás neve a [BX].NEV címen található. A csatoló program feladata, hogy az eljárás név alapján a megfelelő eljárást betöltse, és a programhoz szerkessze. Ehhez természetesen fel kell használnia és módosítania a program szerkesztésekor készült táblákat, a térképet (map) és a globális szimbólumok alapján készített táblázatot. Ha a dinamikusan szerkesztendő programmal szemben megszorításokkal élünk (pl. nem tartalmazhat EXTRN utasítást, egyetlen, a memóriába bárhová betölthető modulból áll), akkor a szerkesztés jelentősen egyszerűsödhet – akár magára a betöltésre, és ennek a tényét rögzítő adminisztrációra (ilyenkor a szerkesztő táblázatokra sincs szükség, csak felhasználható memória területre, ahová a betöltés megtörténhet).

A szerkesztés végeztével a ([BX].CIM címen található) virtuális címet a most a programhoz szerkesztett eljárás kezdőcímére változtatja, és erre a címre adja a vezérlést. Ha ezután újabb hívás történik az imént csatolt eljárásra, akkor az a CSATOLO közbeiktatása nélkül azonnal végrehajtásra kerül.

A módszer egyszerűen továbbfejleszthető a rendszer szolgáltatásainak bővítéséhez is. Az előző módszert úgy kell módosítanunk, hogy a csatoló paraméterként kapja meg a felhívandó eljárás nevét. Ekkor maga a csatoló program hozhat létre egy csatoló táblázatot. A csatoló a táblázatban ellenőrzi, hogy csatolva van-e a kívánt eljárás. Ha nincs, akkor elvégzi a csatolást, ha pedig csatolva van, akkor közvetlenül meghívja az eljárást. Ha ezt a módszert például egy menü vezérelt rendszer esetében használjuk, és úgy készítjük el a menü vezérlőt, hogy a kiválasztott menü elemhez tartozó szövegből generálni lehet az – esetleg csatolandó, majd – végrehajtandó eljárás nevét és az eljárást tartalmazó file nevét, akkor a program bővítését, javítását a megfelelő file-ok hozzáadásával vagy cseréjével és a menü szöveg file-jának cseréjével akár üzemelés közben is elvégezhetjük.

Annak biztosítására, hogy az egyesével csatolt eljárások együtt ne foglaljanak el túlságosan nagy memóriát, lehetőség van arra, hogy a virtuális cím visszairása – illetve a csatoló tábla törlése – után a dinamikusan szerkesztett eljárások által lefoglalt memória területet felszabadítsuk. A felszabadított területre más eljárás csoportok tölthetők be ugyancsak a csatolás módszerével. Ennek kidolgozott technikája az átfedés – overlay – módszere.

10.1.1 Feladatok

1. Készítsünk teszt programot, amellyel szimulálni tudjuk a csatoló működését (a csatoló csatolás helyett írja ki a csatolandó eljárás nevét)!
2. Gondoljuk át, hogyan segíthet az `INT` utasítás a csatolás megvalósításában!
3. Alkalmas lehet-e a csatolás módszere több processzusból hívható, de egy példányban betöltendő eljárások kezelésére? Az ilyen eljárásokra milyen korlátozásokat írna elő?

11. PROGRAMOK HANGOLÁSA

Tekintettel arra, hogy assembly-ben programozva a processzor minden lehetőségét ki tudjuk használni, azt gondolhatjuk, hogy a nagyobb sebesség elérése érdekében a nagy és gyakran használt programokat assembly-ben célszerű megírni. Hogy ez mennyire nem így van, azt a MULTICS és a TSS/67 operációs rendszer összehasonlításával szemléltethetjük. A két rendszer programozását nagyjából egyszerre kezdték, és egyszerre fejezték be. A MULTICS majdnem teljesen (95%-ban) PL/I nyelven, a TSS/67 teljes egészében assembly-ben készült. Mindkét program hatalmas (a MULTICS listája 3.000, a TSS/67-é 30.000 oldal), nagyjából azonos célra használható. Lényeges különbség viszont, hogy amíg a MULTICS kidolgozásában 50 ember dolgozott, a költsége kb. 10 millió dollár volt, addig a TSS/67 esetén ezek az adatok: 300 ember, 50 millió dollár.

A lényeges különbséget megmagyarázza az a megfigyelés, hogy egy programozó néhány évig egy nagyobb feladaton dolgozva havi átlagban csak kb. 100-200 (!!) ellenőrzött utasítást ír, függetlenül az alkalmazott programozási nyelvtől, és egy PL/I utasítás 5-10 assembly utasításnak felel meg.

Továbbra is mondhatjuk, hogy – azonos teljesítményű processzorok esetén a TSS/67 nyilván sokkal gyorsabb. Hogy ez nem igaz, az a MULTICS hangolásának köszönhető (valószínűleg kizárólag ennek, hiszen akkor a fordítók a célprogramok optimalizálására még kevés figyelmet fordítottak).

Irodalmi adatok alapján azt lehet mondani, hogy (hangolás előtti) nagyobb programok 1%-a felelős a program futási idejének kb. 50%-áért, 10%-a a 90%-áért. Program hangoláson azt a folyamatot értjük, amikor megállapítjuk a kritikus részeket, és ezek gyorsításával az egész program futását felgyorsítjuk.

A kritikus részek felderítésére használható a következő (mintavételező) módszer. Miközben a hangolandó programot nagy méretű feladaton vagy feladatokon futtatjuk, rövid, véletlen időnként időzítés szerinti megszakításokat kérünk. A megszakítások címét egy file-ban rögzítjük. Később a megszakítások címét és a szerkesztő által összeállított memória

térképet összevetve, megállapíthatjuk, hogy mely eljárásokban történt a legtöbb megszakítás, ezekben az eljárásokban tartózkodott legtöbbet a program. Ezeket az eljárásokat, eljárás részeket kell felgyorsítani pl. assembly betétekkel, vagy az egész eljárás assembly-be való átírásával.

A program hangolás hatásának szemléltetésére tételezzük fel, hogy ugyanannak a feladatnak – pl. egy fordítóprogramnak – a megoldásához assemblyben 5-ször annyi utasításra (és időre) van szükség, mint probléma orientált nyelv esetén, és az elkészült program 3-szor olyan gyors. A probléma orientált nyelven készült változatának kritikus 10%-át assemblyben újra programozzuk. A költségek és futási idők alakulását az alábbi táblázat mutatja:

	programozó év	futási idő (ms)
Assembly nyelv	50	333
Probléma orientált nyelv	10	1000
Hangolás előtt		
a kritikus 10%	1	900
a többi 90%	9	100
Összesen	10	1000
Hangolás után		
a kritikus 10%	1+5	300
a többi 90%	9	100
Összesen	15	400

A táblázatból leolvasható, hogy a fordító probléma orientált nyelven történő elkészítésének és hangolásának ideje (és költsége) kb. harmada annak, mintha az egészet assemblyben készítenénk, ugyanakkor a sebessége csak 20%-kal gyengébb.

Összefoglalva: A legritkább esetben célszerű egy feladatot teljes egészében assemblyben programozni, de ha a program gyakran fut, akkor célszerű lehet a hangolás során bizonyos részeket assembly-ben átírni.

FÜGGELÉK

A MASM fordító használata

A

MASM /HELP

parancs hatására részletes információt kapunk a MASM parancs paramétereiről és a használható opciókról. A parancs általános alakja:

MASM /options S(.asm),O(.obj),L(.lst),C(.crf);

ahol a zárójelben azok a file kiterjesztések szerepelnek, amit akkor használ a fordítóprogram, ha nem adunk meg file kiterjesztést. S a source, O az object, L a lista, C a kereszt hivatkozási file neve kiterjesztéssel vagy anélkül. Ha O-t, L-et, vagy C-t nem adjuk meg, de a , -t vagy a ; -t kitesszük, akkor ezek a file nevek S alapján generálódnak. Ha sem , -t sem ; -t nem írunk, esetleg S-et sem adjuk meg, akkor a fordító rákérdez a meg nem adott paraméterekre.

A LINK szerkesztő használata

A

LINK /?

parancs hatására kapunk információt a LINK parancs paramétereiről és a használható opciókról. A parancs lehetséges alakjai:

LINK /options O(.obj),E(.exe),M(.map),L(.lib);

LINK @parameters

ahol a zárójelben azok a file kiterjesztések szerepelnek, amit akkor használ a szerkesztő, ha nem adunk meg file kiterjesztést. O az object, E a futtatható program, M a lista (memória térkép, map) és L a programhoz szerkesztendő könyvtár file neve kiterjesztéssel vagy anélkül. + jellel összekapcsolva több összeszerkesztendő object file nevét adhatjuk meg, ezek közül az első nevéből generálódik a futtatható program és a térkép neve, ha E-t illetve M-et nem adjuk meg, de a , -t vagy ; -t kiírjuk. Ugyancsak + jellel összekapcsolva

több könyvárat is felsorolhatunk. Ha L-et hagyjuk üresen, akkor nem kérjük könyvtár hozzászerkesztését a programhoz. Ha sem , -t sem ; -t nem írunk, esetleg o-t sem adjuk meg, akkor a szerkesztő rákérdez a hiányzó paraméterekre.

A szerkesztő hívásának másik alakjában a parameters file tartalmazza a szükséges paramétereket. Ez azon kívül, hogy – parameters elkészítése után – kényelmes, lehetővé teszi, hogy olyan hosszú paraméterezést is használhassunk, ami nem fér el egy sorban.

Megjegyzés: A LINK bizonyos verziói számára további információk is megadhatók (<deffile>, Definitions File). Ezt a paramétert általában üresen hagyhatjuk (ENTER-rel válaszolhatjuk meg). Ha mégis szükség lenne rá – pl. DOS alatt WINDOWS alá szerkesztünk programot – akkor az aktuális verzió leírásában találhatjuk meg a szükséges információkat.

A DEBUG program használata

A

DEBUG /?

parancs hatására információt kapunk a DEBUG program indítási lehetőségeiről, ha már elindítottuk, és a DEBUG kész újabb parancsot fogadni (ezt a – prompt jelzi), akkor ? gépelésével kaphatunk információt a lehetséges parancsokról. Itt csak vázlatosan ismertetjük a legszükségesebb parancsokat, az érdeklődő olvasó részletesebb leírást talál Pethő Ádám „IBM PC/XT assembly alapismeretek” (SZÁMALK) című könyvében.

Általában célszerű – különösen assembly-ben kezdő programozók esetén – ha a program DEBUG-olásához („bogarászás”, nyomkövetés) papíron rendelkezésünkre áll a lefordított programról készült lista. A DEBUG képes ugyan a soron következő utasításokat kiírni (u – unassemble – parancs), de ebben a kiírásban nem szerepelnek a program szimbólumai, csak a mnemonic és a regiszterek megnevezése emlékeztet bennünket a forrás programban írottakra. A program betöltése a

DEBUG FileSpecifikáció

paranccsal történhet, ahol FileSpecifikáció a DOS-ban szokásos lehet.

A betöltött programok nyomkövetése a T és a G parancsokkal lehetséges. T hatására egyetlen, T érték hatására érték számú utasítást hajt végre a DEBUG program, utána leáll. G hatására a program nyomkövetés nélküli futása kezdődik. A G parancs után – szóközzel elválasztva – több címet (break point, töréspont) is megadhatunk. Ha a vezérlés valamely töréspontra adódik, akkor a program végrehajtása (még a töréspontnál lévő utasítás végrehajtása előtt) megszakad. Ha tehát töréspontként az aktuálisan végrehajtandó utasítás címét adjuk meg, akkor a program egyetlen utasítása sem kerül végrehajtásra. Ebből következik az is, hogy egy ciklusmag többszöri futása nem ellenőrizhető azon a kézenfekvő módon, hogy a ciklusmagnak mindig ugyanazt az utasítást adjuk meg töréspontként.

Megjegyezzük, hogy minden G parancs esetén csak az aktuálisan megadott töréspontok érvényesek.

A DEBUG minden leállításakor vagy az R parancs hatására hexadecimálisan kiírja az összes regiszter tartalmát, STATUS bitjeit szimbolikusan az alábbi táblázat szerint:

flag	1	0
Overflow	OV	NV
Direction	DW	UP
Interrupt	EI	DI
Sign	PL	NG
Zero	ZR	NZ
Auxiliary Carry	AU	NA
Parity	PO	PE
Carry	CY	NC

a soron következő utasítás címét és magát az utasítást.

Ha csak néhány utasítás pontos hatására vagyunk kíváncsiak, akkor ehhez nem kell programot írunk, fordítanunk és szerkesztenünk, hanem az A parancs segítségével közvetlenül a memóriába írhatjuk az utasításokat. Az így beírt utasításokban nem szerepelhetnek szimbólumok, ezért pl. egy változó tartalmára úgy hivatkozhatunk, hogy [] -be tesszük a változó címét.

Eltérés az assembly programozástól, hogy az explicit szegmens megadást az utasítás elé kell írni, pl.:

```
ES:MOV AX, 34F[SI][BX]
```

A programból a Q paranccsal léphetünk ki.

Utasítás rendszer (összefoglaló táblázat)

Formátum	Megjegyzés	O	D	I	T	S	Z	A	P	C
MOV op1, op2	$op1 \leftarrow op2$									
XCHG op1, op2	$op1 \leftrightarrow op2$									
XLAT	$AL \leftarrow [BX+AL]$									
LDS reg, mem	$(DS:reg) \leftarrow mem$									
LES reg, mem	$(ES:reg) \leftarrow mem$									
LEA reg, mem	Load Eff. Address									
PUSH op	$SP \leftarrow SP-2; (SS:SP) \leftarrow op$									
PUSHF	push Flags									
POP op	$op \leftarrow (SS:SP); SP \leftarrow SP+2$									
POPF	pop Flags	*	*	*	*	*	*	*	*	*
LAHF	$AH \leftarrow \text{low STATUS}$									
SAHF	$\text{low STATUS} \leftarrow AH$						*	*	*	*
ADD op1, op2	$op1 \leftarrow op1 + op2$	*				*	*	*	*	*
ADC op1, op2	$op1 \leftarrow op1 + op2 + C$	*				*	*	*	*	*
INC op	$op \leftarrow op + 1$	*				*	*	*	*	*
AAA	Ascii Adj. after Add.	?				?	?	*	?	*
DAA	Dec. Adj. after Add.	?				*	*	*	*	*
SUB op1, op2	$op1 \leftarrow op1 - op2$	*				*	*	*	*	*
CMP op1, op2	$op1 - op2$, CoMPare	*				*	*	*	*	*
SBB op1, op2	$op1 \leftarrow op1 - op2 - C$	*				*	*	*	*	*
DEC op	$op \leftarrow op - 1$	*				*	*	*	*	*
AAS	Ascii Adj. after Sub.	?				?	?	*	?	*
DAS	Dec. Adj. after Sub.	?				*	*	*	*	*
NEG op	$op \leftarrow -op$	*				*	*	*	*	*
MUL op	előjel nélküli szorzás	*				?	?	?	?	*
IMUL op	előjeles szorzás	*				?	?	?	?	*
AAM	Ascii Adj. after Mul.	?				*	*	?	*	?
CBW	Convert Byte to Word									
CWD	Convert Word to Double									
DIV op	előjel nélküli osztás	?				?	?	?	?	?
IDIV op	előjeles osztás	?				?	?	?	?	?
AAD	Ascii adj. before div.	?				?	?	?	?	?
CALL op	eljárás hívás									
RET	return									
RET op	return és $SP \leftarrow SP+op$									
JMP op	feltétlen ugrás									
LOOP ipr	$CX \leftarrow CX-1$, $\rightarrow ipr$, ha $CX \neq 0$									
LOOPZ ipr	$\rightarrow ipr$, ha $(CX \neq 0 \ \& \ Z=1)$									
LOOPE ipr	$\rightarrow ipr$, ha $(CX \neq 0 \ \& \ Z=1)$									
LOOPNZ ipr	$\rightarrow ipr$, ha $(CX \neq 0 \ \& \ Z=0)$									
LOOPNE ipr	$\rightarrow ipr$, ha $(CX \neq 0 \ \& \ Z=0)$									
Formátum	Megjegyzés	O	D	I	T	S	Z	A	P	C

Formátum	Megjegyzés	O	D	I	T	S	Z	A	P	C
MOVSB	MOVE String Byte									
MOVSW	MOVE String Word									
MOVS d, s	MOVE String byte/word									
LODSB	LOaD String Byte									
LODSW	LOaD String Word									
LODS s	LOaD String byte/word									
STOSB	STORe String Byte									
STOSW	STORe String Word									
STOS d	STORe String byte/word									
CMP SB	CoMPare String Byte	*				*	*	*	*	*
CMP SW	CoMPare String Word	*				*	*	*	*	*
CMP S d, s	CoMPare String byte/word	*				*	*	*	*	*
SCASB	SCAn String Byte	*				*	*	*	*	*
SCASW	SCAn String Word	*				*	*	*	*	*
SCAS d	SCAn String byte/word	*				*	*	*	*	*
REP	ismétlő prefixum									
REPZ	ismétlő prefixum						*			
REPE	ismétlő prefixum						*			
REP NZ	ismétlő prefixum						*			
REP NE	ismétlő prefixum						*			
AND op1, op2	op1 ← op1 & op2 bitenként	0				*	*	?	*	0
TEST op1, op2	op1 & op2 bitenként	0				*	*	?	*	0
OR op1, op2	op1 ← op1 op2 bitenként	0				*	*	?	*	0
XOR op1, op2	op1 ← op1 ^ op2 bitenként	0				*	*	?	*	0
NOT op	op ← ~op bitenként									
RCR op, 1/CL	Rotate through C Right	*								*
RCL op, 1/CL	Rotate through C Left	*								*
ROR op, 1/CL	Rotate Right	*								*
ROL op, 1/CL	Rotate Left	*								*
SHR op, 1/CL	Shift Right	*				*	*	?	*	*
SHL op, 1/CL	Shift Left	*				*	*	?	*	*
SAR op, 1/CL	Shift Arithmetic Right	*				*	*	?	*	*
SAL op, 1/CL	Shift Arithmetic Left	*				*	*	?	*	*
NOP	NO oPeration									
WAIT	várakozás megszakításig									
HLT	leállítás									
IN AL/AX, port	AL/AX ← a port-ról									
OUT port, AL/AX	port ← AL/AX-ből									
INT i	i. megszakítás (0 ≤ i ≤ 255)		0	0						
INT 3	egy byte-os utasítás		0	0						
INTO	4. megszakítás, ha O=1		*	*						
IRET	visszatérés megsz.-ből	*	*	*	*	*	*	*	*	*
LOCK	prefixum									
Formátum	Megjegyzés	O	D	I	T	S	Z	A	P	C

Egyetlen flag-et módosítanak az alábbi utasítások:

A STATUS bitjeinek módosítása			
flag	Clear	SeT	CoMplement
C	CLC	STC	CMC
D	CLD	STD	
I	CLI	STI	

Nem módosítanak flag-et a feltételes vezérlés átadó utasítások.

Feltételes ugrások, aritmetikai csoport		
Előjeles	Reláció	Előjel nélküli
JZ $\equiv$ JE	=	JZ $\equiv$ JE
JNZ $\equiv$ JNE	$\neq$	JNZ $\equiv$ JNE
JG $\equiv$ JNLE	>	JA $\equiv$ JNBE
JGE $\equiv$ JNL	$\geq$	JA $\equiv$ JNB $\equiv$ JNC
JL $\equiv$ JNGE	<	JB $\equiv$ JNAE $\equiv$ JC
JLE $\equiv$ JGE	$\leq$	JBE $\equiv$ JNA

Feltételes ugrások, logikai csoport		
a flag igaz (1)	flag	a flag hamis (0)
JZ $\equiv$ JE	Zero	JNZ $\equiv$ JNE
JC	Carry	JNC
JS	Sign	JNS
JO	Overflow	JNO
JP $\equiv$ JPE	Parity	JNP $\equiv$ JPO
JCXZ	CX = 0	

Felhasznált irodalom

MACRO ASSEMBLER for the MS-DOS Operating system 5.0
Programmer's Guide
Microsoft

Barry B. Brey
The Intel microprocessors 8086/8088, 80186, 80286, 80386, and 80486
Architecture, Programming, and Interfacing
Prentice-Hall International, Inc, 1994

Barry B. Brey
8086/8088, 80286, 80386, and 80486 Assembly Language Programming
Macmillan Publishing Company, 1994

Kiss Zoltán - Tóth Bertalan
IBM PC assembly összefoglaló és példatár, 1991

Pethő Ádám
IBM PC/XT assembly alapismeretek
SZÁMALK, 1987

Andrew S. Tanenbaum
Structured Computer Organization
Prentice-Hall International, Inc, 1994

TÁRGYMUTATÓ

- (kifejezés) · 68

!

! (makró) · 74, 75, 80

#

⇐ (értékadás) · 20

\$ (helyszámláló) · 64

* (kifejezés) · 67

& (makró) · 74, 75, 80

[] (DEBUG) · 115

<> (kezdőérték) · 60, 62

[] (kifejezés) · 66

<> (makró) · 74, 75, 80

[op] · 20

%

% (makró) · 74, 75, 80

%OUT · 89

(

() (kifejezés) · 66

,

, (elválasztó jel) · 20, 75

.

. (kifejezés) · 66

. (struktúra) · 60

.CREF · 90

.LALL · 90

.LFCOND · 90

.LIST · 89

.RADIX · 63, 89

.SALL · 90

.SFCOND · 90

.TFCOND · 90

.XALL · 90

.XCREF · 90

.XLIST · 90

/

/ (kifejezés) · 67

:

: (címke) · 64

: (explicit szegmens megadás) · 66

: (rekord) · 61

;

;; (makró) · 74, 76, 80

?

? (kezdőérték) · 57

??xxxx (LOCAL) · 76

+

+ (kifejezés) · 68

=

= (konstans definíció) · 64, 93, 97

= (rekord) · 61

2

2-es komplement kód · 11

A

A parancs (DEBUG) · 115

A, Auxiliary Carry (flag) · 13, 22

AAA · 22

AAD · 25

AAM · 24

AAS · 23

Accumulator · 13

adat definíciós utasítások · 57

ADC · 21, 117

ADD · 21, 117, 118

AH · 13

aktuális paraméter · 71

AL · 13

align_type · 84

AND (kifejezés) · 68

AND (utasítás) · 27, 48

argumentum · 71

assembler · 92

assembly · 7

ASSUME · 8, 14, 85

AT (kombinációs típus) · 85

automatikus szegmens regiszter · 14, 16

Auxiliary Carry (flag), A · 13, 22

AX · 13

B

B, bináris · 63

Base Pointer · 12

Base Register · 13

bázis relatív címzés · 15

BH · 13

binding · 106

bit forgató (rotate) utasítások · 49

bit léptető (shift) utasítások · 49

bitenkénti logikai műveletek · 48

BL · 13

blokk ismétlés · 70, 81

BP · 12, 16

break point · 54, 115

BX · 13

byte · 10, 11

BYTE · 67, 84

C

C, Carry (flag) · 13, 21

CALL · 16, 29

Carry (flag), C · 13, 49

CBW · 24

cél terület · 40

CH · 13

cím hozzárendelés · 106

címke · 64

címzési mód byte · 17

CL · 13, 49

CLC · 50, 119

CLD · 50, 119

CLI · 50, 119

CMC · 50, 119

CMP · 23, 117

CMPS · 41, 118

CMPSB · 41

CMPSW · 41

CODE · 84

Code Segment Register · 12

combine_type · 84

COMMENT · 89

COMMON · 84

CONSTANT · 84

Counter Register · 13

cross-reference · 90

CWD · 24

CX · 13, 42

CS · 12, 66

csatoló · 108

D

D, decimális · 63
D, Direction (flag) · 13, 41
DAA · 22
DAS · 23
DATA · 84
Data Register · 13
Data Segment Register · 12
DB · 58
DD · 58
debug · 54
DEBUG · 114
DEC · 23, 117
Define Byte · 58
Define Double · 58
Define Quadword · 58
Define Ten Bytes · 58
Define Word · 58
destination · 40
Destination Index · 12
DH · 13
DI · 12, 66
Direction (flag), D · 13, 41
direkt memória címezés · 14
direkt utasítás címezés · 16
displacement · 14, 15
DIV · 24, 117
DL · 13
double word · 11
DQ · 58
DS · 12, 16, 66
DT · 58
DUP · 58
DW · 58
DWORD · 67
DX · 13

E

EA, Effective Address · 11, 86
eljárás · 29
eljárás deklaráció · 26, 29
eljárás hívás · 16, 26
előre hivatkozás · 92
ELSE · 69, 72
eltolás, displacement · 14, 15
END · 90
ENDIF · 69, 72

ENDS · 59
EQ (kifejezés) · 68
equ · 93
EQU · 64
erőforrás · 54
érték (kifejezés) · 65
ES · 12, 66
explicit szegmens megadás · 17
Extra Segment Register · 12
EXTRN · 87, 104, 105

F

F (flag jelzés) · 13
FAR · 8, 16, 29, 64, 65, 67
feltételes fordítás · 69
feltételes ugrás · 37
feltételes vezérlés átadás · 37
feltétlen ugrás · 37
feltétlen vezérlés átadás · 37
field (rekord) · 61
field (struktúra) · 59
fizikai cím · 11
FLAGS · 12
formális paraméter · 71
forrás terület · 40

G

G parancs (DEBUG) · 115
GE (kifejezés) · 68
gépi kódú programozás · 106
globális szimbólum · 86
globális szimbólum tábla · 103
GROUP · 85, 103
GT (kifejezés) · 68

H

H, hexadecimális · 63
hangolás · 111, 112
helyszámláló · 64, 93
HIGH · 67
HLT · 50

I

I, Interrupt (flag) · 13
 I/O · 51
 IDIV · 25, 117
 időosztásos rendszer · 106, 107
 IF · 69
 IF1 · 70
 IF2 · 70
 IFB · 70, 72
 IFDEF · 70
 IFDIF · 70
 IFE · 69
 IFIDN · 70
 IFNB · 70
 IFNDEF · 70, 88
 illesztés típusa · 84
 immediate · 14
 IMUL · 24, 117
 INC · 22, 117
 INCLUDE · 87, 96
 indexelt címzés · 15
 indirekt utasítás címzés · 17
 input utasítás · 51
 Instruction Pointer · 12
 INT · 54, 118
 INT 3 · 54
 interrupt · 51
 Interrupt (flag), I · 13
 INTO · 54
 Invalid Address · 108
 IP · 12
 IP relatív címzés · 16
 IRET · 54
 IRP · 81
 IRPC · 82
 ismétlődő prefixum · 42

J

JA · 38, 119
 JAE · 38, 119
 JB · 38, 119
 JBE · 38, 119
 JC · 38, 119
 JCXZ · 26, 38, 119
 JE · 38, 119
 JG · 38, 119
 JGE · 38, 119

JL · 38, 119
 JLE · 27, 38, 119
 JMP · 37, 117
 JNA · 38, 119
 JNAE · 38, 119
 JNB · 38, 119
 JNBE · 38, 119
 JNC · 38, 119
 JNE · 38, 119
 JNGE · 38, 119
 JNL · 38, 119
 JNLE · 38, 119
 JNO · 38, 119
 JNP · 38, 119
 JNS · 38, 119
 JNZ · 38, 119
 JO · 38, 119
 JP · 38, 119
 JPE · 38, 119
 JPO · 38, 119
 JS · 38, 119
 JZ · 38, 119

K

karakter konstans · 57
 kereszt hivatkozási tábla · 90
 kezdőérték · 57, 59, 61
 kifejezés · 63, 65
 kódba épített adat · 14
 kombinációs típus · 84
 konstans · 63, 65
 közvetlen operandus · 14
 külső hivatkozások feloldása · 103

L

LABEL · 64
 LAHF · 21, 117
 LDS · 20, 117
 LE (kifejezés) · 68
 LEA · 20, 117
 LENGTH · 66
 LES · 20, 117
 lexikális elemző · 44
 Link · 103
 LINK · 113
 Linkage Editor · 103
 Linkage Loader · 103

Linkage Segment · 108
 Linker · 103
 lista vezérlési utasítások · 88
 literál · 94
 load modul · 103
 LOCAL · 76
 LOCK · 56
 LODS · 41, 118
 LODSB · 41
 LODSW · 41
 logikai cím · 11
 logikai érték · 48
 logikai utasítások · 48
 lokális adat terület · 34
 LOOP · 28, 39, 117
 LOOPE · 39, 117
 LOOPNE · 39, 117
 LOOPNZ · 39, 117
 LOOPZ · 39, 117
 LOW · 67
 LT (kifejezés) · 68

M

MACRO · 71, 101
 makró · 70, 96
 makró definíció · 70
 makró fej · 71
 makró generátor · 101
 makró helyettesítés · 71
 makró hívás · 70, 71
 makró kezdet · 71
 makró kifejtés · 71
 makró törzs · 71
 makró vége · 71
 map · 103
 MASK · 62, 66
 MASM · 113
 megszakítás · 51, 108
 megszakítási ok · 53
 mem · 20
 memória · 11
 MEMORY · 84
 mező (rekord) · 61
 mező (struktúra) · 59
 mező név (rekord) · 62
 mező név (struktúra) · 60
 mező specifikáció (rekord) · 61
 mezőre hivatkozás (rekord) · 62
 mezőre hivatkozás (struktúra) · 60

mnemonic · 7, 17, 94
 MOD · 67
 modul · 83, 94
 modul név · 89
 modulo · 67
 MOV · 20, 117
 MOVS · 41, 118
 MOVSB · 41
 MOVSW · 41
 MUL · 23, 117
 MULTICS · 111
 multiprogramozás · 52
 műveleti kód · 17

N

NAME · 89
 NE (kifejezés) · 68
 NEAR · 8, 16, 29, 64, 65, 67
 NEG · 23, 117
 negatív szám ábrázolása · 11
 NOP · 50, 118
 NOT (kifejezés) · 62, 68
 NOT (utasítás) · 48
 nyomkövető · 54

O

O, Carry (flag) · 21
 O, oktális · 63
 O, Overflow (flag) · 13
 object code · 92
 object file · 103
 OFFSET · 11, 60, 67, 86, 94
 op, op1, op2 · 20
 operációs kód · 17
 operációs kód tábla · 94
 operandus · 17
 OR (kifejezés) · 68
 OR (utasítás) · 48
 ORG · 64, 93, 96
 osztály (utasítás) · 95
 output utasítás · 51
 Overflow (flag), O · 13

P

P, Parity (flag) · 13
 PAGE (illesztés típusa) · 84
 PAGE (lista vezérlő operátor) · 89
 page tábla · 107
 PARA · 84
 paraméter (makró) · 71
 paraméter átadás · 26, 29, 31, 32
 paraméter átadás cím szerint · 32
 paraméter átadás érték szerint · 32
 Parity (flag), P · 13
 PL/1 · 61, 111
 pollozás · 52
 POP · 21, 117
 POPF · 21, 117
 port · 51
 prefixum · 17
 privilegizált állapot · 53
 PROC · 8, 64
 processzor vezérlő utasítások · 50
 program kezdő cím · 90
 protected üzemmód · 11
 pszeudo utasítások · 57
 PTR · 14, 31, 67
 PUBLIC (globális szimbólum) · 87, 104, 105
 PUBLIC (kombinációs típus) · 84
 PURGE · 80
 PUSH · 21, 117
 PUSHF · 21

Q

Q parancs (DEBUG) · 116
 QWORD · 67

R

R parancs (DEBUG) · 115
 RCL · 49
 RCR · 49
 reentrent eljárás · 35
 reg · 20
 regiszter indirekt címzés · 15
 rekord · 59, 61
 rekord definíció · 61
 rekord hívás · 62

rekurzív eljárás · 35
 relokáció · 94, 103
 relokációs flag · 93
 relokációs konstans · 104
 REP · 42
 REPE · 42
 REPNE · 42
 REPNZ · 42
 REPT · 81
 REPZ · 42
 RET · 29, 34
 ROL · 49
 ROR · 27, 49
 rotate · 49

S

S, Sign (flag) · 13
 SAHF · 21
 SAL · 49
 SAR · 49
 SBB · 23, 117
 SCAS · 42, 118
 SCASB · 42
 SCASW · 42
 SEG · 67
 SEGMENT · 83
 segment override prefix · 66
 shift · 49
 SHL (kifejezés) · 68
 SHL (utasítás) · 49
 SHORT (kifejezés) · 68
 SHORT (ugrás) · 38, 39
 SHR (kifejezés) · 68
 SHR (utasítás) · 49
 SI · 12, 66
 Sign (flag), S · 13
 SIZE · 66
 software megszakítás · 53
 source · 40
 Source Index · 12
 SP · 12
 SR · 12
 SS · 12, 16, 66
 stack · 16, 21
 STACK · 84
 STACK (kombinációs típus) · 84
 Stack Pointer · 12
 stack relatív címzés · 16
 Stack Segment Register · 12

STATUS · 12
STC · 50, 119
STD · 50, 119
STI · 50, 119
STOS · 41, 118
STOSB · 41
STOSW · 41
string kezelő utasítások · 40
STRUC · 59
struktúra · 59
struktúra definíció · 59
struktúra hívás · 60
struktúra változó · 60
SUB · 23, 117
SUBTITLE · 88
számrendszer · 63
szegmens · 83
szegmens (memória) · 11
szegmens cím · 11
szegmens csoport · 83
szegmens definíció · 94
szegmens regiszter · 11, 14, 83
szélesség (rekord) · 61
szemafor · 54
szerkesztő · 84, 103
szimbolikus konstans · 64
szimbólum · 64, 65
szimbólum tábla · 93
szöveg konstans · 57, 59, 63

T

T parancs (DEBUG) · 115
T, Trap (flag) · 13
tárgy kód · 92
TBYTE · 67
térkép · 103
TEST · 48
THIS · 67
time sharing · 106
TITLE · 88
többszöri belépést megengedő eljárás · 35
törés pont · 54
töréspont · 115
Trap (flag), T · 13

TSS/67 · 111
TYPE · 67

U

U parancs (DEBUG) · 114
ugró utasítás · 16
utasítás kód · 17

V

valós üzemmód · 11
változó · 65
változó név · 57, 64
védett (protected) üzemmód · 11
verem · 16, 21, 32
vezérlés átadó utasítás · 16, 19, 36, 65
vírus · 53
virtuális cím · 11

W

WAIT · 50
WIDTH · 66
word · 11
WORD · 67, 84

X

XCHG · 20, 21, 117
XLAT · 20, 46
XOR (kifejezés) · 68
XOR (utasítás) · 48

Z

Z, Zero (flag) · 13
Zero (flag), Z · 13